

Bachelor Thesis

im Studiengang Geoinformatik

zur Erlangung des Grades eines
Bachelor of Science (B.Sc.)

über das Thema

LLM-gestützte Generierung von Playwright-E2E-Tests aus Use Cases

Einfluss von UI-Kontext am Beispiel von WebGIS-Anwendungen

von

Nils Große Beikel

Betreuer (Universität): Dr. Christian Knoth
Betreuer (Unternehmen): Dr. Thore Fechner
Matrikelnummer : 546548
Abgabedatum : 31. August 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1. Einleitung	1
1.1. Motivation und Problemstellung	1
1.2. Ziele	1
1.3. Aufbau der Arbeit	2
2. Grundlagen	3
2.1. Large Language Models	3
2.1.1. Funktionsprinzip und Kontext als Qualitätsfaktor	3
2.1.2. Einsatz im Software Engineering	4
2.2. E2E-Testautomatisierung mit Playwright	5
2.2.1. Teststufen und Einordnung von E2E-Tests	5
2.2.2. Playwright: Selektoren, Assertions, asynchrone Uls	6
2.3. Open Pioneer Trails	7
2.3.1. Framework-Überblick und Komponentenmodell	7
2.3.2. data-testid-Konventionen und Testbarkeit	8
2.4. WebGIS-Anwendungen und Testkomplexität	9
3. Stand der Forschung	10
3.1. LLM-gestützte Testgenerierung	10
3.2. Forschungslücke und Einordnung dieser Arbeit	11
4. Methodik und Forschungskonzept	13
4.1. Überblick und Forschungsdesign	13
4.2. Use Cases: Definition und Format	15
4.3. Versuchsaufbau: Die Kontextstufen	15
4.3.1. Stufe 1: Baseline (kein UI-Kontext)	16
4.3.2. Stufe 2: Automatischer Accessibility-Snapshot	16
4.3.3. Stufe 3: Automatisch generierte UI-Map	16
4.3.4. Stufe 4: Manuell erstellte UI-Map	17
4.3.5. Bonus-Stufe 5: Self-Improvement-Loop	17
4.4. Modell und Infrastruktur	19

4.5. Prompt-Aufbau und OPT-Playwright-Skill	19
4.6. Evaluationsverfahren	20
4.6.1. Phase 1: Deterministische Ausführung	20
4.6.2. Phase 2: LLM-as-a-Judge	21
5. Implementierung der Evaluationsumgebung	23
5.1. Demo-WebGIS-Anwendung	23
5.2. Testbarkeitsmaßnahmen der Anwendung	24
5.3. Use-Case-Sammlung	25
5.4. Manuell erstellte Referenztests	26
6. Durchführung und Evaluation	29
6.1. Durchführung des Experiments	29
6.2. Ergebnisse der Ausführung (Phase 1)	30
6.3. Ergebnisse der Qualitätsbewertung (Phase 2)	32
6.3.1. Kontrolle der maschinellen Bewertung	34
6.4. Ergebnisse der Bonus-Stufe 5	35
6.5. GIS-spezifische Besonderheiten	38
7. Diskussion	41
7.1. Interpretation der Ergebnisse	41
7.2. Einordnung in den Stand der Forschung	43
7.3. Limitationen	44
7.4. Handlungsempfehlungen	44
8. Fazit und Ausblick	46
8.1. Fazit	46
8.2. Ausblick	47
Anhang A Bestandteile des Generierungsprompts	48
Anhang B Evaluierung der Phase 2	56
Anhang C Ergebnisse des Generierungslaufs mit GPT-5.4	62
Anhang D Referenzierte Dateien des Repositories	68
Literaturverzeichnis	69

Abbildungsverzeichnis

Abbildung 1: Testpyramide nach Mike Cohn	5
Abbildung 2: Open Pioneer Trails Paketökosystem	8
Abbildung 3: Ausprägungen der Variable UI-Kontext	14
Abbildung 4: Generierungs- und Evaluationspipeline	14
Abbildung 5: Self-Improvement-Loop (Stufe 5)	18
Abbildung 6: Demo-WebGIS-Anwendung	23
Abbildung 7: Ausführungskategorien je Use Case und Stufe	31
Abbildung 8: Mittlerer Judge-Score je Use Case und Dimension	34
Abbildung 9: Ausführungskategorien je Use Case der Bonus-Stufe 5	36
Abbildung 10: Konvergenz des Self-Improvement-Loops	36
Abbildung 11: Mittlerer Judge-Score je Use Case und Dimension der Bonus-Stufe 5	37
Abbildung 12: Mittlerer Judge-Score je Use Case und Dimension (GPT-5.4)	64
Abbildung 13: Loop-Konvergenz und PASS-Iteration je Use Case (GPT-5.4)	65
Abbildung 14: Judge-Score je Use Case und Dimension (GPT-5.4, Stufe 5)	65

Tabellenverzeichnis

Tabelle 1: Übersicht der Use Cases mit Komplexität und getesteter Kernfunktion . . .	26
Tabelle 2: Ausführungsergebnisse je Stufe	30
Tabelle 3: Bewertungsdimensionen je Stufe	32
Tabelle 4: Ergebnis der Kontrolle je Stufe	35
Tabelle 5: Prüfung des Kartenzustands je Stufe	39
Tabelle 6: Skalenstufen der vier Bewertungsdimensionen	59
Tabelle 7: Ergebnis der manuellen Kontrolle je Testdatei und Dimension	60
Tabelle 8: Ausführungsergebnisse je Stufe (GPT-5.4)	62
Tabelle 9: PASS-Rate je Use Case und Stufe (GPT-5.4)	63
Tabelle 10: Median und Mittelwert je Dimension und Stufe (GPT-5.4)	63
Tabelle 11: Kartenbezogene Codemuster je Stufe (GPT-5.4)	66
Tabelle 12: Referenzierte Dateien des Repositories	68

Abkürzungsverzeichnis

API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CSS	Cascading Style Sheets
DOM	Document Object Model
DWD	Deutscher Wetterdienst
E2E	End-to-End
EPSG	European Petroleum Survey Group
GIS	Geoinformationssystem
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
OPT	Open Pioneer Trails
UI	User Interface
URL	Uniform Resource Locator
WFS	Web Feature Service
WMS	Web Map Service

1. Einleitung

1.1. Motivation und Problemstellung

End-to-End (E2E)-Tests prüfen eine Anwendung aus Nutzersicht und sind daher gerade für komplexe und interaktive Oberflächen wertvoll. Ihre Erstellung ist jedoch aufwendig, da jeder Testfall die Bedienschritte und die zu prüfenden Zustände einzeln nachbilden muss. In der Praxis führt das dazu, dass E2E-Tests oft erst spät, unvollständig oder gar nicht geschrieben werden. Large Language Models (LLMs) haben in den letzten Jahren in der Softwareentwicklung stark an Bedeutung gewonnen und werden zunehmend eingesetzt, um diesen Aufwand zu verringern und Tests direkt aus einer fachlichen Beschreibung zu generieren. Für die Testerstellung ist das ein attraktiver Weg, da Use Cases oder User Storys in vielen Projekten ohnehin vorliegen und unmittelbar als Eingabe dienen können.

Ein LLM kennt die konkrete Benutzeroberfläche einer individuell entwickelten Anwendung jedoch nicht von selbst. Ohne Informationen über die tatsächlich vorhandenen Elemente und ihre Bedienung kann das Modell nur plausibel erscheinende Bezeichnungen raten. Ein generierter Test kann dann syntaktisch korrekt sein, aber trotzdem an der ersten Interaktion scheitern, weil das angesprochene Element in der Anwendung nicht existiert. Umgekehrt kann ein automatisch generierter Test auch bestehen, ohne das Richtige zu prüfen. Je nach Design kann er fehlerhaften Code fälschlich bestätigen, statt ihn aufzudecken, und so ein trügerisches Sicherheitsgefühl erzeugen (*Mathews, Nagappan, 2024, S. 5*).

Bei webbasierten Geoinformationssystem (GIS)-Anwendungen verschärft sich dieses Problem zusätzlich, da die Karte als zentraler Bestandteil als Pixelgrafik auf einem Canvas-Element gerendert wird und damit nicht wie gewöhnliche Oberflächenelemente über das Document Object Model (DOM) ansprechbar ist. Genau diese Karteninteraktionen sind aber der Hauptbestandteil von WebGIS-Anwendungen und müssten von einem aussagekräftigen Test erfasst werden.

Damit stellt sich die Frage, wie viele Informationen über die Oberfläche einem LLM zur Testgenerierung bereitgestellt werden müssen und in welcher Form diese Informationen den größten Nutzen bringen.

1.2. Ziele

Diese Arbeit untersucht, welchen Einfluss der im Prompt bereitgestellte User Interface (UI)-Kontext auf die Qualität LLM-generierter Playwright-E2E-Tests aus Use Cases für WebGIS-Anwendungen auf Basis von Open Pioneer Trails (OPT) hat. Als Qualität wird dabei sowohl die Ausführbarkeit der Tests als auch die inhaltliche Korrektheit der Prüfung verstanden.

Daraus ergeben sich drei Teilfragen:

1. Führt ein umfangreicherer UI-Kontext zu einer höheren Ausführbarkeit der generierten Tests?
2. Wirken sich unterschiedliche Formen des Kontexts auf unterschiedliche Qualitätsaspekte der Tests aus?
3. In welchem Verhältnis steht der Aufwand für die Erstellung des Kontexts zum erreichten Qualitätsgewinn?

Ergänzend dazu wird untersucht, ob sich die Grenzen eines einmalig bereitgestellten Kontexts überwinden lassen, wenn das Modell Rückmeldung aus der Ausführung seiner eigenen Tests erhält und diese schrittweise korrigiert. Diese Fragestellung geht über den Kernvergleich hinaus und wird daher als Bonus-Untersuchung behandelt. Zur Einordnung der Übertragbarkeit wird der Generierungsablauf zusätzlich mit einem zweiten LLM wiederholt. Prompting-Strategien und ein Vergleich der Modellfähigkeiten sind damit nicht Gegenstand dieser Arbeit.

1.3. Aufbau der Arbeit

Kapitel 2 erklärt zunächst die Grundlagen zu LLMs, E2E-Testautomatisierung mit Playwright und OPT und geht auf die besondere Testkomplexität von WebGIS-Anwendungen ein. Kapitel 3 ordnet die Arbeit in den Stand der Forschung ein. Die Methodik mit den untersuchten Kontextstufen und das Evaluationsverfahren beschreibt Kapitel 4. Kapitel 5 stellt die dafür implementierte Evaluationsumgebung vor, bestehend aus der Demo-WebGIS-Anwendung, ihren Testbarkeitsmaßnahmen, den Use Cases und den manuellen Referenztests. Kapitel 6 dokumentiert die Durchführung und präsentiert die Ergebnisse des Experiments. Kapitel 7 diskutiert die Ergebnisse, ihre Limitationen und die daraus folgenden Handlungsempfehlungen. Kapitel 8 schließt die Arbeit mit einem Fazit und Ausblick ab.

2. Grundlagen

2.1. Large Language Models

LLMs sind maschinelle Lernmodelle, die auf sehr großen Textmengen trainiert werden, um natürliche Sprache zu verarbeiten und zu erzeugen. Technisch handelt es sich um neuronale Netze mit typischerweise mehreren hundert Milliarden Parametern (*Zhao et al.*, 2026, S. 4). Dadurch sind sie in der Lage, komplexe Muster, sprachliche Semantik und Nuancen zu erlernen, die dann für viele verschiedene sprachbezogene Aufgaben wie Übersetzung, Textzusammenfassungen etc. verwendet werden können (*Celik, Mahmoud*, 2025, S. 2). Den Durchbruch verdanken LLMs durch die von *Vaswani et al.* (2017) eingeführte Transformer-Architektur (*Celik, Mahmoud*, 2025, S. 2).

2.1.1. Funktionsprinzip und Kontext als Qualitätsfaktor

Der zentrale Bestandteil der Transformer-Architektur ist der Self-Attention-Mechanismus. Er setzt die Elemente einer Eingabe direkt miteinander in Beziehung, unabhängig davon, wie weit sie voneinander entfernt stehen (*Vaswani et al.*, 2017, S. 6). Frühere rekurrente Modelle mussten die Eingabe dafür Schritt für Schritt durchlaufen, was eine Parallelisierung innerhalb eines Trainingsbeispiels ausschloss. Der Transformer verzichtet auf diese Rekurrenz, verarbeitet die Eingabe weitgehend parallel und verkürzt damit die Trainingszeit erheblich (*Vaswani et al.*, 2017, S. 1).

Vor der Verarbeitung wird der Text in Tokens zerlegt, also in kleinere Einheiten wie Wörter oder Teilwörter, damit sie von LLMs einfacher und besser verarbeitet werden können (*Hou et al.*, 2024, S. 18). Ein verbreitetes Verfahren zur Zerlegung in solche Teilwörter ist das Byte-Pair-Encoding (BPE), das ursprünglich für die maschinelle Übersetzung entwickelt wurde (*Sennrich, Haddow, Birch*, 2016, S. 2) und seitdem auch bei LLMs zur Tokenisierung eingesetzt wird (*Zhao et al.*, 2026, S. 19). Die Texterzeugung erfolgt autoregressiv. Bei jedem Schritt verwendet das Modell die bereits erzeugten Tokens als zusätzliche Eingabe und berechnet eine Wahrscheinlichkeitsverteilung für das nächste Token (*Vaswani et al.*, 2017, S. 2, 5). Wie stark die Auswahl vom jeweils wahrscheinlichsten Token abweichen darf, steuert unter anderem der Temperatur-Parameter. Er bestimmt den Grad an Zufälligkeit im erzeugten Text (*Plein et al.*, 2023, S. 4).

Brown et al. (2020, S. 1, 4) zeigten, dass große Sprachmodelle Aufgaben allein anhand einer Anweisung oder einzelner Beispiele im Prompt lösen können, ohne dass die Modellgewichte angepasst werden, und bezeichnen dies als „in-context-learning“. Ob das Modell dabei im eigentlichen Sinne lernt oder lediglich im Training gesehene Muster wiedererkennt, lassen die

Autoren offen (*Brown et al.*, 2020, S. 34). Für diese Arbeit genügt die Beobachtung, dass im Prompt bereitgestellte Informationen die Ausgabe unmittelbar bestimmen.

Daraus folgt ein zentraler Punkt für diese Arbeit. Ein LLM hat kein konkretes Wissen über eine individuell entwickelte Benutzeroberfläche. Informationen über die tatsächlich vorhandenen UI-Elemente, deren Selektoren und Zustände stehen weder in den Trainingsdaten noch sind sie zur Laufzeit zugänglich, solange sie nicht explizit im Prompt bereitgestellt werden. Für die Testgenerierung folgt daraus, dass Qualität und Umfang des bereitgestellten Kontexts den generierten Testcode maßgeblich mitbestimmen dürften. Genau dieser Zusammenhang wird in dieser Arbeit untersucht

2.1.2. Einsatz im Software Engineering

LLMs werden zunehmend im Software Engineering eingesetzt, insbesondere zur Generierung von Quellcode. Sichtbar wird das an Werkzeugen wie GitHub Copilot, die auf Grundlage des umgebenden Quellcode-Kontexts Vorschläge zur Codevervollständigung erzeugen (*Hou et al.*, 2024, S. 30). Der systematische Literaturüberblick von *Hou et al.* (2024) beschreibt 85 konkrete Aufgaben aus sechs Kernaktivitäten des Software Engineering, von der Anforderungsanalyse bis zur Wartung, und belegt damit, dass das Feld deutlich über die Codevervollständigung hinausreicht (*Hou et al.*, 2024, S. 4).

Ein Anwendungsfeld ist die automatisierte Testgenerierung, in der aus natürlichsprachlichen oder strukturierten Beschreibungen Tests auf unterschiedlichen Teststufen erzeugt werden (*Celik, Mahmoud*, 2025; *Ferreira et al.*, 2025; *Alian et al.*, 2024). Diesen Möglichkeiten stehen bekannte Grenzen gegenüber. Da LLMs ihre Ausgabe auf Basis von Wahrscheinlichkeiten erzeugen, können sie plausibel wirkende, faktisch aber nicht existierende Inhalte produzieren. *Celik & Mahmoud* (2025) fassen zusammen, dass sich über die von ihnen betrachteten Studien zum Prompt Engineering hinweg wiederkehrende Probleme zeigen. Dazu gehören halluzinierter Code, Kompilierfehler und eine eingeschränkte Übertragbarkeit auf reale Umgebungen, besonders dort, wo der Projektkontext dünn oder unvollständig ist (*Celik, Mahmoud*, 2025, S. 10).

Grundsätzlich lassen sich Sprachmodelle auf zwei Wegen an eine spezifische Aufgabe anpassen. Beim Fine-Tuning werden die Gewichte eines vortrainierten Modells durch Training auf einen aufgabenspezifischen, gelabelten Datensatz angepasst, wofür typischerweise mehrere Tausend bis mehrere Hunderttausend Beispiele nötig sind (*Brown et al.*, 2020, S. 6). Beim Prompting wird das Verhalten allein über die Gestaltung der Eingabe gesteuert. Es beruht auf dem in Abschnitt 2.1.1 beschriebenen In-Context-Learning, das in dieser Arbeit verfolgt wird. Nachteile des Fine-Tuning sind der Bedarf an einem großen Datensatz für jede Aufgabe und eine schlechte Übertragbarkeit außerhalb der Trainingsdaten (*Brown et al.*, 2020, S. 6). Ein

prompt-basierter Workflow lässt sich jedoch ohne erneutes Training auf andere Anwendungen und Use Cases übertragen, während ein feinjustiertes Modell auf seinen Trainingskontext beschränkt bliebe.

2.2. E2E-Testautomatisierung mit Playwright

2.2.1. Teststufen und Einordnung von E2E-Tests

Softwaretests lassen sich nach ihrem Umfang und ihrer Nähe zum Nutzerverhalten in verschiedene Stufen einteilen. Eine verbreitete Einteilung von Softwaretests ist die Testpyramide, die von *Poenisch* (2022) beschrieben wird. Sie unterscheidet drei Ebenen, wie in Abbildung 1 dargestellt. Ganz unten stehen Unit-Tests, die einzelne Funktionen oder Komponenten prüfen. Sie laufen schnell und werden in großer Anzahl geschrieben. Darüber liegen die Integrationstests, die das Zusammenspiel mehrerer Komponenten betrachten. An der Spitze stehen Systemtests, die das Verhalten des gesamten Systems betrachten. Zu ihnen gehören UI-Tests und E2E-Tests. Da dabei ausschließlich das äußere Verhalten der Anwendung geprüft wird, handelt es sich um Black-Box-Tests, welche die Anwendung als Ganzes aus Sicht der Nutzer betrachten.

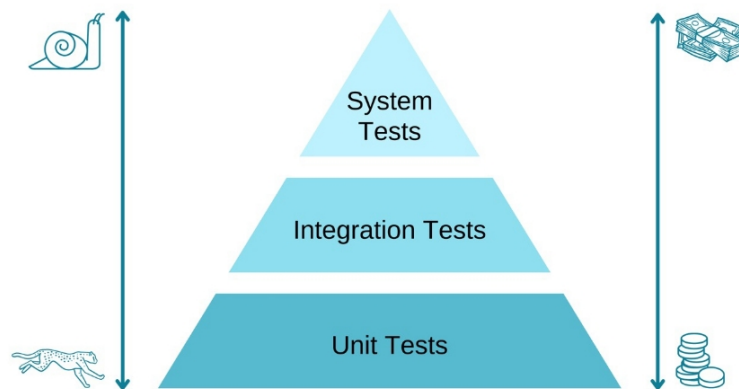


Abbildung 1: Testpyramide nach Mike Cohn mit den drei Teststufen Unit-, Integrations- und Systemtests. Die Breite einer Ebene steht für die empfohlene Anzahl an Tests: viele schnelle Unit-Tests an der Basis, wenige langsame Systemtests an der Spitze. Die in dieser Arbeit generierten E2E-Tests gehören zur obersten Ebene. (Quelle: *Poenisch*, 2022)

Ein E2E-Test steuert dazu einen Browser und führt dieselben Schritte aus wie ein Mensch. Dazu gehören das Klicken auf Schaltflächen, das Ausfüllen von Feldern und das Navigieren durch die Oberfläche. Geprüft wird, ob am Ende das erwartete Ergebnis eintritt. Dadurch geben E2E-Tests eine gute Auskunft darüber, ob eine fachliche Anforderung tatsächlich umgesetzt ist.

Allerdings sind sie langsam, aufwendig zu erstellen und vergleichsweise anfällig und können schon bei kleinen Änderungen an der UI fehlschlagen (Fowler, 2012). Deshalb wird empfohlen, ihre Anzahl, entsprechend der Form der Pyramide, gering zu halten (Poenisch, 2022).

Für WebGIS-Anwendungen sind E2E-Tests besonders relevant. Viele Probleme zeigen sich erst im Zusammenspiel der Komponenten und Interaktionen. Dazu gehört das asynchrone Laden von Geodaten, das Rendern der Karte oder die Auswirkungen der Layersteuerung auf die Kartenanzeige. Solche Abläufe lassen sich mit Unit-Tests kaum abbilden, weil sie den vollständigen, im Browser laufenden Zustand voraussetzen.

2.2.2. Playwright: Selektoren, Assertions, asynchrone Uls

Playwright ist ein Open-Source-E2E-Test-Framework von Microsoft zur Automatisierung von Browsern. Es steuert die Browser-Engines Chromium, Firefox und WebKit über eine einheitliche Programmierschnittstelle und führt Tests in einem echten Browser aus (Microsoft, o. J., Playwright). Die Tests können in JavaScript, TypeScript, Python, Java und .NET geschrieben werden (Microsoft, o. J., Supported languages).

Um mit der Benutzeroberfläche zu interagieren, muss ein Test die jeweiligen Elemente finden. Playwright nutzt dafür sogenannte Lokatoren. Es wird empfohlen, die Elemente über nutzerseitige Eigenschaften, wie über ihre Accessible Rich Internet Applications (ARIA)-Rolle oder ihren sichtbaren Text (`getByText`, `getByRole`), anzusprechen. Solche Selektoren sind robuster gegenüber Änderungen am Markup als Cascading Style Sheets (CSS)- oder XPath-Ausdrücke, die an die konkrete DOM-Struktur gebunden sind. Playwright kann diese Informationen auch gebündelt aus einem Accessibility-Snapshot auslesen. Er gibt Rollen, Attribute, Werte und Textinhalt der Elemente einer Seite als Baumstruktur aus, damit man eine Übersicht erhält, welche Elemente alles vorhanden sind (Microsoft, o. J., Snapshot testing). Eine Alternative sind Test-IDs. Über das Attribut `data-testid` lässt sich ein Element gezielt für Tests markieren und mit `getByTestId` ansprechen, unabhängig von Darstellung und Position. Welches Attribut als Test-ID gilt, ist konfigurierbar (Microsoft, o. J., Locators).

Ein wesentliches Merkmal von Playwright ist das automatische Warten. Vor jeder Aktion prüft das Framework, ob das Zielelement sichtbar, stabil und bedienbar ist, ansonsten wird bis zu einem Timeout gewartet (Microsoft, o. J., Auto-waiting). Dadurch müssen nicht für jede Interaktion manuelle Wartezeiten eingebaut werden, was eine häufige Ursache für instabile Tests beseitigt. Dasselbe Prinzip gilt für Assertions. Mit `expect` formulierte Zusicherungen wie `expect(locator).toBeVisible()` oder `toHaveText()` werden so lange wiederholt, bis die Bedingung erfüllt ist oder die Zeit abläuft (Microsoft, o. J., Assertions). Für Werte, die nicht von einem Lokator stammen, sondern z. B. von einer eigenen Funktion zurückgegeben werden, lässt sich mit `expect.poll` auch wiederholt prüfen (Microsoft, o.

J., expect.poll). Eine Assertion prüft damit nicht einen Momentzustand, sondern einen Zustand, der sich auch verzögert einstellen darf.

Für Fälle, die das automatische Warten nicht abdeckt, stellt Playwright extra Wartemethoden bereit, etwa auf eine bestimmte Netzwerkantwort (`waitForResponse`) oder einen Ladezustand der Seite (`waitForLoadState`) (*Microsoft, o. J., Page*). Das ist gerade bei WebGIS-Anwendungen wichtig, in denen Kartenkacheln und Geodaten asynchron über das Netz geladen werden und erst danach sichtbar auftauchen. Ein Test, der zu früh prüft, würde fehlschlagen, obwohl die Anwendung korrekt arbeitet. Gerade beim asynchronen Laden von Kartenkacheln und Geodaten greift damit beides ineinander: das automatische Warten für DOM-Elemente und gezielte Wartepunkte auf Netzwerkebene.

Neben der Interaktion mit dem DOM kann Playwright mit `page.evaluate()` auch beliebigen JavaScript-Code im Browser-Kontext ausführen und dabei auf globale Objekte der Seite zugreifen (*Microsoft, o. J., Evaluating JavaScript*). Das ermöglicht Zugriffe, die über die reine DOM-Interaktion hinaus gehen.

2.3. Open Pioneer Trails

2.3.1. Framework-Überblick und Komponentenmodell

Open Pioneer Trails ist ein Open-Source-Framework zur Entwicklung webbasierter GeoIT-Anwendungen. Es ist als gemeinsames Projekt der con terra GmbH und 52°North Spatial Information Research GmbH im Rahmen der Open-Pioneer-Initiative entstanden und steht unter der Apache-2.0-Lizenz frei auf GitHub zur Verfügung. Trails ist ein Entwicklungsframework und kein fertiges Produkt. Anwendungen werden also nicht konfiguriert, sondern programmiert, wobei das Framework wiederverwendbare Komponenten und Vorlagen bereitstellt (*52°North GmbH, Matthes Rieke Managing Director, 2026*).

Technisch besteht Trails aus modernen Webtechnologien. Die Anwendungen werden in TypeScript mit React geschrieben, das Styling basiert auf Chakra UI, als Kartenkomponente wird OpenLayers verwendet, die Paketverwaltung läuft über pnpm (Performant Node Package Manager) und als Build-System wird Vite verwendet (*Open Pioneer, 2023c, trails-starter*).

Eine Open Pioneer Trails-Anwendung ist auf Paketen aufgebaut. Ein Paket bündelt React-Komponenten, Services, Übersetzungen und Styling. Die Anwendung selbst ist ebenfalls ein Paket und wird zu einer Web Component gebaut, dadurch kann sie einfach in bestehende Web-Anwendungen integriert werden. Aktuell werden zwei Gruppen wiederverwendbarer Pakete bereitgestellt. Die Core Packages enthalten kartenlose Basisfunktionen wie Authentifizierung, Notifier, Internationalisierungsfunktionen und einen Hypertext Transfer

Protocol (HTTP)-Service (*Open Pioneer*, 2023a, trails-core-packages). Die OpenLayers Base Packages bauen darauf auf und liefern die Kartenkomponenten, darunter Layerübersicht, Kartennavigations-Werkzeuge, Legende, Selektion und Ergebnisliste, Suche, Messwerkzeug, Koordinatenanzeige und Kartendruck (*Open Pioneer*, 2023b, trails-openlayers-base-packages). Abbildung 2 gibt einen Überblick über das Paketökosystem.

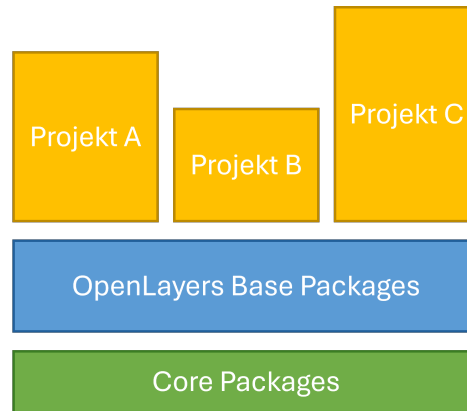


Abbildung 2: Paketökosystem von Open Pioneer Trails. Die Core Packages stellen kartenlose Basisfunktionen bereit, die OpenLayers Base Packages bauen darauf auf und liefern die Kartenkomponenten. Projektspezifische Anwendungen nutzen beide Ebenen in unterschiedlichem Umfang.

Die Anbindung der Karte erfolgt nicht direkt über OpenLayers, sondern über eine eigene Schnittstelle. Diese erweitert das Datenmodell, vereinfacht den Umgang mit dem Kartenzustand und stellt ihn als TypeScript-Schnittstelle (MapModel) bereit. Der direkte Zugriff auf die OpenLayers-Application Programming Interface (API) bleibt bei Bedarf möglich (*52°North GmbH, Arne Vogt*, 2025).

Trails richtet sich explizit an Entwickler und erfordert Programmierkenntnisse. Dafür lässt es Layout und Funktionalität frei gestalten und bringt typische GIS-Widgets als fertige Komponenten mit.

2.3.2. data-testid-Konventionen und Testbarkeit

Trails reicht `data-testid` über `CommonComponentProps` (*Open Pioneer*, o. J., `react-utils`) bzw. den Chakra-Prop-Filter (*Chakra Systems*, 2026) ans DOM weiter.

Neben den Test-IDs sind die Komponenten auf Barrierefreiheit ausgelegt und folgen ARIA-Konventionen (*52°North GmbH, Arne Vogt*, 2025). Elemente lassen sich dadurch beispielsweise über ihre Rolle oder ihren sichtbaren Textinhalt finden. Playwright kann anhand der Rolle und Bezeichnung (`getByRole/getByText`) auf diese Elemente zugreifen. Damit bietet Trails zwei robuste Wege, ein Element zu adressieren.

2.4. WebGIS-Anwendungen und Testkomplexität

WebGIS-Anwendungen stellen die Testautomatisierung vor Herausforderungen, die generische Webanwendungen in dieser Form nicht haben. Der Grund dafür ist, wie Karten und Geodaten im Browser dargestellt und geladen werden. Ein zentrales Problem ist die Darstellung der Karte selbst. OpenLayers rendert die Karte seit Version 3.18.0 standardmäßig über den Canvas-Renderer (*OpenLayers*, 2026a, S. v3.18.0) und zeichnet die Karteninhalte damit als Pixelgrafik statt als einzelne DOM-Elemente. Ein Lokator kann zwar das Canvas-Element finden, aber nicht die Inhalte darin, beispielsweise ob ein bestimmter Layer gerade sichtbar ist. Was in der Karte dargestellt wird, ist also nicht direkt über das DOM zugänglich. Dieses Problem ist nicht auf WebGIS-Anwendungen beschränkt, sondern gilt allgemein für Canvas-Elemente. Sie besitzen keinen repräsentativen DOM-Baum für ihre innere Struktur, weshalb klassische, DOM-basierte Verfahren darauf nicht anwendbar sind (*Bajammal, Mesbah*, 2018, S. 1).

Hinzu kommt, dass Geodaten asynchron über das Netz geladen werden. Kartenkacheln, Web Map Service (WMS)-Layer, GetFeatureInfo-Abfragen oder Anfragen des Geocoders treffen mit unterschiedlicher Verzögerung ein. Ein Test muss abwarten, bis die Daten geladen und gerendert wurden, bevor getestet werden kann. Das in Abschnitt 2.2.2 beschriebene automatische Warten von Playwright hilft hier nur bedingt, da es sich auf die Bedienbarkeit von DOM-Elementen bezieht. Ob Kartenkacheln vollständig dargestellt werden, lässt sich darüber nicht erkennen, sodass auf andere Methoden wie das Abwarten einer Netzwerkantwort zurückgegriffen werden muss.

Die Oberfläche einer WebGIS-Anwendung ist außerdem stark vom aktuellen Zustand abhängig. Ein Layer kann sichtbar oder unsichtbar sein, die Legende bezieht sich auf sichtbare Layer und eine Informationsanzeige erscheint erst, nachdem ein Feature in der Karte angeklickt wurde. Auch hier ist das Verhalten von Interaktionen wichtig. Die Informationsanzeige bei Klick auf ein Feature kann zum Beispiel deaktiviert sein, wenn gerade eine Messfunktion zum Messen einer Linie aktiv ist. Dieselbe Interaktion kann je nach Zustand zu einem anderen Ergebnis führen, weshalb die Vorbedingungen eines Tests genau definiert sein müssen.

Außerdem sind viele Interaktionen abhängig von Koordinaten. Angenommen, der Nutzer klickt auf eine Position, auf eine Markierung oder zeichnet durch mehrere Klicks eine Linie. Der Test muss dafür auf Pixelkoordinaten im Canvas klicken statt auf ein bekanntes Element. Das Ergebnis hängt dann von Kartenausschnitt, Zoomstufe und Projektion ab.

3. Stand der Forschung

3.1. LLM-gestützte Testgenerierung

Die automatisierte Testgenerierung mit LLMs ist eines der Anwendungsfelder des in Kapitel 2.1.2 beschriebenen Einsatzes von LLMs im Software Engineering. *Celik & Mahmoud* (2025, S. 7f) fassen die bestehenden Ansätze in einer Literaturübersicht zusammen und unterscheiden zwischen vier methodischen Kategorien: Prompt Engineering, feedback-getriebene Verfahren, Fine-Tuning sowie hybride Kombinationen aus LLMs und traditionellen Testwerkzeugen. Diese Arbeit gehört zur ersten Kategorie, da sie das Modellverhalten über die Eingabe steuert. Die ausgewerteten Studien betreffen überwiegend Java und Python, weil Testwerkzeuge und Benchmarks vor allem für diese Sprachen verfügbar sind (*Celik, Mahmoud*, 2025, S. 21).

Dass eine natürlichsprachliche Beschreibung als fachliche Eingabe ausreichen kann, zeigen *Plein et al.* (2023) in einer Machbarkeitsstudie. Sie lassen ChatGPT aus natürlichsprachlichen Bug Reports ausführbare Testfälle erzeugen und bewerten das Ergebnis anhand der Ausführbarkeit, Validität und Relevanz. Bei fünf Generierungsversuchen je Bug Report entsteht für 50 % der Eingaben ein ausführbarer Test, für etwa 30 % ein Test, der den berichteten Fehler tatsächlich reproduziert, und für etwa 9 % ein Test, der zusätzlich die korrigierte Programmversion passiert (*Plein et al.*, 2023, S. 5). Die häufigsten Ursachen der nicht ausführbaren Ausgaben sind fehlende Imports, doppelte Methodennamen und veraltete Funktionsaufrufe (*Plein et al.*, 2023, S. 10). Für die Fragestellung dieser Arbeit ist die Studie allerdings nur bedingt aussagekräftig, da sie Java-Projekte betrachtet und die generierten JUnit-Testfälle an die bestehende Testsuite des jeweiligen Projektes anhängt. Eine Benutzeroberfläche kommt nicht vor, weshalb sich die Ergebnisse nur bedingt auf E2E-Tests übertragen lassen.

Für den E2E-Testbereich liegt eine industrielle Fallstudie von *Ferreira et al.* (2025) vor. Bei dem dort vorgestellten Verfahren erzeugt das Werkzeug AutoUAT mit GPT-4 Turbo zunächst aus User Stories Testszenarien im Gherkin-Format. Das zweite Werkzeug, Test-Flow, generiert daraus anschließend ausführbare Tests für das Testframework Cypress. Als Kontext erhält das Modell neben der User Story auch den Hypertext Markup Language (HTML)-Code der betroffenen Seiten, aus dem Style- und Skript-Elemente vorab entfernt werden (*Ferreira et al.*, 2025, S. 2f).

Von 65 abgegebenen Nutzerrückmeldungen zu AutoUAT bewerteten 95 % das generierte Szenario als hilfreich (*Ferreira et al.*, 2025, S. 4). Test-Flow wurde an 13 User Stories mit insgesamt 50 Gherkin-Szenarien gemessen. Von den 50 generierten Testfällen waren 60 % unverändert einsetzbar und 8 % nach kleinen Korrekturen. 24 % ließen sich erst verwenden,

nachdem die Eingabe um den fehlenden Kontext ergänzt wurde, und 8 % mussten wegen schwerer Fehler verworfen werden (*Ferreira et al.*, 2025, S. 6). Fehlender Kontext war mit 12 von 20 fehlerhaften Testfällen die häufigste Fehlerursache (*Ferreira et al.*, 2025, S. 5). Die verwendeten User Stories betrafen fünf Seiten eines einzelnen E-Commerce-Produkts (*Ferreira et al.*, 2025, S. 5).

Einen anderen Weg zur Kontextgewinnung beschreiben *Alian et al.* (2024) mit AutoE2E. Das Verfahren leitet die Features einer Webanwendung aus deren beobachtetem Zustand ab, statt sie einer Anforderungsbeschreibung zu entnehmen. Eine Breitensuche crawlt den Zustandsraum einer Anwendung, bis keine neuen Zustände mehr gefunden werden, und ein LLM erzeugt daraus ausführbare E2E-Testfälle (*Alian et al.*, 2024, S. 6). Auf dem von den Autoren erstellten Benchmark E2EBench erreicht AutoE2E eine durchschnittliche Feature-Abdeckung von 79 %, gegenüber 12 % beim klassischen Verfahren mit Crawljax (*Alian et al.*, 2024, S. 9). Nach deren Angabe ist AutoE2E das erste Verfahren, das feature-getriebene E2E-Tests autonom generiert (*Alian et al.*, 2024, S. 1).

Auf Ebene von Unit-Tests zeigen *Schäfer et al.* (2023) mit TestPilot einen weiteren Weg der Kontextgewinnung. Der Prompt für GPT-3.5-turbo kombiniert Signatur und Implementierung der zu testenden Funktion mit aus der Projektdokumentation extrahierten Anwendungsbeispielen. Die damit erzeugten Tests erreichen für JavaScript-Bibliotheken eine mediane Statement-Coverage von 70,2 %, während das Vergleichsverfahren Nessie nur 51,3 % erreicht (*Schäfer et al.*, 2023, S. 1). Schlägt ein generierter Test fehl, wird das Modell erneut mit dem fehlgeschlagenen Test und der zugehörigen Fehlermeldung angesprochen, um eine korrigierte Fassung zu erzeugen. Auf diese Weise werden im Durchschnitt 15,6 % der zunächst fehlgeschlagenen Tests erfolgreich behoben (*Schäfer et al.*, 2023, S. 2).

Gemeinsam zeigen die betrachteten Arbeiten, dass sich mit reinem Prompting ausführbare Tests erzeugen lassen (*Plein et al.*, 2023; *Ferreira et al.*, 2025; *Alian et al.*, 2024; *Schäfer et al.*, 2023). Alle Ansätze nutzen dabei jeweils eine etwas andere Form von Anwendungs- oder Eingabe-Kontext. Allerdings untersucht keine der Arbeiten, wie sich Form und Umfang des bereitgestellten UI-Kontexts auf die Qualität der generierten Tests auswirken.

3.2. Forschungslücke und Einordnung dieser Arbeit

Ein Bezug zu WebGIS-Anwendungen ist in keiner der genannten Arbeiten vorhanden. Nach eigener Recherche liegen keine wissenschaftlichen Arbeiten zur LLM-gestützten Testgenerierung für WebGIS-Anwendungen mit Karteninteraktionen vor. Auch für OPT als Framework existieren keine Untersuchungen. Die in Kapitel 3.1 betrachteten Ansätze zur Testgenerierung für Webanwendungen setzen voraus, dass die zu prüfenden Elemente über das DOM erreichbar und ansprechbar sind. Für die Karte gilt diese Annahme nicht (vgl.

Kapitel 2.4), sodass offen ist, ob sich ihre Ergebnisse auf diesen Anwendungsfall übertragen lassen. Der Einfluss des UI-Kontexts wiederum ist durch das In-Context Learning erklärbar (vgl. Kapitel 2.1.1), wurde in der Literatur bisher aber nur als Nebenbefund berichtet und nicht kontrolliert untersucht.

Diese Arbeit setzt an beiden Lücken an. Sie knüpft an den Workflow von der Anforderung zum Test an (*Plein et al., 2023; Ferreira et al., 2025*) und grenzt sich von editorbasierten Werkzeugen ab, die aus dem umgebenden Quellcode generieren. Ihr Beitrag besteht darin, den UI-Kontext als kontrollierte Variable zu untersuchen und erstmals auf WebGIS-Anwendungen einschließlich Karteninteraktionen zu beziehen.

4. Methodik und Forschungskonzept

4.1. Überblick und Forschungsdesign

Im Folgenden werden die Methodik und das Forschungskonzept zur Beantwortung der Forschungsfrage dargestellt, welchen Einfluss der UI-Kontext auf die Qualität LLM-generierter Playwright-E2E-Tests für OPT-basierte WebGIS-Anwendungen hat. Die in der Einleitung formulierten Teilfragen werden dabei in das Untersuchungsdesign einbezogen.

Der UI-Kontext ist die einzige unabhängige Variable. Alle anderen Faktoren wie Use Cases, LLM und dessen Konfiguration und OPT-Playwright-Skill werden konstant gehalten. Unter diesen Bedingungen werden die Qualitätsunterschiede der generierten Tests dem Kontext zurechenbar und werden nicht durch andere Einflüsse verfälscht. Dieses Vorgehen knüpft an das in Kapitel 2.1.1 beschriebene In-Context Learning an, bei dem der Kontext im Prompt die Ausgabe maßgeblich bestimmt.

Was dem LLM im Prompt mitgeteilt wird, beschreibt der UI-Kontext. Davon zu unterscheiden sind die technischen Eigenschaften, die die Anwendung unabhängig vom Prompt bereitstellt, wie die Vergabe von data-testid-Attributen und der Zugriff auf das Kartenmodell über `__openPioneerMap` (siehe Kapitel 5.2). Diese Möglichkeiten bestehen bereits ab Stufe 1 und bleiben über alle vier Stufen unverändert. Variabel ist nur, ob und in welcher Form dem LLM im Prompt mitgeteilt wird, dass sie existieren.

Generiert werden die Tests aus den fachlichen Anforderungen des jeweiligen Use Case und nicht aus dem Quellcode der Anwendung. Die Struktur des Prompts wird dabei über alle vier Stufen konstant gehalten. Es ändert sich nur der enthaltene UI-Kontext. Verschiedene Prompting-Strategien sind nicht Teil dieser Untersuchung. Da die Generierung nicht deterministisch erfolgt, wird jeder Use Case pro Stufe mehrfach generiert (N=50). Verglichen werden dadurch Verteilungen der Testqualität je nach Stufe, nicht einzelne Ausgaben.

Technische Details zur Erstellung der Tests werden in einem OPT-Playwright-Skill festgehalten, der in Kapitel 4.5 beschrieben wird.

Die vier Stufen unterscheiden sich im Umfang und in der Strukturierung des bereitgestellten UI-Kontexts. Stufe 1 gibt dem LLM keine Informationen über die Oberfläche, während die höheren Stufen den Kontext schrittweise erweitern und strukturieren. Abbildung 3 stellt die vier Ausprägungen dar.

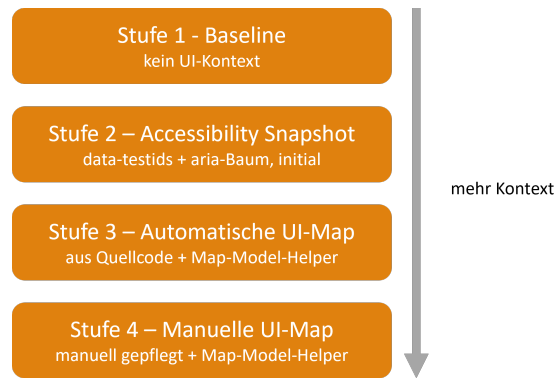


Abbildung 3: Die vier Ausprägungen der unabhängigen Variable UI-Kontext mit steigendem Informationsgehalt von Stufe 1 bis 4.

Wie ein einzelner Generierungsvorgang abläuft, zeigt Abbildung 4. Use Case, OPT-Playwright-Skill und der UI-Kontext der jeweiligen Stufe bilden zusammen den Prompt für das LLM. Die daraus erzeugten Playwright-Tests durchlaufen ein zweiphasiges Evaluationsverfahren. In der ersten Phase werden sie gegen die laufende Demo-Anwendung ausgeführt und ihr Ausführungsergebnis klassifiziert. In der zweiten Phase bewertet ein zweites LLM die Testqualität anhand fester Kriterien, wobei die manuellen Referenztests aus Kapitel 5.4 als Vergleichsmaßstab dienen. Beide Phasen werden in Kapitel 4.6 beschrieben.

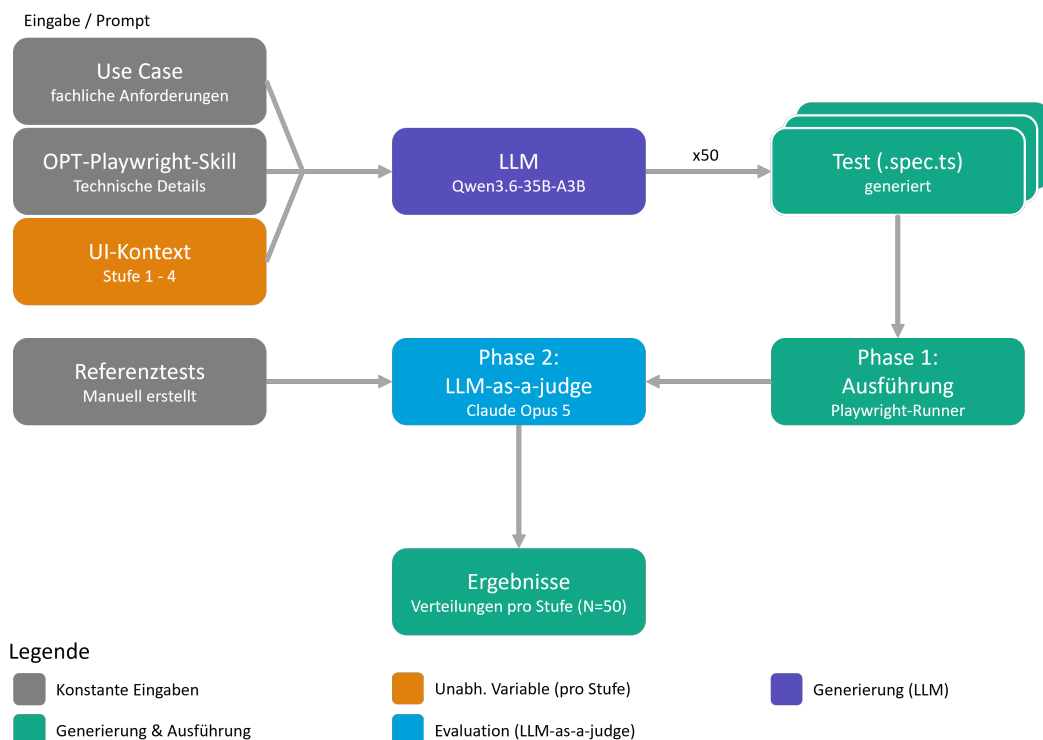


Abbildung 4: Generierungs- und Evaluationspipeline eines Durchlaufs. Use Case, OPT-Playwright-Skill und der UI-Kontext bilden gemeinsam den Prompt. Je Use Case und Stufe entstehen 50 Testdateien. Phase 1 führt die Tests gegen die laufende Demo-Anwendung aus und klassifiziert das Ergebnis, Phase 2 bewertet die Testqualität per LLM-as-a-Judge mit den manuellen Referenztests als Vergleichsmaßstab.

4.2. Use Cases: Definition und Format

Die Use Cases sind die fachliche Eingabe, aus der die Tests generiert werden. Sie sind über alle vier Stufen identisch und gehören damit zu den konstant gehaltenen Faktoren.

Jeder Use Case beschreibt eine Nutzerinteraktion mit der WebGIS-Anwendung auf fachlicher Ebene aus Sicht des Anwenders. Die Schritte enthalten bewusst keine Selektoren oder andere technische Details der Oberfläche, damit das Wissen allein aus dem UI-Kontext der jeweiligen Stufe kommt. Zudem ist es nicht sinnvoll, in einer Nutzerinteraktion technische Details der Anwendung festzuhalten, da der Anwender diese ebenfalls nicht kennt.

Die Use Cases werden im Markdown-Format erfasst, damit sie sowohl menschenlesbar als auch für LLMs einfach zu verarbeiten sind. Jeder Use Case besteht aus Informationen zu `title`, `description`, `preconditions`, `steps`, `expected results` und `complexity`. Titel und Beschreibung geben an, welche Interaktion der Use Case darstellt. Die Vorbedingungen halten fest, was vor der Ausführung erfüllt sein muss. Die Schritte beschreiben die durchzuführende Interaktion des Anwenders und die erwarteten Ergebnisse legen fest, welcher Zustand danach eintreten soll.

Die Eigenschaft `complexity` gruppiert die Use Cases in drei Schwierigkeitsstufen: `easy`, `medium`, `hard`. Die Einstufung basiert nicht allein auf der Anzahl der Schritte, sondern vor allem auf der Art und Tiefe der Interaktionen. Einfache Use Cases prüfen einzelne Schaltflächen oder Sichtbarkeiten, komplexere fordern hingegen asynchrone Prozesse, Karteninteraktionen oder das Zusammenspiel mehrerer Komponenten. Ausführliche Informationen zum Inhalt der Use Cases stehen in Kapitel 5.3.

4.3. Versuchsaufbau: Die Kontextstufen

Die Generierung ist für jede Stufe in einem eigenen Skript umgesetzt (`generate_tests_stage_1.py` bis `generate_tests_stage_5.py`). Die Skripte der Stufen 1 bis 4 unterscheiden sich ausschließlich im Aufbau des UI-Kontext-Blocks, alle übrigen Parameter sind identisch. Das Skript der Bonus-Stufe 5 weicht strukturell ab, da es die Ausführung und die Iterationsschleife steuert.¹

¹ Repository der Arbeit: <https://github.com/nils-gb156/ba-webgis-llm-e2e>. Der hier beschriebene Durchlauf mit Demo-Anwendung, Skripten und Ergebnissen ist unter dem Tag `qwen-v1.0` zu finden, der ergänzende Durchlauf aus Anhang C unter `gpt-v1.0`. Dateipfade sind, sofern nicht anders angegeben, relativ zu `src/app/llm/` zu lesen. Eine Übersicht aller referenzierten Dateien steht in Anhang D.

4.3.1. Stufe 1: Baseline (kein UI-Kontext)

In Stufe 1 bleibt der UI-Kontext-Block des Prompts leer. Das LLM erhält nur den Use Case und den OPT-Playwright-Skill, sonst nichts. Selektoren und Verifikationsstrategien muss es aus allgemeinem Wissen über Webanwendungen ableiten, z. B. durch das Raten plausibler Beschriftungen oder ARIA-Rollen. Diese Stufe ist der Referenzpunkt des Experiments und soll zeigen, was ohne Informationen über die konkrete Oberfläche erreichbar ist. Die in Kapitel 5.2 beschriebenen Testbarkeitsmaßnahmen der Anwendung existieren auch hier schon, dem LLM wird ihre Existenz aber nicht mitgeteilt.

4.3.2. Stufe 2: Automatischer Accessibility-Snapshot

Stufe 2 erhält den UI-Kontext automatisch aus der laufenden Anwendung. Ein Headless-Browser lädt die Demo-App und erfasst alle im DOM vorhandenen `data-testid`-Werte sowie den in Abschnitt 2.2.2 beschriebenen Accessibility-Snapshot von Playwright. Der Snapshot wird einmal vor dem Generierungslauf erfasst und für alle Use Cases identisch verwendet.

Der Ansatz benötigt keinen manuellen Pflegeaufwand und wäre damit in der Praxis ein günstiges Szenario. Erfasst wird allerdings nur der initiale Seitenzustand. Elemente, die erst nach Interaktionen erscheinen, fehlen im Snapshot. Auch das Kartenmodell taucht in diesem Kontext nicht auf, da `__openPioneerMap` ein globales JavaScript-Objekt ist und weder im DOM noch im Accessibility-Tree vorhanden ist.

4.3.3. Stufe 3: Automatisch generierte UI-Map

Den gesamten Quellcode der Anwendung in den Prompt zu übernehmen, ist wegen der begrenzten Kontextlänge des LLM nicht praktikabel. Stufe 3 verwendet deshalb eine automatisch aus dem Quellcode generierte UI-Map (`generated-ui-map.md`). Die regex-basierte Extraktion (`generate-ui-map.ts`) durchsucht alle `.ts`- und `.tsx`-Dateien der Anwendung und erfasst `data-testids`, Zustandsabhängigkeiten von Elementen, Sichtbarkeit im Standardzustand und die auslösenden Interaktionen. Informationen zur Sichtbarkeit der Layer und Hintergrundkarten werden zusätzlich aus der Service-Konfiguration (`services.ts`) gelesen. Das Ergebnis wird dann als Markdown-Tabelle gespeichert und in den Prompt übernommen. Zusätzlich enthält der Prompt erstmals die Datei `map-model-helpers.ts` (siehe Listing 4 im Anhang A.1). Deren Funktionen helfen beim Zugriff auf das Kartenmodell und geben dem LLM damit ein Mittel, den Kartenzustand in Assertions zu prüfen. Genauere Informationen zu den Map-Model-Helfern stehen in Kapitel 5.2.

Da der Kontext in dieser Stufe aus dem Quellcode statt aus der laufenden Seite stammt, enthält er deshalb auch Elemente, die im initialen Zustand nicht sichtbar sind. Der Ansatz

bleibt automatisiert, setzt aber Zugriff auf den Quellcode voraus, und seine Qualität hängt davon ab, wie zuverlässig die regexbasierte Extraktion die relevanten Informationen findet.

Mit Stufe 3 ändern sich mit der Beschreibung der Oberflächenelemente und der Zugang zum Kartenzustand zwei Bestandteile des Kontexts gleichzeitig. In der Auswertung muss daher etwas genauer hingesehen werden, welche Auswirkungen auf den UI-Kontext und welche auf die Map-Model-Helfer zurückgehen.

4.3.4. Stufe 4: Manuell erstellte UI-Map

Stufe 4 folgt dem Aufbau von Stufe 3, ersetzt die generierte UI-Map allerdings durch eine manuell erstellte Fassung (`manual-ui-map.json`). Sie wurde mit vollständiger Kenntnis der Anwendung erstellt und enthält alle Informationen zu Selektoren sowie Hinweise zu Interaktionen und Zuständen einzelner Komponenten. Anders als die Tabelle aus Stufe 3 bildet sie die Komponenten hierarchisch ab und modelliert dynamische Sichtbarkeiten mit expliziten Bedingungen. Damit Hierarchie und Beziehungen effizient dargestellt werden können, werden die Informationen im JavaScript Object Notation (JSON)-Format gespeichert. Die Map-Model-Helfer sind identisch zu Stufe 3.

Diese Stufe bildet die theoretische Obergrenze des untersuchten Ansatzes ab. Sie soll zeigen, welche Testqualität erreichbar ist, wenn der Kontext vollständig und geprüft vorliegt. Dem steht allerdings der Aufwand für die Erstellung und Pflege gegenüber. Jede Änderung an der Oberfläche muss in der UI-Map nachgezogen werden, sonst enthält sie veraltete Selektoren oder falsche Informationen zu Sichtbarkeiten und Interaktionen. In der Evaluation wird sich zeigen, ob der Qualitätsgewinn diesen Aufwand im Vergleich zu den Testergebnissen der anderen Stufen rechtfertigt.

4.3.5. Bonus-Stufe 5: Self-Improvement-Loop

Ergänzend zum Vergleich der vier Stufen wird in der Bonus-Stufe 5 eine Selbstverbesserungsschleife untersucht. Sie dient als Erweiterung der anderen Stufen und verfolgt einen etwas anderen Ansatz. Statt den statischen UI-Kontext im Prompt weiter zu verbessern, erhält das LLM Rückmeldung aus der Ausführung seines eigenen Tests, um diesen weiter zu optimieren.

Als Ausgangspunkt dient der Kontext aus dem automatischen Snapshot des initialen Seitenzustands aus Stufe 2, ergänzt um einen Screenshot der Anwendung und die Map-Model-Helfer aus Stufe 3 und 4. Den Ablauf der Schleife zeigt Abbildung 5. Zuerst wird aus dem initialen Kontext ein Test generiert und mit Playwright direkt gegen die laufende Anwendung ausgeführt. Besteht er, endet die Schleife. Schlägt er fehl, wird der nächste Prompt um den fehlgeschlagenen Testcode, die Playwright-Fehlermeldung und den Failure-Snapshot

inkl. Screenshot ergänzt. Das Modell erhält damit die Anweisung, den vorhandenen Test zu korrigieren, statt einen neuen zu erzeugen. Nach spätestens 10 Iterationen bricht die Schleife ab.

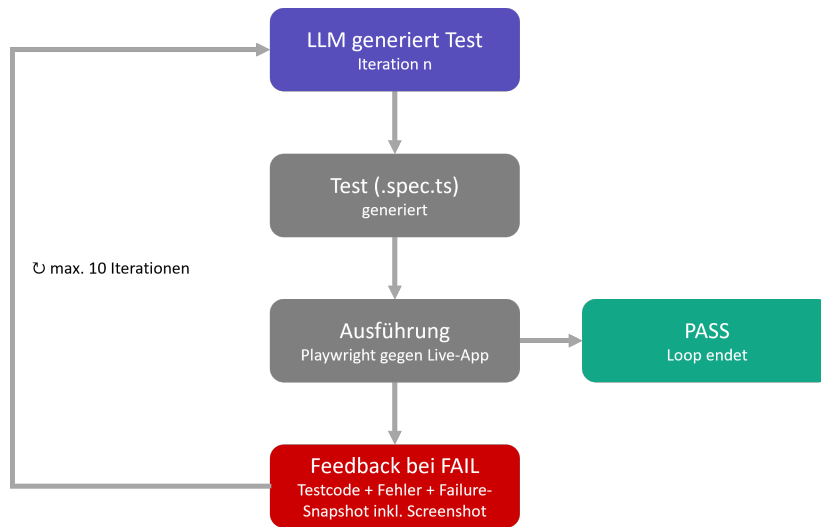


Abbildung 5: Self-Improvement-Loop der Bonus-Stufe 5. Nach jedem fehlgeschlagenen Testlauf fließen Testcode, Playwright-Fehlermeldung und Failure-Snapshot inkl. Screenshot in den nächsten Generierungsprompt mit ein. Die Schleife endet mit dem ersten bestandenen Testlauf oder nach maximal 10 Iterationen.

Der Failure-Snapshot hält den Zustand der Anwendung zum Zeitpunkt des Fehlschlags fest und besteht aus Accessibility-Snapshot und data-testids. Anders als in Stufe 2 wird er nicht beim initialen Laden der Seite erfasst, sondern erst nach dem die bisherigen Testschritte bereits ausgeführt wurden. Das LLM sieht damit, wie die Oberfläche aussah, als der Test scheiterte. Zusätzlich erhält das LLM in der initialen und jeder weiteren Iteration einen Screenshot der Anwendung als Bildeingabe. Dadurch bekommt es auch erstmals visuelle Informationen über den gerenderten Kartenzustand und sieht optisch, welche Elemente sichtbar sind. Als Pass-Kriterium dient der Screenshot nicht. Die Schleife endet nur über das Ausführungsergebnis des Tests. Schlägt der Test beispielsweise fehl, obwohl der zu prüfende Layer im Screenshot sichtbar ist, liegt der Fehler in der Prüflogik des Tests, und die Iteration wird fortgesetzt. Der Screenshot ist nur ein Hilfsmittel für die Korrektur.

Jede Iteration verwendet einen eigenständigen Prompt ohne Konversationshistorie. Alle Informationen aus dem vorherigen Durchlauf werden ausschließlich über den vorherigen Testcode, die Fehlermeldung und den Snapshot übergeben. Für die Aufnahme des Failure-Snapshots wird eine Playwright-Fixture (`failure-snapshot-fixture.ts`) benötigt, die in einer Ausführungskopie des Tests eingebunden wird. Die vom LLM generierte Originaldatei bleibt dabei unverändert und ist Grundlage der Bewertung.

4.4. Modell und Infrastruktur

Für alle Generierungen wird Qwen3.6-35B-A3B als LLM verwendet. Es ist ein offenes Sprachmodell des Qwen-Teams von Alibaba und wird in der FP8-quantisierten Variante eingesetzt (FP8: 8-Bit-Gleitkommaformat zur Reduktion von Speicherbedarf und Rechenaufwand). Das Modell verwendet eine Sparse-Mixture-of-Experts-Architektur (MoE) mit 35 Milliarden Parametern insgesamt, von denen pro Token nur etwa 3 Milliarden aktiv sind. Zudem ist es multimodal, verarbeitet also neben Text auch Bilder. Insgesamt unterstützt es eine Kontextlänge von 262.144 Token (*Hugging Face*, 2026; *Qwen Team*, *Alibaba*, 2026).

Das Modell wird auf einem NVIDIA-DGX-Spark im Netzwerk der con terra GmbH betrieben und dort über vLLM (eine Inferenz-Engine für LLMs) bereitgestellt, das mithilfe der Docker-Konfiguration `eugr/spark-vllm-docker`² auf dem Spark eingerichtet wurde. Die Generierungsskripte greifen darauf über die OpenAI-kompatible REST-API (Representational State Transfer) von vLLM zu.

Die Temperatur ist auf 0,6 gesetzt, da dieser Wert vom Hersteller für präzise Coding-Aufgaben im Thinking-Mode empfohlen wird (*Hugging Face*, 2026). Damit Reasoning und Testcode zusammen ausreichend Platz haben, beträgt die maximale Ausgabelänge 65.536 (64×1024) Token. Diese Konfiguration ist über alle Stufen und Use Cases identisch. Der vollständige Generierungsdurchlauf wird pro Stufe 50-mal wiederholt. Jeder Durchlauf erzeugt alle zehn Use Cases in einem eigenen Verzeichnis, sodass pro Stufe 500 Tests generiert werden.

4.5. Prompt-Aufbau und OPT-Playwright-Skill

Der Prompt besteht aus zwei Teilen. Der User-Prompt enthält die Anweisung, aus dem Use Case mit dem jeweiligen UI-Kontext einen Playwright-E2E-Test zu generieren, den Use Case selbst, die Uniform Resource Locator (URL) der Anwendung und den UI-Kontext-Block der jeweiligen Stufe. Im System-Prompt ist der OPT-Playwright-Skill (`SKILL.md`) enthalten.

Der Skill legt fest, wie der erzeugte Testcode auszusehen hat. Er schreibt die Verwendung von Playwright mit TypeScript vor, verlangt die durchgängige Nutzung von `async` und `await`, priorisiert `getByTestId` gegenüber anderen Lokator-Strategien und definiert das Ausgabeformat. Die Ausgabe soll eine einzelne, vollständige Testdatei ohne erläuternden Text sein. Außerdem enthält der Skill weitere technische Vorgaben, etwa zum Umgang mit Lokatoren, Assertions, Wartebedingungen und Chakra-UI-Elementen. Der Skill ist so aufgebaut, dass er auch für andere WebGIS-Anwendungen wiederverwendet werden kann, die mit Open Pioneer Trails erstellt wurden. Informationen über App-spezifische Selektoren oder Elemente enthält er daher bewusst nicht.

² <https://github.com/eugr/spark-vllm-docker>

4.6. Evaluationsverfahren

Die Evaluation der generierten Tests erfolgt in zwei Phasen. Zuerst werden alle Tests gegen die laufende Demo-Anwendung ausgeführt und anschließend mit einem LLM-as-a-Judge-Verfahren semantisch bewertet. Beide Phasen sollen sich ergänzen, da ein erfolgreich ausgeführter Test nicht zwangsläufig das im Use Case vorgegebene Verhalten prüft, während umgekehrt ein fachlich korrekter Test bereits an einem einzelnen fehlerhaften Selektor scheitern kann. Eine manuelle semantische Bewertung ist aufgrund der Menge von 2.500 Testdateien (50 Generierungen $\times$ 10 Use Cases $\times$ 5 Stufen) ausgeschlossen. Als Bewertungseinheit dient jede einzelne Testdatei und ausgewertet werden die Ergebnisse anschließend je Stufe als Verteilung, nicht als Einzelurteile.

4.6.1. Phase 1: Deterministische Ausführung

In Phase 1 werden alle generierten Testdateien mit Playwright gegen die lokal laufende Demo-Anwendung ausgeführt. Die Ausführung erfolgt nicht manuell, sondern über ein Skript (`run_phase1_eval.py`). Nach jedem Testlauf liest es das Ausführungsergebnis über den JSON-Reporter von Playwright ein und ordnet es einer von sechs Kategorien zu.

GENERATION_ERROR Der Test entsteht erst gar nicht als valider Code, da die Modellantwort abgeschnitten ist oder Wiederholungsmuster enthält. Erkannt wird dies unter anderem durch eine Klammerprüfung.

COMPILE_ERROR TypeScript- oder Importfehler verhindern die Ausführung, ohne dass die Datei als abgeschnitten erkannt wurde, beispielsweise ein fehlerhafter Import.

INFRA_FAIL Der Test läuft, scheitert aber, bevor er eine inhaltliche Prüfung erreicht, etwa weil ein Selektor sein Element nicht findet oder mehrdeutig ist. Die `expect`-Aussage wird nie erreicht.

ASSERTION_FAIL Der Test erreicht seine Prüfung und die Selektoren lösen ihre Elemente auf, aber die Erwartung trifft nicht zu: Das Ergebnis oder der Zustand ist ein anderer als angenommen.

TIMEOUT Ein äußerer Prozess-Guard bricht den Lauf ab, weil der Playwright-Test selbst nicht terminiert.

PASS Der Test läuft vollständig und erfolgreich durch.

Die Zuordnung erfolgt regelbasiert über Muster in den Fehlermeldungen des Playwright-Reports. Fälle, die keinem Muster eindeutig entsprechen, werden vorläufig als `TIMEOUT` klassifiziert und im Nachgang manuell geprüft und korrigiert.

Für Stufe 5 entfällt eine separate Ausführung, da die generierten Tests schon in jeder Iteration des Loops gegen die Anwendung ausgeführt werden. Das Ausführungsergebnis der letzten Iteration wird im Loop-Protokoll gespeichert, sodass ein Abbildungsskript (`map_stage5_phase1.py`) mit identischer Klassifikationslogik die Tests aus dieser Stufe in dasselbe Kategorieschema überführt.

4.6.2. Phase 2: LLM-as-a-Judge

Die semantische Bewertung übernimmt ein Sprachmodell, das die generierten Tests anhand vorgegebener Bewertungskriterien beurteilt. *Zheng et al. (2023)* haben dieses als LLM-as-a-Judge bezeichnete Verfahren untersucht und für GPT-4 eine Übereinstimmung mit menschlichen Bewertungen von über 85 % nachgewiesen, wobei die Übereinstimmung zwischen zwei menschlichen Bewertern bei 81 % liegt (*Zheng et al., 2023, S. 7*). Sie unterscheiden drei Bewertungstypen, die einzeln oder kombiniert eingesetzt werden können (*Zheng et al., 2023, S. 4*). Beim „Pairwise Comparison“ werden zwei Antworten auf dieselbe Frage gegeneinander verglichen, beim „Single Answer Grading“ wird eine einzelne Antwort auf einer absoluten Skala eingeordnet, und das „Reference-guided Grading“ zieht dafür zusätzlich eine vorgelegte Musterlösung heran. Diese Arbeit kombiniert die beiden letztgenannten Typen: Jede Testdatei wird einzeln gegen eine Skala mit definierten Stufenbeschreibungen bewertet, wobei der manuell erstellte Referenztest als Musterlösung dient. Als mögliche Verzerrung der Ergebnisse nennen sie den Self-Enhancement-Bias, also die Bevorzugung selbst erzeugter Ausgaben eines Modells (*Zheng et al., 2023, S. 5*). Ob dieser Effekt wirklich auftritt, lässt ihre Untersuchung offen (*Zheng et al., 2023, S. 5*). Als Vorsichtsmaßnahme stammen Generierungs- und Bewertungsmodell in dieser Arbeit daher von verschiedenen Herstellern.

Als Judge-Modell wird Opus 5 von Anthropic verwendet, das am 24. Juli 2026 veröffentlicht wurde. Opus 5 wurde vom Hersteller als neues Spitzenmodell für Coding- und Wissensarbeitsaufgaben vorgestellt und soll den Vorgänger Opus 4.8 bei Coding-Aufgaben deutlich übertreffen (*Anthropic, 2026*). Die Bewertung läuft agentisch über Claude Code direkt im Repository. Das Modell liest die generierten Testdateien, die Use Cases, die Referenztests und die Ergebnisdatei aus Phase 1 ein und schreibt die Bewertungen fortlaufend in die Ausgabedateien.

Der Bewertungsprompt (Anhang B.1) ist bis auf die Verzeichnispfade über alle Stufen identisch. Die Stufen werden getrennt bewertet und jede Testdatei wird ausschließlich an der Skala der Stufenbeschreibung gemessen, nicht im Vergleich zu anderen Dateien oder Stufen. Bewertet wird blockweise je Use Case über alle 50 Generierungen hinweg. Der Referenztest gilt als Maßstab für korrekte Selektoren, kartenspezifische Interaktionen und vollständige Abdeckung, nicht als Vorlage für die Syntax. Dateien, die in Phase 1 als

GENERATION_ERROR klassifiziert wurden, bleiben unbewertet, da kein valider Testcode vorliegt.

Für Stufe 5 weicht der Bewertungsprompt (Anhang B.1.2) ab, da die Ergebnisse dort wie schon in Phase 1 in einer etwas anderen Form vorliegen. Bewertet wird ausschließlich der Testcode der letzten Iteration. Die Anzahl der benötigten Iterationen erhält das Modell zwar über die Ergebnisdatei aus Phase 1, sie ist aber ausdrücklich kein Bewertungskriterium. Ein Test, der erst nach vier Iterationen besteht, wird genauso bewertet wie einer nach der ersten Iteration.

Bewertet wird in vier Dimensionen, jede auf einer Skala von 1 bis 4 mit ausformulierten Beschreibungen der einzelnen Skalenstufen. Die vollständige Beschreibung ist in Anhang B.2 hinterlegt. Zu jeder Dimension wird zuerst eine kurze Begründung verlangt und erst danach der Punktwert vergeben. Die Dimension *coverage* erfasst, wie vollständig die Schritte und die erwarteten Ergebnisse des Use Cases abgedeckt sind, *selector* die Korrektheit der verwendeten Selektoren, *map_interaction* die Qualität der kartenspezifischen Interaktionen und *assertion*, ob die Prüfungen die Erwartungen des Use Cases tatsächlich verifizieren würden.

Die Dimension *map_interaction* ist nur auf einen Teil der Use Cases anwendbar, bei denen für das erwartete Ergebnis tatsächlich eine Karteninteraktion nötig ist. Use Cases, in denen z. B. nur die Sichtbarkeit einer Schaltfläche geprüft wird, erhalten den Wert n/a . Bei den übrigen Use Cases fließt die Bewertung der Karteninteraktion dagegen in die Auswertung ein.

Zusätzlich wird für jede Datei das Merkmal *vacuous_pass* bestimmt. Es trifft zu, wenn ein Test in Phase 1 als *PASS* klassifiziert wurde, die Dimension *assertion* aber nur mit 1 oder 2 bewertet ist. Der Test ist also erfolgreich, ohne das geforderte Verhalten zu prüfen.

Zur Kontrolle der maschinellen Bewertung wird je Stufe eine Stichprobe von 10 Testdateien manuell überprüft, insgesamt also 50 Dateien. Die Stichprobe ist nach Use Cases geschichtet, d. h. aus jedem Use Case wird pro Stufe genau eine Generierung zufällig gezogen. Geprüft wird, ob die vergebenen Punktwerte den Skalenbeschreibungen der Rubrik entsprechen. Dazu werden der generierte Testcode, die Punktwerte und die zugehörigen Begründungen des Modells gemeinsam betrachtet und je Dimension erfasst, ob die Bewertung bestätigt wird oder Abweichungen vorliegen.

5. Implementierung der Evaluationsumgebung

5.1. Demo-WebGIS-Anwendung

Die Demo-WebGIS-Anwendung stellt eine realistische, aber kontrollierte Testumgebung für die Evaluation bereit. Sie ist als Single-Page-Webanwendung umgesetzt und dient der Erkundung von weltweiten Wetterdaten und -vorhersagen. Die Anwendung ist unter <https://nils-gb156.github.io/ba-webgis-llm-e2e/> erreichbar und kann auch lokal gestartet werden. Der Quellcode liegt im Repository der Arbeit (siehe Fußnote 1).

Abbildung 6 zeigt die Oberfläche der Anwendung. Den Mittelpunkt bildet die Karte. Auf der linken Seite befinden sich die Layersteuerung und die Legende, die über Buttons in der Werkzeugleiste am unteren Rand optional ein- und ausgeblendet werden können. Die Layersteuerung ermöglicht das Umschalten zwischen Hintergrundkarten sowie das Ein- und Ausblenden von Layern. Die Legende darunter zeigt die Symbolerklärungen der jeweils aktiven Layer.

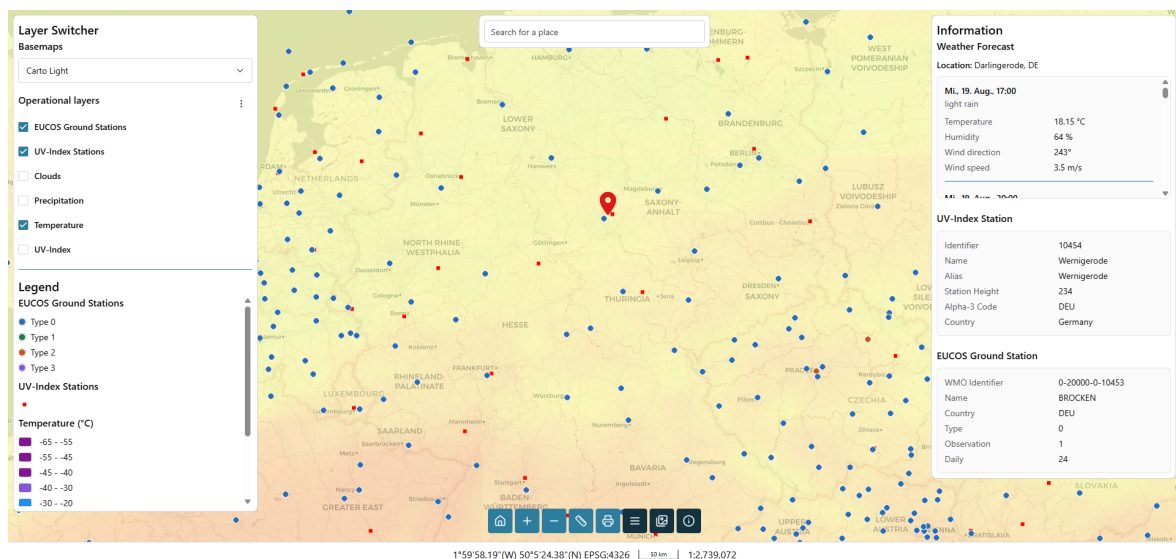


Abbildung 6: Oberfläche der Demo-WebGIS-Anwendung nach einem Klick in die Karte. Links Layersteuerung und Legende, oben mittig die Freitextsuche des Geocoders, rechts das Infopanel mit der Wettervorhersage der geklickten Koordinate und Feature-Informationen.

Die Anwendung bietet drei verschiedene Hintergrundkarten: Carto Light (Standard), Carto Dark und OpenStreetMap. Initial sichtbar sind zwei Punktlayer des Deutschen Wetterdienstes (DWD) für EUCOS-Bodenstationen und UV-Index-Stationen. Zusätzlich ist noch ein flächendeckender Layer der aktuellen Temperatur von OpenWeatherMap sichtbar. Über die Layersteuerung können weitere flächendeckende Layer für Niederschlag und Wolkenbedeckung, ebenfalls von OpenWeatherMap, sowie ein europaweiter UV-Index-Layer des DWD eingeblendet werden.

Ein Klick auf die Karte öffnet ein Infopanel, das den Ort und die Wettervorhersage für die nächsten drei Tage im Drei-Stunden-Takt anzeigt. Die insgesamt 24 Einträge werden über die OpenWeatherMap-API abgerufen und enthalten jeweils eine kurze Beschreibung des Wetters, Temperatur, Luftfeuchtigkeit sowie Windrichtung und -geschwindigkeit. Wird zusätzlich auf eine EUCOS-Bodenstation oder UV-Index-Station geklickt, werden die Stationsinformationen unterhalb der Wettervorhersage angezeigt. Die EUCOS-Bodenstationen sind als Web Feature Service (WFS) eingebunden, sodass die Informationen direkt aus den geladenen Vektor-Features stammen. Die UV-Index-Stationen werden hingegen als WMS bereitgestellt, und die Informationen werden über eine GetFeatureInfo-Anfrage abgerufen.

Oben in der Mitte der Karte befindet sich ein Feld für eine Freitextsuche, die über die Nominatim-Geocoding-API nach Orten sucht. Bei Auswahl eines Suchergebnisses wird die Karte auf den Ort zentriert und das Infopanel mit der Wettervorhersage angezeigt.

Über die Werkzeugleiste lässt sich außerdem zur initialen Ausdehnung zurücknavigieren und ebenso kann der Kartenausschnitt vergrößert oder verkleinert werden. Zwei weitere Buttons rechts daneben öffnen verschiebbare Floating-Panels für Mess- und Druckfunktion. Das Messwerkzeug ermöglicht das direkte Messen von Linien und Flächen in der Karte. Mit der Druckfunktion kann der aktuelle Kartenausschnitt mit allen aktiven Layern als PDF oder PNG exportiert werden. Im Footer werden die aktuellen Kartenkoordinaten in WGS84 (World Geodetic System 1984) sowie eine Maßstabsleiste und der Maßstab angezeigt.

Die Anwendung wurde mit Open Pioneer Trails entwickelt und basiert dementsprechend auf OpenLayers, React, Chakra UI und TypeScript. Sie nutzt die verfügbaren OPT-Pakete für Layersteuerung, Kartennavigation, Legende, Messung, Druck und Koordinatenanzeige. Für die Umsetzung dieser Demo-Anwendung eignet sich OPT durch die freie Gestaltbarkeit von Layout und Funktionalität bei gleichzeitig vorhandenen GIS-Widgets für genau diese Funktionen. Die Kartenprojektion ist European Petroleum Survey Group (EPSG):3857, die Koordinatenanzeige im Footer transformiert diese zu EPSG:4326.

5.2. Testbarkeitsmaßnahmen der Anwendung

Die Anwendung wurde so umgesetzt, dass ihr Zustand für einen E2E-Test prüfbar ist. Auf DOM-Ebene geschieht das über `data-testid`-Attribute nach der in Kapitel 2.3.2 beschriebenen Trails-Konvention. Insgesamt sind 39 Attribute vergeben, von denen 24 bereits im initialen Seitenzustand sichtbar sind. Die restlichen erscheinen erst nach einer Interaktion.

Für den Kartenzustand reicht das nicht aus, da die Karte auf einem Canvas gerendert wird (siehe Kapitel 2.4). Aktive Layer, Zoomstufe, Zentrum und Highlight sind über das DOM nicht direkt auslesbar. Die Anwendung exponiert deshalb das Kartenmodell als globales JavaScript-Objekt `__openPioneerMap`. Die Zuweisung erfolgt in der Kartenkomponente in einem

`useEffect`, sobald das Objekt verfügbar ist. Exponiert wird dabei nicht die OpenLayers-Karte selbst, sondern das `MapModel` von OPT. Über dessen Layer-Sammlung (`map.layers`) lassen sich beispielsweise die aktive Hintergrundkarte und die Sichtbarkeiten der Layer erreichen. Auf Zoomstufe, Zentrum und Highlight kann das `MapModel` über `map.olMap` zugreifen.

Die Datei `map-model-helpers.ts` stellt fertige Funktionen für den Zugriff auf dieses Objekt bereit. Jede Funktion enthält einen `page.evaluate()`-Aufruf, liest den Zustand im Browser-Kontext aus und gibt einen prüfbaren Wert zurück. Sie sind bewusst ausgelagert, da sie unabhängig von dieser Anwendung sind und in anderen OPT-Anwendungen wiederverwendet werden können. Passend zu den Use Cases stellt die Datei die Funktionen `getActiveBaseLayerTitle()`, `isLayerRendered()`, `getMapZoomLevel()`, `getMapCenter()` und `getHighlightedCoordinate()` bereit.

Diese Maßnahmen sind Eigenschaften der Anwendung und keine Bestandteile des UI-Kontexts. Sie sind in allen Stufen unverändert vorhanden. Variabel ist nur, ob dem LLM im Prompt mitgeteilt wird, dass sie existieren (vgl. Kapitel 4.1).

5.3. Use-Case-Sammlung

Für das Experiment wurden zehn Use Cases (`use_cases.md`) erstellt, die typische Nutzerinteraktionen mit der Demo-WebGIS-Anwendung abbilden. Sie folgen dem in Kapitel 4.2 beschriebenen Format und verteilen sich auf drei einfache, zwei mittlere und fünf schwere Use Cases. Tabelle 1 zeigt eine Übersicht, die vollständigen Beschreibungen sind in Anhang A.3 hinterlegt.

Die einfachen Use Cases prüfen einzelne Bedienelemente über den DOM, etwa das Ein- und Ausblenden der Layersteuerung, den Wechsel der Hintergrundkarte oder die Bedienung der Zoom-Buttons. Die mittleren Use Cases aktivieren zunächst ausgeblendete Layer und prüfen anschließend einen daraus resultierenden Zustand. Bei UC-04 ist es das tatsächliche Rendern der Kacheln auf der Karte und bei UC-05 die Aktualisierung der Legende.

Die schweren Use Cases hingegen erfordern Karteninteraktionen, asynchrone Prozesse oder das Zusammenspiel mehrerer Komponenten. UC-06 prüft die Wettervorhersage durch einen Klick auf eine beliebige Position auf der Karte, während UC-07 das gezielte Treffen einer vorgegebenen Koordinate erfordert. Hierbei muss das LLM die Koordinate zuerst in Pixelkoordinaten umrechnen, bevor der Klick simuliert werden kann. UC-08 (Streckenmessung) verlangt eine mehrstufige Zeichnung auf dem Canvas inklusive Doppelklick zum Abschluss, und UC-09 prüft den PNG-Export der aktuellen Kartenansicht. Das Besondere an diesen beiden Use Cases ist, dass diese Funktionen direkt über OPT-Komponenten eingebunden

Tabelle 1: Übersicht der zehn Use Cases mit Komplexitätsstufe und der jeweils getesteten Kernfunktion der Demo-Anwendung. Die vollständigen Beschreibungen mit Vorbedingungen, Schritten und erwarteten Ergebnissen stehen in Anhang A.3.

Nr	Titel	Komplexität	Kernfunktion
01	Layer Switcher ein-/ausblenden	easy	Panel-Sichtbarkeit
02	Hintergrundkarte wechseln	easy	Basemap-Wechsel
03	Zoom-Buttons verwenden	easy	Zoom-Steuerung
04	UV-Index-Overlay aktivieren	medium	Layer-Rendering
05	Niederschlags-Overlay + Legende	medium	Layer + Legende
06	Wettervorhersage per Kartenklick	hard	Kartenklick-Abfrage
07	Stations-Info abfragen (WMS + WFS)	hard	GetFeatureInfo, GetFeature
08	Distanz messen	hard	Messwerkzeug
09	Karte als PNG exportieren	hard	Druckfunktion
10	Layer konfigurieren, Suche + Vorhersage	hard	Geocoder + Workflow

sind. Bei der Erstellung der WebGIS-Anwendung lässt sich daher nicht beeinflussen, wie im Test mit diesen Komponenten interagiert werden kann. Der komplexeste ist UC-10, da er Layer-Konfiguration, Geocoder-Suche, Kartennavigation und Wettervorhersage in einem einzigen Ablauf kombiniert.

5.4. Manuell erstellte Referenztests

Für jeden Use Case wurde manuell ein Playwright-Test erstellt. Die Erstellung erfolgte vor der LLM-Generierung, um eine Beeinflussung zu vermeiden. Die Referenztests dienen zum einen als fachlich korrekte Referenzimplementierung, die zeigt, wie eine saubere Umsetzung eines Use Cases aussieht, und zum anderen bilden sie den Vergleichsmaßstab für die LLM-generierten Tests in der Evaluation.

Die Tests sind mit Playwright in TypeScript umgesetzt. Pro Use Case gibt es eine eigene Spec-Datei nach dem Namensschema `uc-01-*.spec.ts` bis `uc-10-*.spec.ts`. Jeder Test folgt demselben Aufbau, der die Struktur des Use Cases widerspiegelt: Zuerst wird geprüft, ob die Vorbedingungen erfüllt sind, dann werden die Schritte ausgeführt und zuletzt die erwarteten Ergebnisse überprüft. Listing 1 zeigt diesen Aufbau am Beispiel von UC-04.

Listing 1: UC-04: UV-Index-Layer einblenden und prüfen, ob er auf der Karte angezeigt wird (Playwright-Test)

```

1 // SPDX-FileCopyrightText: 2023-2025 Open Pioneer project
  (https://github.com/open-pioneer)
2 // SPDX-License-Identifier: Apache-2.0
3 import { test, expect } from "@playwright/test";
4 import { isLayerRendered } from "../../map-model-helpers";
5
6 const UVI_LAYER_TITLE = "UV-Index";
7
8 test("UC-4: activate the UV-Index overlay and verify it is rendered on the map", async ({
  page }) => {
9   await page.goto("http://localhost:5173/ba-webgis-llm-e2e/");
10
11   const map = page.getByTestId("map-container");
12   const layerSwitcher = page.getByTestId("layer-switcher");
13   // The TOC renders each layer with a checkbox whose accessible name is the layer
14   // title. `exact` avoids also matching the "UV-Index Stations" layer.
15   const uviToggle = layerSwitcher.getByRole("checkbox", { name: UVI_LAYER_TITLE, exact:
16     true });
17   const uviLabel = layerSwitcher.getByText(UVI_LAYER_TITLE, { exact: true });
18
19   // Preconditions
20   await expect(map).toBeVisible();
21   await page.waitForFunction(
22     () => (globalThis as { __openPioneerMap?: unknown }).__openPioneerMap != null
23   );
24   await expect(layerSwitcher).toBeVisible();
25   await expect(uviToggle).not.toBeChecked();
26   expect(await isLayerRendered(page, UVI_LAYER_TITLE)).toBe(false);
27
28   // Steps
29   await uviLabel.click();
30
31   // Expected result
32   await expect(uviToggle).toBeChecked();
33   await expect.poll(() => isLayerRendered(page, UVI_LAYER_TITLE)).toBe(true);
34 }

```

Bevor ein Test mit der Webanwendung interagiert, muss sichergestellt sein, dass die Anwendung vollständig geladen ist. Statt einer festen Wartezeit (Timeout), die unzuverlässig ist, wird mit `page.waitForFunction(() => globalThis.__openPioneerMap != null)` gewartet, bis das Kartenmodell verfügbar ist. Sobald das Objekt existiert, ist die Karte einsatzbereit. Der Test wartet damit auf ein deterministisches Signal und nicht auf eine geschätzte Dauer.

Die Locator-Strategie basiert auf der in Kapitel 2.2.2 beschriebenen Hierarchie und nutzt bevorzugt `getByTestId`, dann `getByRole` und dann `getByText`. Die Selektoren müssen zudem bewusst präzise gewählt werden, da es sonst zu Fehlzugriffen kommen kann. Zum Beispiel wird in UC-04 der UV-Index-Layer über `getByRole("checkbox", { name: "UV-Index", exact: true })` angesprochen. Die Option `exact` ist hier notwendig, weil sonst auch der Layer „UV-Index Stations“ mitgetroffen werden würde, was im Rahmen des Use Cases falsch wäre. Eine Ausnahme ist UC-09. Dort werden zwei Elemente über CSS-Klassen angesprochen, weil die betreffenden internen Elemente der Druckfunktion von Open Pioneer Trails keine eigene Test-ID besitzen.

Klicks auf der Karte erfolgen über `page.mouse.click(x, y)` mit Pixelkoordinaten, die aus der `boundingBox()` des Kartencontainers berechnet werden. Bei UC-06 genügt ein Klick auf eine beliebige Position, z. B. die Kartenmitte. Bei UC-07 muss dagegen ein konkreter Punkt getroffen werden. Dafür wird die bekannte Koordinate (EPSG:3857) über die OpenLayers-Funktion `getPixelFromCoordinate()` in eine Canvas-Pixelposition umgerechnet.

In UC-04 lässt sich das Aktivieren des UV-Index-Layers auf DOM-Ebene über den Zustand des Toggles prüfen (`toBeChecked`). Dass der Layer in der Layersteuerung aktiviert ist, bedeutet jedoch nicht, dass die Umschaltung im Kartenmodell angekommen ist. Da der Layer asynchron geladen wird, prüft der Test dies zusätzlich wiederholt mit `await expect.poll(() => isLayerRendered(page, UVI_LAYER_TITLE)).toBe(true)`.

6. Durchführung und Evaluation

6.1. Durchführung des Experiments

Die Generierungsabläufe aller Stufen fanden zwischen dem 25. und 27. Juli 2026 statt. Modellkonfiguration und Stand der Demo-Anwendung blieben über diesen Zeitraum unverändert. Verwendet wurden exakt die Parameter aus Kapitel 4.4.

Vorgesehen waren pro Stufe 500 Testdateien, allerdings wurde diese Anzahl nur in den Stufen 1, 2, 4 und 5 erreicht. Stufe 3 umfasst nur 499 Dateien, da im Durchlauf `run_20` der Test zu UC-02 fehlt. Da weder die Testdatei noch der zugehörige Prompt oder die Rohantwort des Modells geschrieben wurde, lässt sich der Fehler auf den Aufruf des Modells eingrenzen, denn erst nach Rückgabe der Antwort werden alle drei Dateien angelegt. Nach einem fehlgeschlagenen Aufruf macht das Skript mit dem nächsten Use Case weiter, statt den gesamten Durchlauf abubrechen. Die genaue Ursache ist nachträglich nicht mehr bestimmbar, da die Konsolenausgabe nicht gespeichert wurde.

Ausgeführt wurden die Tests mit Playwright 1.60.0 und dem darin enthaltenen Chromium unter Node.js 26.5.0 und pnpm 11.1.2. Alle Tests liefen nacheinander gegen die unter `http://localhost:5173/ba-webgis-llm-e2e/` lokal laufende Demo-Anwendung.

Die semantische Bewertung nach Kapitel 4.6.2 wurde zwischen dem 31. Juli und dem 3. August 2026 mit Claude Opus 5 über Claude Code 2.1.220 durchgeführt.

Die Kennzahlen dieses Kapitels wurden skriptgestützt mit den Analyseskripten in `eval_extract/` berechnet und beinhalten die Auswertungen der beiden Evaluationsphasen aus den Ergebnisdateien sowie die Auszählung wiederkehrender Codemuster durch reguläre Ausdrücke über alle generierten Testdateien. Die Skripte wurden ebenfalls mit einem Prompt (`phase2_judge_evaluation_prompt.md`) in Claude Code direkt im Repository mit Opus 5 erstellt. Die Abbildungen dieses Kapitels wurden mit `plot_stage.py` und `plot_all.py` erzeugt. In dieser Arbeit werden nur die wichtigsten Befunde dargestellt. Die vollständigen Auswertungsberichte liegen unter `docs/eval/` im Repository.

Zusätzlich wurde der komplette Generierungsablauf mit GPT-5.4 statt Qwen3.6-35B-A3B durchgeführt (Konfiguration und Ergebnisse siehe Anhang C). Er dient ausschließlich der Frage, ob die beobachtete Wirkung der Kontextstufen an das verwendete Modell gebunden ist, und nicht dem Vergleich der Modellfähigkeiten.

Die vier Stufen unterscheiden sich, wie in Kapitel 4.3 beschrieben, nicht nur im Informationsgehalt, sondern auch im Aufwand ihrer Vorbereitung. Stufe 1 erfordert keine Vorbereitung. Der Accessibility-Snapshot in Stufe 2 und die generierte UI-Map in Stufe 3 werden vor dem Generierungslauf skriptgestützt erzeugt und laufen innerhalb weniger

Sekunden durch. Die manuelle UI-Map für Stufe 4 wurde in ca. 2 h 15 min erstellt. Dies umfasst das Sichten der Komponenten, das Erfassen der Selektoren sowie die Erstellung der Hierarchie und der bedingten Sichtbarkeiten. Hinzu kommt, dass sie bei jeder Änderung an Selektoren, Komponentenstruktur oder Sichtbarkeitsbedingungen händisch aktualisiert werden muss.

6.2. Ergebnisse der Ausführung (Phase 1)

Grundlage der Auswertung sind 1.999 Testdateien der Stufen 1 bis 4. Die Ergebnisse der Bonus-Stufe 5 werden wegen des abweichenden Ausführungsmodells getrennt in Kapitel 6.4 dargestellt. Jede Datei wurde einer der sechs Kategorien aus Kapitel 4.6.1 zugeordnet. Zehn Dateien entsprachen keinem Muster der regelbasierten Zuordnung. Die manuelle Prüfung ergab in neun Fällen einen Syntaxfehler (`COMPILE_ERROR`) und in einem Fall eine Wiederholung ganzer Codeblöcke (`GENERATION_ERROR`).

Tabelle 2 zeigt die Ausführungsergebnisse. Die PASS-Rate steigt monoton von 20,0 % in Stufe 1 auf 38,6 % in Stufe 4. Der größte Einzelschritt liegt mit 13,7 Prozentpunkten zwischen Stufe 2 und Stufe 3, also bei der Einführung der UI-Map mit Informationen über die gesamte Web-Anwendung und der Map-Model-Helfer. Die Übergänge von Stufe 1 zu 2 (+2,0 Prozentpunkte) und von Stufe 3 zu 4 (+2,9 Prozentpunkte) verändern die PASS-Rate dagegen kaum.

Tabelle 2: Ausführungsergebnisse der Stufen 1 bis 4 nach den in Kapitel 4.6.1 definierten Kategorien, absolut und in Prozent der Stufengrundmenge. Erkennbar sind die monoton ansteigende PASS-Rate und die Verschiebung der `INFRA_FAIL`- und `ASSERTION_FAIL`-Anteile.

Kategorie	Stufe 1	Stufe 2	Stufe 3	Stufe 4
PASS	100 (20,0 %)	110 (22,0 %)	178 (35,7 %)	193 (38,6 %)
INFRA_FAIL	345 (69,0 %)	211 (42,2 %)	133 (26,7 %)	145 (29,0 %)
ASSERTION_FAIL	43 (8,6 %)	166 (33,2 %)	184 (36,9 %)	154 (30,8 %)
COMPILE_ERROR	3 (0,6 %)	1 (0,2 %)	3 (0,6 %)	4 (0,8 %)
GENERATION_ERROR	9 (1,8 %)	12 (2,4 %)	1 (0,2 %)	4 (0,8 %)
TIMEOUT	0	0	0	0
<i>n</i>	500	500	499	500

Zwischen Stufe 1 und 2 sinkt der `INFRA_FAIL`-Anteil von 69,0 % auf 42,2 %, während der `ASSERTION_FAIL`-Anteil im selben Schritt von 8,6 % auf 33,2 % steigt. Der Accessibility-Snapshot verbessert demnach das Auffinden der Elemente, sodass die Tests nicht mehr am Selektor scheitern, sondern erst an der inhaltlichen Prüfung. Nicht bei allen Use Cases tritt diese Verschiebung gleich stark auf (Abbildung 7). Bei UC-06, UC-07 und UC-08 wandelt sich der in Stufe 1 fast vollständige `INFRA_FAIL`-Anteil bis Stufe 2 überwiegend

in `ASSERTION_FAIL` um, bei UC-09 und UC-10 fällt diese Verschiebung deutlich geringer aus.

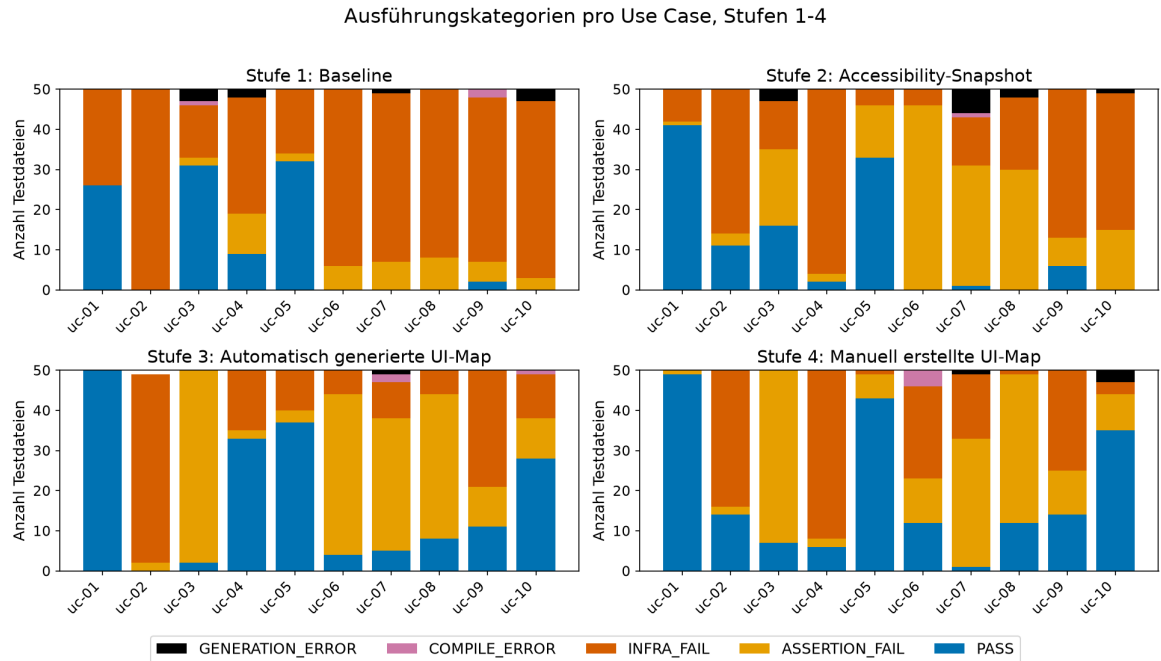


Abbildung 7: Verteilung der Ausführungskategorien je Use Case für die Stufen 1 bis 4, je Use Case und Stufe 50 Testdateien. Der in Stufe 1 dominierende `INFRA_FAIL`-Anteil wandelt sich ab Stufe 2 überwiegend in `ASSERTION_FAIL`, während die karteninteraktiven Use Cases in allen Stufen niedrige `PASS`-Anteile behalten.

Innerhalb der Stufe 4 reicht die `PASS`-Rate von 2 % (UC-07) bis 98 % (UC-01). Use Cases mit einfachen Sichtbarkeitsprüfungen wie UC-01 und UC-05 erreichen bereits in den unteren Stufen hohe Werte, während die karteninteraktiven UC-06, UC-07 und UC-08 in allen vier Stufen unter 25 % bleiben.

Über die Stufen hinweg steigt die `PASS`-Rate bei den meisten Use Cases an. UC-03 ist der einzige Use Case, dessen `PASS`-Rate dabei deutlich schlechter wird: 62 % in Stufe 1, 32 % in Stufe 2, 4 % in Stufe 3 und 14 % in Stufe 4. UC-04 bricht von Stufe 3 auf Stufe 4 von 66 % auf 12 % ein, obwohl die Gesamtrate in diesem Schritt steigt. UC-02 erreicht in Stufe 3 keinen einzigen `PASS`.

Der Rückgang bei UC-03 lässt sich auf einen Fehler bei der Prüfung des Zoomverhaltens zurückführen. Ab Stufe 3 fragen die Tests den numerischen Zoomwert über `getMapZoomLevel` ab. In Stufe 3 entfallen 58,3 % der Fehlschläge dieses Use Cases auf denselben Match-Fehler („expected value must be a number or bigint“), in Stufe 4 sind es 53,5 %. Die Ursache zeigt der generierte Code in Listing 2: Die Tests weisen den Rückgabewert von `expect.poll(...)` einer Variablen zu, um ihn im nächsten Schritt als Vergleichswert zu verwenden. Da `expect.poll` als Assertion aber keinen Wert zurückgibt, erhält die Variable `undefined`, und der Vergleich scheitert. Der manuelle Referenztest in Listing 3

fragt den Zoomwert vor der Interaktion direkt über `getMapZoomLevel (page)` ab und setzt `expect .poll` nur zum Warten auf die Zustandsänderung ein, ohne dessen Rückgabewert weiterzuverwenden.

Listing 2: Fehlerhaftes Muster in den generierten Tests (gekürzt, Stufe 4, run_01)

```
1 | const initialZoom = await expect.poll(() => getMapZoomLevel(page)).toBeTruthy();
2 | await page.getByTestId('zoom-in-button').click();
3 | const zoomedInLevel = await expect.poll(() =>
   | getMapZoomLevel(page)).toBeGreaterThan(initialZoom);
```

Listing 3: Korrektes Muster im manuellen Referenztest (gekürzt)

```
1 | const initialZoom = await getMapZoomLevel(page);
2 | await page.getByTestId("zoom-in-button").click();
3 | await expect.poll(() => getMapZoomLevel(page)).toBeGreaterThan(initialZoom as number);
```

In den Stufen 1 und 2, in denen die Map-Model-Helfer nicht im Kontext stehen, prüfen die generierten Tests für UC-03 stattdessen nur Sichtbarkeiten im DOM. Die Prüfung des Zoomlevels wird nur als Kommentar aufgeführt, und dadurch laufen die Tests häufiger durch.

Die übrigen Kategorien machen nur einen geringen Anteil aus. `COMPILE_ERROR` und `GENERATION_ERROR` machen zusammen je Stufe zwischen 0,8% und 2,6% aus. Abgeschnittene oder degenerierte Modellausgaben (`GENERATION_ERROR`) treten fast nur in den Stufen 1 und 2 auf (9 und 12 Fälle) und gehen danach auf 1 (Stufe 3) und 4 (Stufe 4) zurück. Die vier Kompilierfehler in Stufe 4 gehen alle auf einen falschen relativen Importpfad der Helferdatei in UC-06 zurück.

6.3. Ergebnisse der Qualitätsbewertung (Phase 2)

Grundlage sind je Stufe die 488 bis 498 Dateien, die nach Abzug der `GENERATION_ERROR`-Fälle semantisch bewertet wurden. Die Dimension *map_interaction* ist nur für die kartenrelevanten Use Cases UC-04, UC-06, UC-07, UC-08 und UC-10 anwendbar. Tabelle 3 zeigt Median und Mittelwert je Dimension und Stufe.

Tabelle 3: Median (Md) und Mittelwert (Ø) der vier Bewertungsdimensionen je Stufe (Phase 2, Skala 1–4). Grundlage sind je Stufe die 488 bis 498 Dateien, die nach Abzug der `GENERATION_ERROR`-Fälle bewertet wurden. *map_interaction* ist nur für die kartenrelevanten UC-04, UC-06, UC-07, UC-08 und UC-10 anwendbar.

Stufe	coverage		selector		map_interaction		assertion	
	Md	Ø	Md	Ø	Md	Ø	Md	Ø
1	4	3,68	2	2,48	1	1,40	4	3,29
2	4	3,73	3	3,00	2	1,76	4	3,26
3	4	3,72	4	3,13	3	2,85	4	3,56
4	4	3,91	4	3,41	3	3,01	4	3,59

Die Dimensionen *coverage* und *assertion* haben über alle vier Stufen den Median 4. Der Mittelwert steigt bei *coverage* nur geringfügig von 3,68 auf 3,91 und bei *assertion* von 3,29 auf 3,59. Ob ein Test die geforderten Schritte abdeckt und inhaltlich sinnvoll prüft, hängt demnach stärker vom Verständnis des Use Cases ab als vom bereitgestellten UI-Kontext.

Ein deutlicher Stufenunterschied zeigt sich dagegen in *selector* und *map_interaction*. Der Median von *selector* steigt bereits zwischen Stufe 1 und 2 von 2 auf 3 und erreicht ab Stufe 3 den Wert 4. Der größte Sprung liegt hier beim Übergang zum Accessibility-Snapshot in Stufe 2 ($\emptyset +0,52$). Der Median von *map_interaction* bleibt in Stufe 2 noch bei 2 und steigt erst ab Stufe 3 mit der Einführung der Map-Model-Helfer auf 3 ($\emptyset +1,09$). Die Nutzung einer Helferfunktion springt im selben Schritt von keiner auf 416 Dateien. Beide Dimensionen reagieren also auf unterschiedliche Bestandteile des Kontexts: *selector* auf den erstmals bereitgestellten UI-Kontext durch den Accessibility-Snapshot und *map_interaction* auf die Map-Model-Helfer.

In Stufe 1 sind 33 der 100 PASS-Fälle (33 %) als *vacuous_pass* markiert, davon 31 allein bei UC-03. In den übrigen Stufen liegt der Anteil nur noch zwischen 2,7 und 7,8 % und betrifft vor allem UC-08. Der hohe Wert bei UC-03 in Stufe 1 bestätigt den beschriebenen Befund aus Abschnitt 6.2. Ohne Kenntnis der Map-Model-Helfer prüfen die generierten Tests dort nur Sichtbarkeiten im DOM und haben dadurch eine höhere PASS-Rate, ohne das im Use Case geforderte Zoomverhalten tatsächlich zu verifizieren. Die im Footer angezeigte Maßstabsleiste bietet eine alternative Prüfmöglichkeit, die in dieser Stufe jedoch kein einziger Test nutzt.

Auch innerhalb einer Stufe streuen die Dimensionswerte über die Use Cases (Abbildung 8). Der Widerspruch aus hohem *selector*- und niedrigem *assertion*-Wert, der bei UC-03 in Stufe 1 auftritt, findet sich in gleicher Form auch in Stufe 4 wieder: *coverage* und *selector* erreichen dort 4,0, *assertion* bleibt bei 2,3. Bei UC-07 schwankt *map_interaction* über alle Stufen zwischen 1,4 und 2,3. Ein Anstieg durch die Map-Model-Helfer ist im Vergleich zu den anderen Use Cases nicht zu erkennen. Das liegt daran, dass die Helfer für diesen Fall keine Funktion bereitstellen, da es um einen gezielten Klick auf eine Koordinate in der Karte geht.

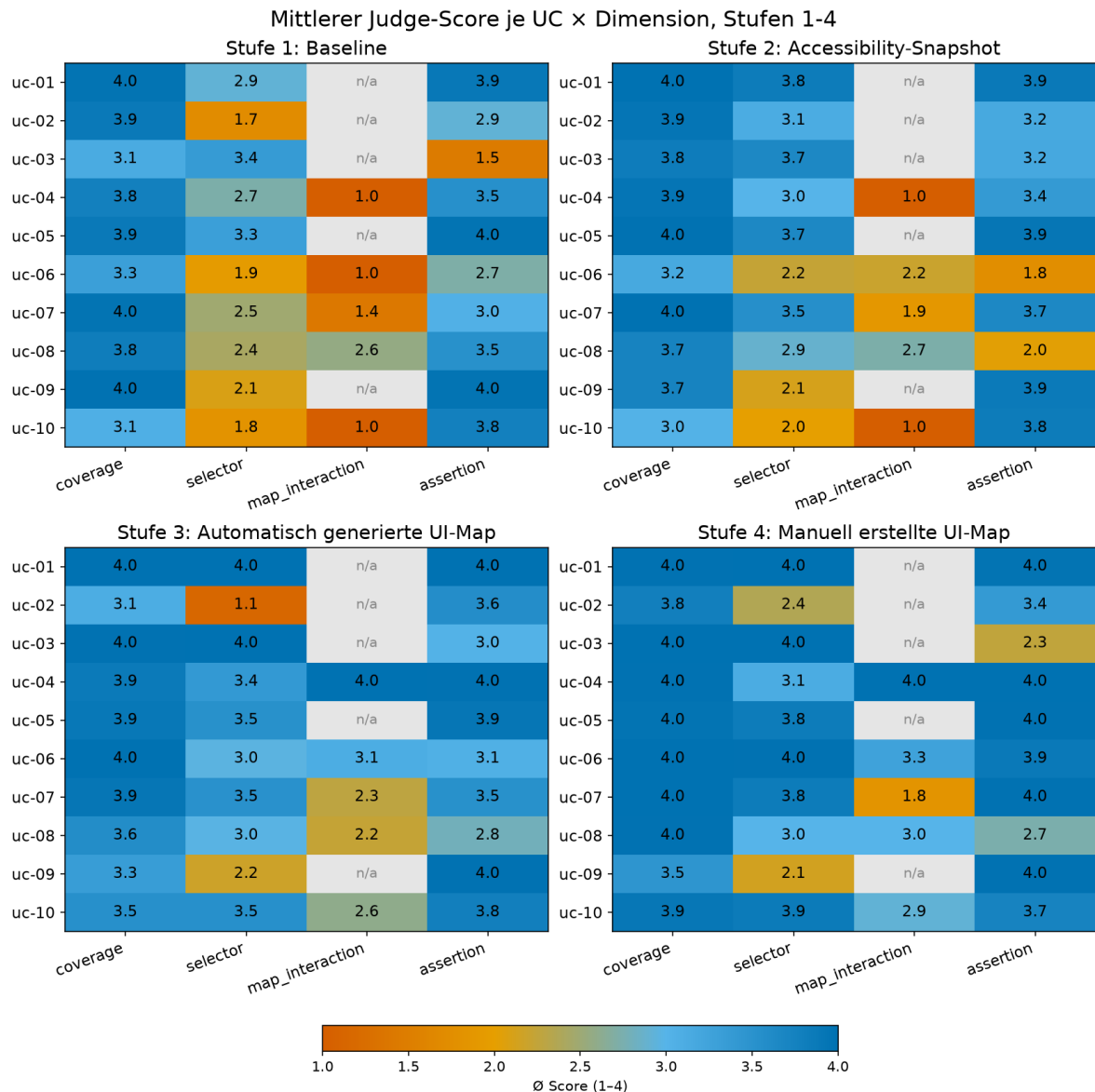


Abbildung 8: Mittlerer Judge-Score je Use Case und Bewertungsdimension für die Stufen 1 bis 4. Graue Felder (n/a) kennzeichnen Use Cases ohne Kartenrelevanz, für die *map_interaction* nicht bewertet wird.

6.3.1. Kontrolle der maschinellen Bewertung

Die Kontrolle der 50 Dateien der Stichprobe mit 175 bewertbaren Einzelurteilen ergab, dass 149 (85,1 %) als bestätigt, 18 als grenzwertig, aber vertretbar, und 8 als abweichend eingestuft wurden. Am niedrigsten liegt die Bestätigungsquote in der Dimension *assertion* (38 von 50), am höchsten in *coverage* (44 von 50) und *selector* (45 von 50). Tabelle 4 fasst das Ergebnis je Stufe zusammen, und die vollständige Aufstellung der Einzelurteile enthält Anhang B.3.

Tabelle 4: Ergebnis der manuellen Kontrolle der maschinellen Bewertung je Stufe, über die vier Dimensionen zusammengefasst. Grundlage sind 50 stichprobenartig gezogene Testdateien mit 175 bewertbaren Einzelurteilen, eingeordnet als bestätigt, grenzwertig, aber vertretbar und abweichend. Anhang B.3 enthält die vollständige Aufstellung der Einzelurteile.

Stufe	bestätigt	grenzwertig	abweichend	<i>n</i>
1	28	6	1	35
2	28	5	2	35
3	29	4	2	35
4	30	3	2	35
5	34	0	1	35
gesamt	149	18	8	175

Auffällig ist, dass die Begründungen der Scores auf einen nummerierten Regelkatalog verweisen, der im Prompt nicht vorgegeben wurde. In der Stichprobe betrifft das 18 der 50 Dateien, verteilt auf alle Stufen außer Stufe 3. Bemerkenswert ist dabei, dass dieselbe Nummer stufenübergreifend für den gleichen Sachverhalt steht. So bezeichnet Regel 16 in den Stufen 1, 2, 4 und 5 jeweils eine falsche Koordinatenlogik und taucht nur bei UC-07 auf. Das Modell muss diese Regeln während der Bewertung selbst gebildet und über die einzelnen Läufe hinweg angewendet haben, obwohl die Bewertung jeder Stufe in einem eigenen Kontextfenster stattfand.

Zudem werden gleiche Befunde nicht durchgängig gleich bewertet. Zwei Dateien der Stufe 2 prüfen die im Use Case geforderten 24 Forecast-Einträge über einen Zähler: einmal über beliebige `div`-Elemente, einmal über eine Rolle, die die Einträge nicht enthält. Keine der beiden Zählungen erfasst damit die im Use Case geforderte Bedingung. In *assertion* erhält die erste Datei dennoch den Wert 1 und die zweite den Wert 4, mit der Begründung, dass eine Zählprüfung auch mit erfundenem Lokator anzuerkennen sei, während dieselbe Zählprüfung im ersten Fall als Argument für den niedrigsten Wert dient. Im Prompt steht jedoch keine Vorgabe, die erfundene Lokatoren zulässt.

6.4. Ergebnisse der Bonus-Stufe 5

Von den 500 Läufen erreichen 366 (73,2 %) ein beständenes Ausführungsergebnis und damit mit Abstand den höchsten Wert aller Stufen (+34,6 Prozentpunkte zu Stufe 4). Die Kategorie `INFRA_FAIL` fällt auf 15 Fälle, `COMPILE_ERROR` auf null, `GENERATION_ERROR` auf 1, während `ASSERTION_FAIL` mit 118 die restlichen Fehlschläge nahezu vollständig ausmacht. Die Use Cases UC-01, UC-02, UC-04 und UC-05 bestehen sogar in allen 50 Läufen, kein Use Case bleibt bei null. Am unteren Ende liegen UC-03 mit 18 % und UC-08 mit 28 %. Abbildung 9 zeigt die Verteilungen der Ausführungskategorien.

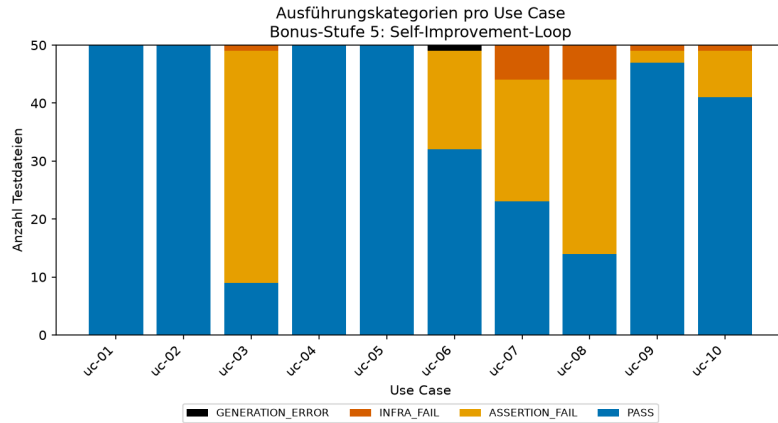


Abbildung 9: Verteilung der Ausführungskategorien je Use Case in der Bonus-Stufe 5 (Phase 1, je Use Case 50 Läufe). Gewertet ist das Ergebnis eines Laufs nach maximal 10 Iterationen. Die verbleibenden Fehlschläge sind fast ausschließlich `ASSERTION_FAIL`.

Den größten Teil dieses Zugewinns liefern die ersten beiden Iterationen (Abbildung 10). Bereits in Iteration 0 ohne Korrektur bestehen 146 Läufe (29,2%), nach dem ersten Durchlauf (Iteration 1) sind es zusammen 274 (54,8%). Ab rund 70 % (Iteration 5) verläuft die Kurve nahezu flach und endet bei 73,2%. 136 Läufe erreichen die letzte Iteration, von denen nur zwei bestehen. Bestandene Läufe benötigen im Mittel 2,22 Iterationen.

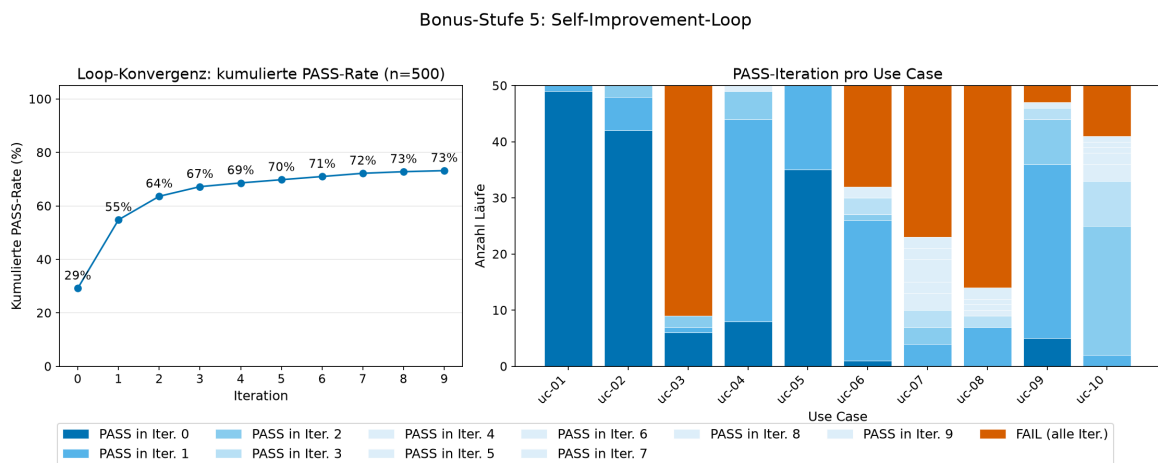


Abbildung 10: Konvergenz des Self-Improvement-Loops der Bonus-Stufe 5. Links die kumulierte PASS-Rate über die Iterationen, rechts die Iteration, in der ein Lauf besteht, je Use Case. Die nur DOM-basierten Use Cases bestehen überwiegend in den ersten Iterationen, die karteninteraktiven erst deutlich später.

Zwischen den Iterationen bleibt die Fehlerklasse überwiegend erhalten. Von den 1.654 Korrekturaufrufen, die aus einem vorherigen Fehlschlag entstanden, führen 1.197 (72,4 %) in dieselbe Klasse zurück. Aus `ASSERTION_FAIL` führen 83,3% der Übergänge wieder zu `ASSERTION_FAIL`, nur 9,2% zu `PASS` und 7,4% zu `INFRA_FAIL` oder `COMPILE_ERROR`. Bei `INFRA_FAIL` bleibt der größte Teil der 484 Übergänge in derselben

Klasse (46,3%), der Rest wechselt vor allem zu `ASSERTION_FAIL` (30,8%) oder direkt zu `PASS` (22,9%).

Die schlechte Performance von UC-03 ist auf denselben Fehler zurückzuführen, der schon in Abschnitt 6.2 beschrieben wurde. 40 der 41 Abbrüche gehen auch hier auf die Weiterverwendung von `expect.poll` zurück. Bei UC-07 und UC-08 dominieren kartenbezogene Fehler. Abgefangene Pointer-Events treten in 122 fehlgeschlagenen Iterationen auf, davon 74 bei UC-07 und 24 bei UC-08.

In der Qualitätsbewertung erreicht Stufe 5 mit 3,96 in *coverage*, 3,77 in *selector*, 3,62 in *assertion* und 3,06 in *map_interaction* die besten Mittelwerte aller Stufen. Die *map_interaction*-Werte der einzelnen Use Cases haben sich gegenüber den Stufen 3 und 4 allerdings nur wenig verbessert (Abbildung 11).

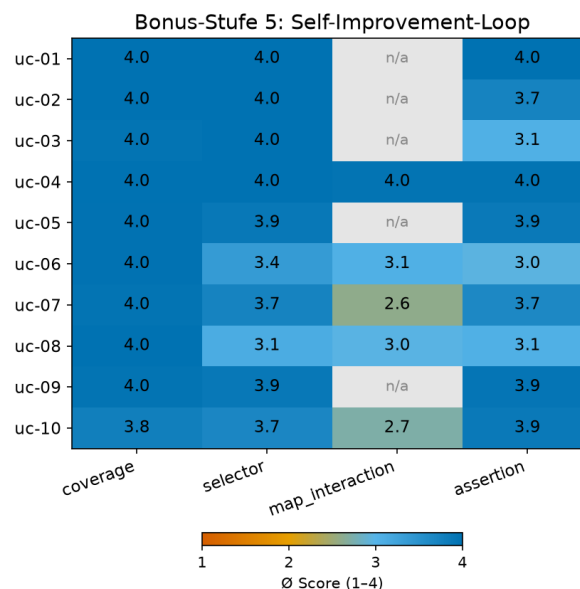


Abbildung 11: Mittlerer Judge-Score je Use Case und Dimension der Bonus-Stufe 5. Über alle Stufen gemittelt erreicht Stufe 5 in allen vier Dimensionen die besten Werte aller Stufen.

Von den bestandenen Tests sind 14 (3,8%) als `vacuous_pass` eingestuft. Sie treten ausschließlich bei UC-07 (8) und UC-08 (6) auf und entsprechen damit den Use Cases mit den kartenbezogenen Fehlern.

Der Ressourcenaufwand unterscheidet sich deutlich gegenüber den anderen Stufen. Über alle Iterationen entstanden 2.154 Generierungsaufrufe gegenüber 500 je Stufe. Zudem ist die Gesamtdauer der Generierung mit 15 h 1 min im Vergleich zu Stufe 4 mit 3 h 46 min deutlich länger und aufwendiger.

6.5. GIS-spezifische Besonderheiten

Während Abschnitt 6.3 die Karteninteraktion mit *map_interaction* semantisch bewertet, wird nun anhand deterministischer Codemuster analysiert, wie die generierten Tests mit dem Kartenzustand und der Karteninteraktion tatsächlich umgehen.

Ohne UI-Kontext erfindet das Modell Selektoren. Sobald ein Kontext vorliegt, geschieht das kaum noch. In Stufe 1 verwenden 197 der 500 Dateien (39,4 %) mindestens einen `getById`-Aufruf mit einem Wert, der in der Anwendung nicht existiert. Bereits in Stufe 2 sinkt dieser Anteil auf 2 Dateien (0,4 %). In den Stufen 3 und 4 bleibt er mit 1,0 % bzw. 1,8 % sehr niedrig.

Die verbleibenden Fälle der Stufen 2 bis 4 betreffen überwiegend die Druckfunktion in UC-09. Sie ist als fertige OPT-Komponente eingebunden, weshalb ihre Elemente keine `data-testid` haben. Vergeben sind nur der Toolbar-Toggle für die Umstellung der Sichtbarkeit und das umgebende Panel. Für Titelfeld, Formatauswahl und Export-Button bleibt dem Modell damit nur der Rollen-Selektor oder die Erfindung.

Welchen Zugang die Tests zum Prüfen des Kartenzustands wählen, hängt davon ab, was der Kontext anbietet. In Stufe 1 bekommt das LLM keine Informationen, weshalb 30,8 % der Dateien über CSS auf das Canvas zugreifen, meistens mit `locator('canvas')`. Dieser Selektor ist allerdings mehrdeutig, da OpenLayers seit Version 6 je Layer einen eigenen Renderer verwendet (*OpenLayers*, o. J.) und die Karte dadurch aus mehreren Canvas-Elementen zusammensetzt, die per CSS über das DOM zum finalen Kartenbild zusammengefügt werden (*Hocevar*, 2020). Das führt in 70 Dateien unmittelbar zum Fehlschlag. Stufe 2 liefert mit `map-container` erstmals einen Namen für die Karte, sodass der Canvas-Zugriff auf 1,4 % einbricht, während `getById('map-container')` von 13,0 auf 53,4 % steigt. Die Prüfung bleibt aber DOM-basiert: Geprüft wird, dass das Kartenelement vorhanden ist, aber nicht, was die Karte zeigt. Erst Stufe 3 öffnet mit den Map-Model-Helfern den Kartenzustand selbst. 83,4 % der Dateien nutzen sie, in den Stufen 1 und 2 wird keine selbst erstellte Helferfunktion genutzt.³ Tabelle 5 fasst die Anteile dieser Muster über die Stufen zusammen.

³ Die regelbasierte Auszählung weist für die Stufen 1 und 2 je eine Datei (0,2 %) aus. In beiden wird kein Helfer verwendet: in Stufe 1 steht der Funktionsname nur in einem Kommentar, in Stufe 2 ruft der Test ein selbst erfundenes `Global __getMapZoomLevel` auf. Die Suchmuster prüfen den bloßen Funktionsnamen und erfassen daher auch solche Stellen.

Tabelle 5: Anteil der Testdateien je Stufe, die auf die Karte mit einem Canvas-Selektor, map-container-Selektor, Canvas-Assertion und Map-Model-Helfer-Nutzung zugreifen. Die Muster schließen sich nicht gegenseitig aus, die Spalten summieren sich daher nicht auf 100 %.

Muster	Stufe 1	Stufe 2	Stufe 3	Stufe 4
<code>locator('canvas')/.ol-viewport)</code>	30,8 %	1,4 %	0,4 %	0,4 %
<code>getByTestId('map-container')</code>	13,0 %	53,4 %	50,1 %	31,8 %
Assertion auf map-container/Canvas	35,2 %	32,2 %	29,7 %	7,0 %
Nutzung eines Map-Model-Helfers	0,0 %	0,0 %	83,4 %	74,0 %

In UC-04 wird erwartet, dass der Layer auf der Karte gerendert wird. Ohne Zugriff auf das Kartenmodell weichen die Tests auf Elemente im DOM aus. In Stufe 1 prüfen alle 50 UC-04-Dateien den Zustand der Checkbox und sonst nichts. In Stufe 2 kommt in 49 Dateien der Legendeneintrag als zusätzliche Prüfung hinzu. Ab Stufe 3 nutzen alle UC-04-Dateien `isLayerRendered`, und der Legendeneintrag verschwindet wieder weitgehend, die Checkbox-Prüfung bleibt aber erhalten.

Der Klick auf die Karte bleibt in allen Stufen schwach. UC-06 und UC-07 verlangen beide eine Klick-Interaktion mit der Karte, einmal beliebig und einmal mit vorgegebener Koordinate. Playwright klickt jedoch Pixel relativ zur Elementecke, und für die Umrechnung stellt der Kontext keine Funktion bereit. In vielen Fällen rechnet das Modell die Koordinate in UC-07 nicht um, sondern übergibt die Kartenkoordinate einfach als Pixelposition. In Stufe 2 betrifft das 41 der 50 UC-07-Tests. Die Tests scheitern dann, da die Klickposition weit außerhalb des Elements liegt. In den bestandenen UC-07-Tests der Stufen 3 und 4 wird die Koordinate korrekt in Pixel umgerechnet.

UC-06 scheitert aus einem anderen Grund. In Stufe 3 rufen alle 50 UC-06-Dateien `getHighlightedCoordinate` auf und prüfen den Rückgabewert auf Existenz. Diese Prüfung schlägt in 32 der 50 Dateien fehl, weil der Wert `undefined` ist, was darauf schließen lässt, dass der Klick die Karte nie erreicht hat. Die Bedienpanels sind jeweils als `MapAnchor` auf dem `MapContainer` verankert und liegen damit über der Kartenfläche. Links oben ist die Layersteuerung, rechts oben das Info Panel usw. (siehe Abbildung 6 in Kapitel 5.1). 24 der 50 Dateien in Stufe 3 klicken auf die Position (100, 100) und treffen damit die Layersteuerung statt der Kartenfläche.

Stufe 5 zeigt, dass der Loop diese Fehler teilweise selbst behebt. Die kartenbezogenen Use Cases starten in der initialen Iteration nahe bei null: UC-06 besteht 1 von 50 Läufen, UC-07 und UC-08 keinen einzigen. In Abbildung 10 ist zu erkennen, dass diese Use Cases, wenn sie erfolgreich sind, dies erst in späteren Iterationen erreichen. UC-07 benötigt im Schnitt 7,8 und UC-08 8,5 Iterationen, während die rein DOM-basierten UC-01, UC-02 und UC-05 mit 1,0 bis 1,3 Iterationen auskommen.

Besonders deutlich wird das bei UC-08, bei dem eine Linienmessung auf der Karte verlangt wird. In den Stufen 1 bis 4 bleibt die Bestehensquote mit 0 %, 0 %, 8 % und 12 % sehr niedrig. Die Ursache liegt schon in der Interaktion und nicht erst in der Prüfung. Die Tests öffnen zwar das richtige Panel, es entsteht aber in vielen Fällen keine Messung, da das Messpanel bei Aktivierung an einer festen Position (420, 10) über der Karte gerendert wird und somit zum Hindernis für die Zeichnung wird. Auch in Stufe 5 bleibt dieses Grundproblem bestehen. Mit 28 % hat UC-08 dort nach UC-03 die schlechteste PASS-Rate. Er verlangt mehrere Klicks und einen Doppelklick zum Abschluss. Bleibt ein Stützpunkt an einem überlagernden Panel hängen, entsteht kein Fehler an der Klickstelle, sondern der Test scheitert erst am erwarteten Textmuster für das Messergebnis. Das erklärt auch die 30 von 50 `ASSERTION_FAIL`-Fälle. Beobachtbar ist, dass das Modell daraufhin die Prüfung umschreibt, die Klickfolge selbst aber unverändert lässt und die eigentliche Ursache damit nicht behebt. Von den 14 bestandenen Läufen sind zudem 6 als `vacuous_pass` identifiziert worden.

7. Diskussion

7.1. Interpretation der Ergebnisse

Unterschiedliche Bestandteile des UI-Kontexts wirken auf unterschiedliche Qualitätsdimensionen der generierten Tests. Der Accessibility-Snapshot aus Stufe 2 behebt vor allem die Selektorenprobleme. Erfundene `data-testid`-Werte fallen von 39,4% auf 0,4% der Dateien (vgl. Kapitel 6.5). Die Map-Model-Helfer aus Stufe 3 wirken dagegen auf die inhaltliche Prüfung der Karteninteraktion und heben die PASS-Rate um 13,7 Prozentpunkte (vgl. Kapitel 6.2). Die PASS-Rate allein erlaubt für den Übergang von Stufe 2 auf 3 keine Zuordnung, da dort UI-Kontext und Map-Model-Helfer gemeinsam hinzukommen. Die Dimensionswerte trennen die Anteile trotzdem, da *selector* bereits mit dem Snapshot in Stufe 2 steigt und sich mit den Helfern kaum noch ändert, während *map_interaction* erst in Stufe 3 anzieht und die Helferfunktionen erstmals genutzt werden. Der Zuwachs der PASS-Rate in Stufe 3 setzt sich damit aus zwei Anteilen zusammen, die jeweils unterschiedliche Fehlerursachen betreffen. In Stufe 1 wird entsprechend sichtbar, dass die Tests nicht scheitern, weil das LLM Playwright nicht beherrscht, sondern weil es die Anwendung nicht kennt.

Ein bestandener Test sagt für sich genommen wenig über die Aussagekraft der Prüfung aus. Das deckt sich mit dem in Kapitel 1.1 angesprochenen Risiko coverage-getriebener LLM-Testgeneratoren. Bei CoverUp und CodiumAI bestätigen laut *Mathews & Nagappan* (2024, S. 4) sogar 68,1 % bzw. 59,6 % der final erzeugten Tests fehlerhaften Code fälschlich. In Stufe 1 sind 33 % der bestandenen Tests als `vacuous_pass` eingestuft, nahezu alle davon bei UC-03. Ohne die Map-Model-Helfer werden nur Sichtbarkeiten der Elemente im DOM geprüft, nicht aber das im Use Case geforderte Zoomverhalten, genau deshalb ist die PASS-Rate auch so hoch. Ab Stufe 3 versuchen die Tests die richtige Prüfung und scheitern an der Umsetzung, wodurch die PASS-Rate deutlich sinkt. Der Rückgang ist allerdings kein Qualitätsverlust, sondern das Gegenteil. Eine reine Bewertung nach der PASS-Rate hätte für Stufe 1 das beste Ergebnis ausgewiesen, obwohl die Prüfung nicht die ist, nach der verlangt wurde. Die Evaluation aus Phase 2 zeigt also, dass das eigentliche Kriterium nicht ist, ob ein Test besteht, sondern ob er bei einem tatsächlichen Fehler in der Anwendung auch wirklich fehlschlagen würde.

Die manuell erstellte UI-Map in Stufe 4 enthält das vollständige Wissen der Struktur, macht aber keinen großen Unterschied zu Stufe 3. Die häufigste Fehlerkategorie bleibt `ASSERTION_FAIL`. Welches Element aufzufinden ist, lässt sich auch aus einer einfachen Elementliste ableiten, aber wie der erwartete Zustand genau zu prüfen ist, hingegen nicht. Die Übergangsraten aus Stufe 5 verdeutlichen dies: Die Übergangsrate von einem `INFRA_FAIL` zu einem bestandenen Test ist höher als aus einem `ASSERTION_FAIL` und Fehler aus `ASSERTION_FAIL` bleiben auch überwiegend in dieser Kategorie. Der Fehler bei der

Weiterverwendung von `expect.poll` wird auch durch den Loop nicht vollständig behoben. *Olausson et al.* (2024, S. 1) schreiben, dass der Nutzen von Self-Repair durch die Fähigkeit des Modells begrenzt ist, präzises Feedback zum eigenen Code zu geben. Der hier betrachteten Fall betrifft allerdings eher ein Wissensproblem, da dem Modell die Information fehlt, dass `expect.poll` einen Rückgabewert statt einer Assertion erwartet. Stattdessen sollte die Information im OPT-Playwright-Skill ergänzt werden. Betroffen sind nur Stufe 3 und höher, wo die Map-Model-Helfer im Kontext stehen. Die PASS-Raten dieser Stufen sind dadurch nach unten verzerrt, der beobachtete Anstieg über die Stufen fällt also geringer aus als der tatsächliche Effekt des Kontexts.

Der bereitgestellte UI-Kontext ist in Stufe 4 noch nicht an seinem Maximum angekommen. Was noch fehlt, ist Wissen über Lage, Größe und Überlagerung der Elemente. Die in Abschnitt 6.5 beschriebenen Fehlklicks auf ein Bedienpanel statt auf die Kartenfläche, das Messpanel über dem Zeichenbereich in UC-08 und die ohne passende Funktion praktisch nicht lösbare Koordinatenumrechnung in UC-07 haben dieselbe Ursache: Eine Elementliste beschreibt, was existiert, nicht wo es liegt. Das ist eine grundsätzliche Grenze textbasierter Repräsentationen wie HTML oder Accessibility-Tree, die laut *Gou et al.* (2025, S. 1) häufig zu Rauschen, Unvollständigkeit und einem erhöhten Rechenaufwand führen. Als alternative schlagen sie ein rein visuelles, pixelbasiertes Grounding vor, bei dem GUI-Agenten die Umgebung rein visuell wahrnehmen und Operationen direkt auf der Pixelebene ausführen.

Die Bonus-Stufe 5 hat mit 73,2% die höchste PASS-Rate, benötigt dafür aber 2.154 statt 500 Generierungsaufrufe und etwa die vierfache Laufzeit. Der Nutzen liegt aber deutlich in der Korrektur der ersten Iterationen. Einige Tests scheitern nach der ersten Generierung an Kleinigkeiten, die in den ersten Iterationen durch den neuen Kontext und die Fehlermeldung von Playwright direkt behoben werden können. Falsch gewählte Prüfstrategien und Karteninteraktionen können dadurch aber nicht abgeleitet und verbessert werden. Der zusätzliche Screenshot dient zwar als visuelle Hilfe, reicht aber nicht immer aus. Außerdem ist der Loop so konfiguriert, dass er bei einem PASS abbricht und damit auf ein Kriterium optimiert, das mit der Testqualität nur bedingt zusammenhängt. Die Gefahr für einen `vacuous_pass` steigt damit. In dieser Arbeit ist das allerdings kein Problem, da die manuellen Referenztests belegen, dass die Anwendung funktioniert und ein erfolgreicher Test daher plausibel ist. In einem produktiven Umfeld, in dem getestet wird, ob tatsächlich Fehler in der Anwendung sind, kann das zu einem Problem werden.

Die Bewertung der Tests durch LLM-as-a-Judge bedarf ebenfalls einer kurzen Einordnung. Der in Abschnitt 6.3.1 beschriebene Regelkatalog war im Prompt nicht vorgegeben, wurde aber in getrennten Kontextfenstern wiederholt verwendet und nicht durchgängig gleich angewendet. Die Punkte aus der Bewertung sind daher nicht vollständig mit den bereitgestellten Rubriken in Anhang B.2 reproduzierbar. Der Grund liegt aber wohl eher in der Rubrik selbst, da nicht für alle Fälle klar definiert war, wie das LLM die Bewertung einordnen soll. Dieses Muster

deckt sich mit Befunden von *Stureborg, Alikaniotis & Suhara* (2024, S. 1), wonach LLMs als Bewerter zu Verzerrungen neigen, bei mehrdimensionalen Bewertungen Ankereffekten unterliegen und eine geringe Übereinstimmung zwischen verschiedenen Stichproben aufweisen. Die Bestätigungsquote der Stichprobenkontrolle steigt über die Stufen deutlich an, demnach erzeugen bessere Tests auch weniger Grenzfälle.

Der zweite Generierungslauf mit GPT-5.4 zeigt, dass die Auswirkungen des UI-Kontexts nicht an das verwendete Modell gebunden sind. Wichtig anzumerken dabei ist, dass beide Modelle nicht direkt vergleichbar sind. GPT-5.4 lässt sich nicht über eine Temperatur, sondern über den Parameter `reasoning_effort` steuern, und es liegt nur dieses eine Modellpaar vor. Belastbar ist daher allein die Beobachtung, dass sich die Stufenübergänge auch bei einem anderen Modell zeigen, nicht ein Leistungsvergleich der Modelle. Auch hier verschwinden erfundene Test-IDs von Stufe 1 auf 2, von Stufe 2 auf 3 steigt *map_interaction* deutlich an und die PASS-Raten steigen über die Stufen 1 bis 3 an. Stufe 4 fällt allerdings gegenüber Stufe 3 etwas schlechter aus, was noch einmal verdeutlicht, dass der Unterschied zwischen diesen beiden Stufen nicht sonderlich groß ist.

7.2. Einordnung in den Stand der Forschung

Die in Kapitel 3 benannte Lücke lässt sich damit teilweise schließen. *Ferreira et al.* (2025) nennen fehlenden Kontext als häufigste Fehlerursache, allerdings auf Ebene der Anforderungsbeschreibung. Ihre User Stories beschreiben die zu prüfende Funktionalität nicht eindeutig, weshalb einige Tests erst nach Ergänzung des Kontextes verwendbar waren. Der HTML-Code der betroffenen Seiten lag dem Modell hingegen immer vor. In dieser Arbeit ist es andersherum: Die Use-Case-Beschreibung bleibt über alle Stufen gleich, variiert wird allein der UI-Kontext. Beide Arbeiten betrachten damit verschiedene Ebenen desselben Problems und die Ergebnisse zeigen, dass auch auf der UI-Ebene deutliche Unterschiede entstehen. Dass die Anforderungsbeschreibungen ebenfalls eine Rolle spielen, wird hier auch sichtbar, da die Use Cases zwar das erwartete Ergebnis benennen, nicht aber, auf welcher Ebene es nachzuweisen ist, was sich auf die Dimension *map_interaction* auswirkt (vgl. Abschnitt 7.3).

Bei den Fehlerursachen zeigt sich ein Unterschied. *Plein et al.* (2023) nennen fehlende Imports, doppelte Methodennamen und veraltete Funktionsaufrufe, also Fehler auf Codeebene. In dieser Arbeit treten sie zwar auch auf, aber `COMPILE_ERROR` und `GENERATION_ERROR` machen zusammen je Stufe nur zwischen 0,8 und 2,6 % aus, während `INFRA_FAIL` und `ASSERTION_FAIL` am häufigsten vorkommen. Allerdings lassen sich die Ergebnisse nicht ohne Weiteres übertragen, da die Auswertung der Ausführungsergebnisse in dieser Arbeit auf Playwright-E2E-Tests spezialisiert ist.

Alian et al. (2024) bekommen den Kontext durch Crawling des Zustandsraums der Anwendung und adressieren damit genau die Grenze, die in Stufe 2 sichtbar wird: Ein Snapshot des

initialen Seitenzustandes erfasst nur einen Teil der Oberfläche und ist für Use Cases, die viele Interaktionen erfordern, nicht immer ausreichend. Deutlich wird dies auch am Zugewinn der PASS-Rate mit der automatisch aus dem Quellcode erzeugten UI-Map in Stufe 3.

7.3. Limitationen

Die Anwendbarkeit der Dimension *map_interaction* wurde aus der fachlichen Anforderung des Use Cases abgeleitet und nicht aus der Umsetzung des Referenztests. Daher ist im Nachhinein aufgefallen, dass es zu Abweichungen bei der Anwendung der Dimension kam. Die Prüfung des Legendeneintrags nach Anschalten eines Layers ist im DOM prüfbar. Der Referenztest nutzt aber zusätzlich einen Map-Model-Helfer zur Überprüfung der Sichtbarkeit des Layers, da es der Prüfung nicht schadet. Daher sollte im Use Case schon definiert werden, auf welcher Ebene die Prüfung genau stattfinden sollte. Zum Beispiel durch Angabe, ob nur DOM-basiert, Kartenzustand oder Canvas-Interaktion.

Untersucht wurde eine einzelne Anwendung mit festem Technologiestack. Mehrere Befunde hängen unmittelbar davon ab, etwa die Panel-Überlagerungen über die *MapAnchor*-Elemente oder die abweichenden Erreichbarkeiten der Chakra-Komponenten. Ob sich der Einfluss des Kontexts auch auf andere WebGIS-Stacks übertragen lässt, kann mit diesen Daten nicht vollständig beantwortet werden. Ebenso ist die Sammlung von zehn Use Cases bewusst begrenzt gehalten und deckt nicht den Umfang realer WebGIS-Anwendungen ab.

Die Idee eines deterministischen Aufbaus mit Temperatur 0 ohne Thinking-Modus wurde bewusst verworfen. *Ouyang et al. (2025, S. 1)* zeigen, dass die Einstellung der Temperatur auf 0 zwar zu weniger Nicht-Determinismus führt als die Konfiguration Temperatur 1, aber trotzdem keinen Determinismus bei der Codegenerierung garantiert. Nicht protokolliert wurden Token-Verbrauch und Generierungszeiten je Aufruf, weshalb der Aufwandsvergleich der Bonus-Stufe 5 nur auf der Anzahl der Aufrufe und der gesamten Generierungszeit beruht.

Der OPT-Playwright-Skill wurde als Konstante über alle Stufen eingesetzt, wirkt aber nicht in jeder Stufe gleich. Die in Abschnitt 7.1 beschriebene fehlende Vorgabe zu `expect.poll` kann nur dort zum Problem werden, wo die Map-Model-Helfer im Kontext stehen. Der Skill wirkt hier also stufenabhängig, was für die Befunde unkritisch ist, sich aber in der Höhe der PASS-Raten ab Stufe 3 widerspiegelt.

7.4. Handlungsempfehlungen

Für Projekte, die LLM-gestützte Testgenerierung einsetzen wollen, ist der Accessibility-Snapshot der einfachste Einstieg, da er den größten Teil der Selektorenprobleme behebt. Seine

Grenze hängt allerdings von der Interaktivität der Anwendung ab. In der hier untersuchten Anwendung enthält der Snapshot 24 der 39 vergebenen Test-IDs (vgl. Kapitel 5.2). Für Anwendungen, in denen die meisten Elemente erst später sichtbar werden, stößt er schnell an seine Grenzen. Dort lohnt es sich dann, die in Stufe 3 automatisch erzeugte UI-Map zu verwenden, da sie auch Elemente enthält, die erst nach einer Interaktion erscheinen. Die manuell gepflegte UI-Map kostet dagegen gut zwei Stunden Erstellung plus laufende Pflege bei jeder Änderung und hebt die PASS-Rate gegenüber Stufe 3 nur um 2,9 Prozentpunkte. Unabhängig davon sollten Test-IDs im Entwicklungsprozess konsequent vergeben werden, um die Auffindbarkeit der Elemente zu verbessern.

Bei kartenbasierten Anwendungen kommt die Exposition des Kartenmodells hinzu, über die sich Zustände prüfen lassen, die auf dem Map-Canvas gezeichnet werden. Der Zugriff darauf lässt sich über eine Umgebungsvariable auf die Testumgebung beschränken. Auch die Verwendung der Map-Model-Helfer ist zu empfehlen, da sie dem LLM den Zugriff erleichtern und sich für verschiedene OPT-Anwendungen mit OpenLayers wiederverwenden lassen.

Wenn genügend Ressourcen vorhanden sind, lohnt sich die Optimierung der Tests durch das LLM selbst in einem Loop. So können kleinere Fehler schnell behoben werden, die erst bei der Ausführung mit Playwright sichtbar werden. Außerdem beeinflusst die Auswahl des Modells die Testqualität erkennbar. Die Bonus-Stufe 5 fällt mit GPT-5.4 deutlich besser aus als mit Qwen3.6-35B-A3B. Eine belastbare Empfehlung für ein bestimmtes Modell lässt sich daraus aber nicht ableiten, da nur dieses eine, nicht direkt vergleichbare Modellpaar untersucht wurde (vgl. Kapitel 7.1).

Die Definition und das Format der Use Cases sowie der OPT-Playwright-Skill wirken sich ebenfalls auf die Qualität der Ergebnisse aus. Sie sollten vorher klar definiert werden, damit das Modell weiß, was und wie es zu prüfen hat. Ansonsten ziehen sich technische Fehler und falsche Prüfstrategien durch alle Durchläufe hindurch. Zudem sollten sie regelmäßig überprüft und verbessert werden, um wiederkehrende Fehlermuster zu vermeiden.

8. Fazit und Ausblick

8.1. Fazit

Der im Prompt bereitgestellte UI-Kontext beeinflusst die Qualität der generierten Tests für WebGIS-Anwendungen deutlich und wirkt sich dabei auf zwei getrennte Aspekte aus. Die Ausführbarkeit der Tests hängt vor allem davon ab, ob das Modell die Elemente der Oberfläche überhaupt findet. Dieses Problem wird größtenteils schon durch den Accessibility-Snapshot behoben. Ob die generierten Tests aber auch inhaltlich das Richtige prüfen, hängt dagegen mit dem Zugriff auf das Kartenmodell zusammen. Erst mit den Map-Model-Helfern lässt sich der Kartenzustand verlässlich abfragen und erst dann steigt die inhaltliche Qualität der Karteninteraktionsprüfungen an (vgl. Kapitel 6.3). Die Dimensionen *coverage* und *assertion* bleiben davon dagegen fast unberührt und über alle Stufen nahezu konstant. Sie hängen also stärker vom Verständnis des Use Cases ab als vom bereitgestellten UI-Kontext (vgl. Kapitel 6.3). Der eigentliche Hebel ist also nicht einfach nur „mehr Kontext“, sondern dem LLM gezielt die Mittel bereitzustellen, die es für das Auffinden der Elemente und die Prüfung des Kartenzustands benötigt.

Zwischen der automatisch generierten und manuell erstellten UI-Map liegt kaum ein Unterschied in der Testqualität, dafür aber ein erheblicher Unterschied in Erstellungsaufwand (vgl. Kapitel 6.1, 7.4). Der zusätzliche Pflegeaufwand einer manuell erstellten UI-Map lohnt sich für diese Anwendung damit nicht.

An seiner Grenze ist der bereitgestellte Kontext aber noch nicht angekommen. Was auch der Stufe 4 fehlt, ist Wissen über Lage, Größe und Überlagerung von Elementen. Die Karte lässt sich zwar über das Kartenmodell inhaltlich abfragen, bleibt aber ein Canvas-Element ohne direkte DOM-Struktur. Bedienpanels, die über der Kartenfläche liegen, führen deshalb wiederholt zu Fehlklicks, weil sie die Stellen auf der Karte verdecken, auf die geklickt werden soll. Auch die Umrechnung einer Kartenkoordinate in eine Klickposition gelingt ohne passende Hilfsfunktion nur unzuverlässig.

Auch die iterative Rückmeldung aus der Testausführung stößt an eine Grenze. Sie hebt die Ausführbarkeit noch einmal deutlich an, allerdings auf Kosten von Aufwand und Ressourcen (vgl. Kapitel 6.4) und bringt eine Schwäche mit. Ein Loop, der erst bei einem bestandenen Ergebnis abbricht, optimiert auf dieses Kriterium und nicht auf die Korrektheit der Prüfung. In einem produktiven Umfeld wäre dieses Verhalten sehr riskant (vgl. Kapitel 7.1).

Der ergänzende Lauf mit einem zweiten Modell bestätigt den beobachteten Kontexteffekt. Das stützt die Annahme, dass es sich tatsächlich um einen Effekt des Kontexts handelt und nicht um eine Eigenheit eines einzelnen Modells.

Ob ein generierter Test besteht, sagt wenig darüber aus, ob er auch das prüft, was der Use Case fordert. Das zeigt sich besonders deutlich in den unteren Kontextstufen, in denen ein großer Anteil bestandener Tests auf einer unzureichenden Prüfung beruht (vgl. Kapitel 6.3). Erst die Kombination aus deterministischer Ausführung und semantischer Bewertung macht diesen Unterschied sichtbar. Eine vollständig automatische Generierung, ohne dass jemand nachprüft, was ein Test tatsächlich tut, bleibt damit auch mit umfangreichem UI-Kontext nicht zuverlässig möglich.

8.2. Ausblick

Die Ergebnisse eröffnen mehrere Anschlusspunkte für zukünftige Arbeiten. Dazu gehört zunächst die Erweiterung der Untersuchungsumgebung. Die Demo-Anwendung ist als Single-Page-Anwendung umgesetzt. Reale WebGIS-Portale, wie etwa Klima- oder Umweltportale, verteilen ihre Funktionen häufig über mehrere Seiten. Seitenübergreifende und deutlich anspruchsvollere Use Cases stellen den Workflow nochmal vor zusätzliche Herausforderungen. Zudem bleiben Karteninteraktionen außerhalb des sichtbaren Kartenausschnitts unberücksichtigt. Wenn ein Feature erst durch Verschieben oder Zoomen der Karte zugänglich wird, kommt neben der Koordinatenumrechnung noch eine Navigationsplanung hinzu, die der Kontext bislang nicht unterstützt. Offen ist auch wie sich die Befunde auf andere WebGIS-Frameworks wie Leaflet oder MapLibre übertragen lassen.

Auch in der Methodik ist noch Spielraum für Anpassungen am Generierungs-Workflow. Der hier untersuchte Loop verwendet ein einzelnes Modell, das Generierung und Korrektur in einem Schritt übernimmt. Playwright stellt mit seinen Test Agents mittlerweile eine rollenbasierte Alternative bereit: Ein Planner erkundet die Anwendung und erstellt einen Testplan, ein Generator setzt diesen Plan in Testcode um, und ein Healer prüft fehlschlagende Tests zur Laufzeit und korrigiert sie (*Microsoft*, o. J.). Diese Agenten erkunden die Anwendung dabei selbst, statt auf einen vorab erfassten Kontext angewiesen zu sein, und reihen sich damit in den von *Kim* (2026, S. 81) beschriebenen Ansatz ein, bei dem Agenten Testaufgaben selbstständig planen und ausführen, statt starren Skripten zu folgen. Ob eine solche Aufteilung in getrennte Rollen die beobachteten Schwächen beheben würde und wie sie mit GIS-spezifischen Anforderungen umgehen, wäre ein naheliegender Anschlusspunkt.

Der letzte Punkt betrifft die Erfassung von Aufwand und Kosten. In dieser Arbeit wurde nur die Gesamtlaufzeit je Stufe erfasst, nicht aber Token- oder Zeitverbrauch jedes einzelnen Generierungsaufrufs. Gerade für die Bonus-Stufe 5 mit der hohen Anzahl an Aufrufen wäre interessant, an welcher Stelle im Loop der größte Aufwand anfällt.

Anhang A: Bestandteile des Generierungsprompts

A.1: Map-Model-Helfer

Das folgende Listing zeigt die Map-Model-Helfer (`map-model-helpers.ts`) für den Zugriff auf den Kartenzustand. Die Kommentare sind zur besseren Lesbarkeit gekürzt, die Funktionslogik ist unverändert.

Listing 4: Map-Model-Helfer für den Zugriff auf den Kartenzustand (`map-model-helpers.ts`)

```

1  // SPDX-FileCopyrightText: 2023-2025 Open Pioneer project
   (https://github.com/open-pioneer)
2  // SPDX-License-Identifier: Apache-2.0
3  import type { Page } from "@playwright/test";
4  import type { MapModel } from "@open-pioneer/map";
5
6  /** The type of the value exposed on `globalThis.__openPioneerMap` (see `MapComponent`). */
7  type ExposedMapModel = MapModel;
8
9  /** Returns the title of the currently active base layer. */
10 export function getActiveBaseLayerTitle(page: Page): Promise<string | undefined> {
11     return page.evaluate(() => {
12         const map = (globalThis as { __openPioneerMap?: ExposedMapModel
13             }).__openPioneerMap;
14         return map?.layers.getActiveBaseLayer()?.title;
15     });
16 }
17
18 /** Returns `true` if the operational layer with the given `title` is currently rendered
   on the map. */
19 export function isLayerRendered(page: Page, title: string): Promise<boolean> {
20     return page.evaluate((layerTitle) => {
21         const map = (globalThis as { __openPioneerMap?: ExposedMapModel
22             }).__openPioneerMap;
23         const layer = map?.layers
24             .getOperationalLayers()
25             .find((entry) => entry.title === layerTitle);
26         return layer?.visible === true;
27     }, title);
28 }
29
30 /** Returns the current zoom level of the map. */
31 export function getMapZoomLevel(page: Page): Promise<number | undefined> {
32     return page.evaluate(() => {
33         const map = (globalThis as { __openPioneerMap?: ExposedMapModel
34             }).__openPioneerMap;
35         return map?.olMap.getView().getZoom();
36     });
37 }
38
39 /** Returns the current center of the map in the map projection as [x, y]. */
40 export function getMapCenter(page: Page): Promise<[number, number] | undefined> {
41     return page.evaluate(() => {
42         const map = (globalThis as { __openPioneerMap?: ExposedMapModel
43             }).__openPioneerMap;
44         const center = map?.olMap.getView().getCenter();

```

```

41     return center && center.length >= 2
42         ? ([center[0], center[1]] as [number, number])
43           : undefined;
44     });
45 }
46
47 /** Returns the coordinate of the first active highlight on the map in the map projection
48 as [x, y]. */
49 export function getHighlightedCoordinate(page: Page): Promise<[number, number] |
50 undefined> {
51     return page.evaluate(() => {
52         const map = (globalThis as { __openPioneerMap?: ExposedMapModel
53         }).__openPioneerMap;
54         if (!map) return undefined;
55         // The Highlights class registers a VectorLayer with className "highlight-layer"
56         // on the underlying OL map. Iterate all OL layers to find it.
57         const layers = map.olMap.getLayers().getArray();
58         const highlightLayer = layers.find(
59             (l) => (l as { getClassName?: () => string }).getClassName?.() ===
60                 "highlight-layer"
61         ) as
62             | {
63                 getSource?: () => {
64                     getFeatures?: () => {
65                         getGeometry?: () => { getCoordinates?: () => number[] };
66                     }[];
67                 };
68             }
69             | undefined;
70         const features = highlightLayer?.getSource?.()?.getFeatures?.() ?? [];
71         const coords = features[0]?.getGeometry?.()?.getCoordinates?.();
72         return Array.isArray(coords) && coords.length >= 2
73             ? ([coords[0], coords[1]] as [number, number])
74               : undefined;
75     });
76 }

```

A.2: OPT-Playwright-Skill

Das folgende Listing zeigt den OPT-Playwright-Skill (SKILL.md), der dem LLM im System-Prompt mitgegeben wird.

Listing 5: OPT-Playwright-Skill im System-Prompt (SKILL.md)

```

1  # OPT Playwright Test Generation -- Skill
2
3  You generate end-to-end tests for a web application using Playwright with TypeScript.
4  These conventions are fixed and apply to every test you produce.
5
6  ## Output
7
8  - Return exactly ONE Playwright test file as valid TypeScript and nothing else.
9  - No markdown code fences, no explanation before or after the code.
10 - The first two lines must be the SPDX license header:
11     ...
12     // SPDX-FileCopyrightText: 2023-2025 Open Pioneer project
13     (https://github.com/open-pioneer)

```

```

12     // SPDX-License-Identifier: Apache-2.0
13     ...
14     - Followed by: `import { test, expect } from '@playwright/test';`
15     - Use a single `test(...)` block. The test title must contain the use case id and title.
16     - Do not use `test.describe`, `test.beforeEach`, or custom fixtures. All logic, including
17       precondition checks, belongs in the single test body.
18     - Begin the test by navigating to the base URL given in the prompt
19       (`await page.goto(...)`).
20
21 ## Locators
22
23     - Prefer `getByTestId` whenever a test id is available -- test ids are stable and
24       unambiguous.
25     - Fall back to user-facing properties (`getByRole`, `getByText`, `getByLabel`) only for
26       elements without a test id. Prefer `getByRole` with an accessible name over `getByText`,
27       as plain text matches are often ambiguous.
28     - Do not use CSS selectors or XPath bound to the DOM structure.
29     - If an element has no accessible role, label, visible text, or test id, a scoped CSS
30       class selector may be used as a last resort.
31     - Chakra UI form controls (checkbox, switch, radio) render the real `role`-bearing
32       `` visually hidden underneath a decorative control element
33       (`chakra-checkbox__control` etc.) that intercepts pointer events. Clicking the
34       `getByRole('checkbox' | 'switch' | 'radio')` locator therefore hangs until timeout ("...
35       intercepts pointer events"). Use `click({ force: true })` on the role locator to toggle
36       such a control, and assert the result separately (`toBeChecked()`). Do not switch to
37       `getByText()` for clicking -- visible label texts are often ambiguous (headings, list
38       items) and cause strict mode violations.
39     - Accessible names must be unambiguous. When one control's name is a substring of
40       another's (e.g. "Search" vs "Search Address", or a toolbar toggle and a button of the same
41       name inside a dialog), `getByRole(..., { name })` matches several elements and fails with
42       a strict mode violation. Use `{ exact: true }` and/or scope the lookup to the relevant
43       container (`page.getByRole('dialog', { name }).getByRole(...)`).
44     - A `data-testid` is NOT an element `id`. Never target it with a CSS id selector such as
45       `page.locator('#some-testid')` -- that selector matches nothing and the call hangs until
46       timeout. Always use `getByTestId('some-testid')`. To interact with the map, click the map
47       container element (identified via the context provided in the prompt) with a `position`
48       option.
49     - Toolbar toggle buttons may already be in the active state (`aria-pressed="true"`).
50       Clicking such a toggle blindly closes the panel instead of opening it. Treat the desired
51       end state as the source of truth: assert the panel's visibility, and only click the toggle
52       when its current pressed state does not already match the state the use case requires.
53
54 ## Waiting and assertions
55
56     - Use `async`/`await` for every Playwright call.
57     - Use web-first, auto-retrying `expect` assertions (e.g. `await
58       expect(locator).toBeVisible()`).
59     - For asynchronous values that are not covered by Playwright's built-in auto-retrying
60       assertions (e.g. values read from application state rather than the DOM), use
61       `expect.poll(() => ...)` instead of a single `expect(await ...)` , since the latter
62       evaluates the value only once and does not wait for it to settle.
63     - `expect.poll(callback)` already awaits the callback's return value. Do NOT chain
64       `.resolves` after it (`expect.poll(...).resolves` is unsupported and throws) -- return the
65       value from the callback and assert on it directly.
66     - Choose the matcher to fit the polled value: `.toBe` / `.toEqual` for equality,
67       `.toMatch(/regex/)` for a string pattern, `.toContain(x)` only for a substring or array
68       membership. Never pass a regex to `.toContain` -- use `.toMatch` instead.
69     - Do not use fixed waits (no `waitForTimeout`, no `sleep`).
70     - For steps that depend on network responses or page loads, use `waitForResponse` /
71       `waitForLoadState`.

```



```

40 - To verify that a specific network request was sent, register a `page.on('request', ...)`
    listener before performing the triggering action, then assert on the captured request
    (e.g. with `expect.poll()`).
41 - For actions that trigger a file download, call `page.waitForEvent('download')` before
    performing the triggering action, then await the resulting download and assert on it (e.g.
    `suggestedFilename()`).
42 - Derive the assertions from the `expected_result` field of the use case.
43 - Cover the steps in order as a single user flow.
44
45 ## Application under test (framework-level background)
46
47 - The application is built with Open Pioneer Trails (React + Chakra UI, TypeScript).
48 - The map is rendered with OpenLayers onto an HTML ``. Map content -- the active
    layers, features, zoom level and map position -- is NOT represented as DOM elements and
    therefore cannot be asserted through DOM locators.
49 - Open Pioneer Trails components follow ARIA conventions and can expose a `data-testid`.
    Test ids are not assigned automatically; they exist only where set in the application
    code.
50 - Geodata (map tiles, WMS layers, GetFeatureInfo, geocoder requests) load asynchronously
    over the network and appear only after the response has arrived.
51
52 ## Map state via helper functions (only if provided in the prompt)
53
54 If the prompt provides map model helper functions, the following rules apply. If no
    helpers are provided, this section is irrelevant -- do not invent or import any helper
    module.
55
56 - Map state (active base layer, operational layer visibility, zoom level, center,
    highlighted coordinate) is not in the DOM. Read it only through the helper functions
    provided in the prompt.
57 - Import the helpers with a single STATIC top-level import using exactly the import path
    stated in the prompt. Never use a dynamic `await import(...)` -- it fails at runtime with
    "SyntaxError: Unexpected token 'export'". Never guess a different relative path -- it
    fails with "Cannot find module".
58 - Every helper returns `undefined` until the map is ready, and reflects the result of a
    triggering action only after its asynchronous effect has completed. Never assert on a
    single `await helper(page)`; always wrap the call in `expect.poll(() => helper(page))` so
    it retries until the value settles.

```

A.3: Vollständige Use-Case-Beschreibungen

Die Use Cases (use_cases.md) sind in ihrer originalen englischen Fassung wiedergegeben.

UC-01: Show and hide the layer switcher via the toolbar button

easy

Description	The user toggles the layer switcher (TOC) panel off and on using the toolbar button.
Preconditions	• The app is loaded successfully. • The layer switcher (TOC) is initially visible.
Steps	1. The user clicks the „Layer Switcher“ button in the toolbar to hide the panel. 2. The user clicks the „Layer Switcher“ button again to show the panel.

Expected results

- After the first click, the layer switcher panel is no longer visible.
- After the second click, the layer switcher panel is visible again.

UC-02: Switch the base map from Carto Light to OpenStreetMap*easy***Description**

The user changes the active base map in the layer switcher.

Preconditions

- The app is loaded successfully.
- The Carto Light base map is active by default.
- The layer switcher (TOC) is visible.

Steps

1. The user opens the base map selector in the layer switcher.
2. The user selects „OpenStreetMap“ as the base map.

Expected results

- The OpenStreetMap base map is selected.
- The Carto Light base map is no longer selected.

UC-03: Zoom in and out using the zoom buttons*easy***Description**

The user changes the map zoom level using the zoom in and zoom out buttons.

Preconditions

- The app is loaded successfully.
- The zoom in and zoom out buttons are visible on the map.

Steps

1. The user clicks the „Zoom in“ button to increase the zoom level.
2. The user clicks the „Zoom out“ button to decrease the zoom level.

Expected results

- After clicking the „Zoom in“ button, the map zoom level is higher than before.
- After clicking the „Zoom out“ button, the map zoom level is lower than after zooming in.

UC-04: Activate the UV-Index overlay and verify it is rendered on the map*medium***Description**

The user activates the initially hidden UV-Index overlay and verifies that the layer is requested and rendered on the map.

Preconditions

- The app is loaded successfully.
- The layer switcher (TOC) is visible.
- The UV-Index overlay layer is initially hidden.

Steps

1. The user clicks the visibility toggle of the UV-Index overlay layer to show it.
2. The user waits for the map to load the layer tiles.

Expected results

- The UV-Index overlay layer toggle is in the enabled (checked) state.
- The UV-Index overlay tiles are rendered on the map canvas.

UC-05: Activate the Precipitation overlay and verify the legend updates*medium***Description**

The user activates the Precipitation overlay and checks that the legend reflects the newly active layer.

Preconditions

- The app is loaded successfully.
- The layer switcher (TOC) and legend are visible.
- The Precipitation overlay layer is initially hidden.

Steps

1. The user clicks the visibility toggle of the Precipitation overlay layer to show it.
2. The user views the legend.

Expected results

- The Precipitation overlay layer toggle is in the enabled (checked) state.
- The legend displays an entry corresponding to the Precipitation layer.

UC-06: Click a map position to show the weather forecast*hard***Description**

The user clicks a position on the map and checks that the weather forecast appears in the info panel.

Preconditions

- The app is loaded successfully.
- The info panel is visible.
- The map canvas is interactive.

Steps

1. The user clicks on a position on the map canvas.
2. The user waits for the info panel to load the forecast.

Expected results

- The clicked position is highlighted on the map.
- The info panel displays a weather forecast section.
- The forecast contains 24 entries.

UC-07: Click both point station layers to show feature info*hard***Description**

The user clicks on a location where both a UV-Index Station (WMS) and an EUCOS Ground Station (WFS) are located and checks that the feature info for both point layers appears in the info panel.

Preconditions

- The app is loaded successfully.
- The info panel is visible.
- The UV-Index Stations layer (WMS) is active.
- The EUCOS Ground Stations layer (WFS) is active.
- Both a UVI station and an EUCOS ground station are located at map coordinates [1188692.84, 6767643.28] (EPSG:3857).
- No measurement tool is active.

Steps

1. The user clicks at map coordinates [1188692.84, 6767643.28] (EPSG:3857) on the map canvas.
2. The user waits for the info panel to load the station info for both layers.

Expected results

- The info panel displays a „UV-Index Station“ section with feature information.
- The info panel displays an „EUCOS Ground Station“ section with feature information.

UC-08: Measure a distance by drawing a line on the map*hard*

Description	The user activates the measurement tool, draws a line on the map and checks that a measurement result is shown.
Preconditions	• The app is loaded successfully. • The measurement tool is accessible via the toolbar. • The map canvas is interactive.
Steps	1. The user clicks the „Measurement“ button in the toolbar to open the measurement panel. 2. The user clicks several points on the map canvas to draw a line. 3. The user double-clicks to finish the measurement.
Expected results	• The measurement panel is visible. • The measurement panel displays a length value with a unit.

UC-09: Print the current map view as a PNG*hard*

Description	The user opens the printing panel, enters a title, selects the PNG format and triggers the export of the current map view.
Preconditions	• The app is loaded successfully. • The printing tool is accessible via the toolbar. • At least one base map and one overlay layer are visible on the map.
Steps	1. The user clicks the „Print Map“ button in the toolbar to open the printing panel. 2. The user enters a title for the printout. 3. The user selects the PNG file format. 4. The user clicks the export/print button.
Expected results	• The printing panel is visible. • A PNG file containing the current map view is generated and downloaded. • The printed image shows the visible base map and overlay layers as well as the scale bar.

UC-10: Configure layers, search for a location and load the weather forecast*hard*

Description	The user reconfigures the layer visibility, searches for a location using the geocoder, navigates the map to the selected result, and loads the weather forecast for that position.
--------------------	---

Preconditions

- The app is loaded successfully.
- The layer switcher (TOC) is visible.
- The Temperature overlay layer is initially visible.
- The Precipitation overlay layer is initially hidden.
- The search field (geocoder) is accessible.
- The info panel is visible.
- No measurement tool is active.

Steps

1. The user clicks the visibility toggle of the Temperature overlay layer to hide it.
2. The user clicks the visibility toggle of the Precipitation overlay layer to show it.
3. The user clicks the search field and types a place name (e.g. „Münster“).
4. The user waits for the result list to appear and selects the first result.
5. The user waits for the map to navigate to the selected location.
6. The user waits for the info panel to load the forecast.

Expected results

- The Precipitation overlay layer toggle is in the disabled state.
 - The Temperature overlay layer toggle is in the enabled state.
 - After selecting the search result, the map navigates to the searched location.
 - The info panel displays a weather forecast section with 24 entries.
-

Anhang B: Evaluierung der Phase 2

B.1: Judge-Prompt

B.1.1: Stufen 1 bis 4

Das folgende Listing zeigt den Bewertungsprompt (`phase2_judge_prompt.md`) der Phase 2 für die Stufen 1 bis 4.

Listing 6: Prompt der LLM-Judge-Bewertung (Stufen 1 bis 4)

```

1  # Phase 2: LLM-Judge-Bewertung - Stufe X
2
3  ## Rolle
4
5  Du bewertest die semantische Qualität LLM-generierter Playwright-E2E-Tests. Die
6  deterministische Ausführung (Phase 1) ist bereits erfolgt. Du führst KEINE Tests aus und
7  änderst KEINE Dateien außer den OUTPUT-Dateien.
8
9  ## Parameter
10
11  - STAGE_DIR: src/app/llm/tests/stage_X/
12  - PHASE1_CSV: src/app/llm/tests/stage_X/_phase1_results.csv
13  - REFERENZTESTS: src/app/tests/manual (pro UC eine Datei)
14  - USE_CASES: src/app/llm/use_cases.md
15  - OUTPUT: src/app/llm/tests/stage_X/_phase2_judge.csv (+ _phase2_judge.json)
16  - MAP_UCS: uc-04, uc-06, uc-07, uc-08, uc-10 # UCs mit erforderlicher kartenspezifischer
17  Interaktion
18
19  ## Vorgehen
20
21  Bewertet wird pro Use Case, nicht pro Run: Iteriere über die UCs (uc-01 bis uc-10) und
22  innerhalb jedes UC über alle 50 Runs.
23
24  Pro UC:
25
26  1. Lade EINMAL die Use-Case-Beschreibung aus USE_CASES und den Referenztest aus
27  REFERENZTESTS (uc-01 -> "UC-1:").
28  2. Iteriere über alle Zeilen in PHASE1_CSV mit dieser uc_id (50 Dateien) und bewerte jede
29  Datei unabhängig. Lies dafür ZUERST den generierten Testcode (Datei aus Spalte 'file') und
30  bilde dir ein Urteil anhand von UC und Referenztest. Ziehe exec_category und error_summary
31  aus PHASE1_CSV erst DANACH als Plausibilitätscheck heran.
32  3. Schreibe die Ergebnisse fortlaufend nach jeder Datei, damit bei Unterbrechung nichts
33  verloren geht.
34
35  Überspringe die Bewertung bei exec_category=GENERATION_ERROR: nicht bewertbar, alle
36  Score-Spalten leer lassen, reasoning="GENERATION_ERROR: kein valider Testcode".
37
38  Am Ende MUSS die Zeilenzahl in _phase2_judge.csv der in PHASE1_CSV entsprechen (abzüglich
39  Kopfzeile). Fehlt eine Datei, hole sie nach.
40
41  ## Bewertung
42
43  Bewerte vier Dimensionen in der folgenden Reihenfolge auf Skala 1-4 (map_interaction_score
44  zusätzlich mit n/a). Schreibe pro Dimension IMMER ZUERST eine kurze Begründung (2-3
45  Sätze), DANN den Score.

```

```

33
34 ### coverage_score (Use-Case-Abdeckung)
35
36 1 = Testet den Use Case nicht oder etwas völlig anderes
37 2 = Deckt weniger als die Hälfte der UC-Schritte ab
38 3 = Deckt die wesentlichen Schritte ab, einzelne Lücken
39 4 = Alle Schritte und erwarteten Ergebnisse des UC abgedeckt
40
41 ### selector_score (Selektor-Korrektheit)
42
43 1 = Selektoren überwiegend erfunden/falsch (existieren nicht in der App)
44 2 = Mischung aus korrekten und erfundenen Selektoren
45 3 = Selektoren überwiegend korrekt, kleinere Abweichungen
46 4 = Alle Selektoren korrekt (stimmen mit Referenztest/App überein)
47
48 ### map_interaction_score (kartenspezifische Interaktion)
49
50 n/a = UC erfordert keine kartenspezifische Interaktion (maßgeblich ist MAP_UCS: nicht
    gelistete UCs erhalten n/a)
51 1 = Keine kartenspezifische Aktion/Prüfung, obwohl der UC sie erfordert (z. B. Canvas nie
    angeklickt, Kartenzustand nie geprüft; Aktionen und Prüfungen ausschließlich DOM-basiert)
52 2 = Kartenspezifische Interaktion versucht, aber fehlerhaft (erfundene
    Map-Model-API-Aufrufe, falsche Objektpfade, falsche Koordinaten-/Pixel-Logik) oder ohne
    Warten auf den Kartenzustand
53 3 = Im Wesentlichen korrekt wie im Referenztest, kleinere Abweichungen (z. B.
    unvollständiges Warten, schwächere Zustandsprüfung)
54 4 = Interaktion und Assertions funktional gleichwertig zum Referenztest (korrekte
    Canvas-Aktionen bzw. Map-Model-Nutzung, korrektes Warten, prüft den relevanten
    Kartenzustand)
55
56 ### assertion_score (Assertion-Angemessenheit)
57
58 1 = Keine Assertions oder ausschließlich triviale (vacuous pass möglich)
59 2 = Assertions vorhanden, prüfen aber nicht die UC-Erwartungen
60 3 = Assertions prüfen die UC-Erwartungen im Wesentlichen, kleinere Lücken
61 4 = Assertions prüfen exakt die erwarteten Ergebnisse; Test würde bei defekter Funktion
    zuverlässig fehlschlagen
62
63 ### vacuous_pass (Boolean, abgeleitet - keine eigene Ermessensentscheidung)
64
65 true genau dann, wenn exec_category=PASS UND assertion_score <= 2. Sonst false.
66
67 ## Wichtige Regeln
68
69 - Bewerte den TESTCODE, nicht das Ausführungsergebnis. Ein Test mit INFRA_FAIL oder
    COMPILE_ERROR kann trotzdem coverage_score 4 haben, wenn der Code die richtigen Schritte
    adressiert.
70 - Das Ausführungssignal (exec_category, error_summary) ist nur ein HINWEIS: "element(s)
    not found" stützt einen niedrigen selector_score, ein PASS ist Voraussetzung (nicht
    Beweis) für vacuous_pass.
71 - Der Referenztest ist die Gold-Referenz für korrekte Selektoren, kartenspezifische
    Interaktion und vollständige Abdeckung. Verlange keine identische Formulierung -
    funktional gleichwertige Lösungen sind gleichwertig zu bewerten.
72 - KONSISTENZ ist bei N=50 zentral: gleiche Fehlermuster über verschiedene Runs identisch
    bewerten. Orientiere dich beim Score strikt an den Stufendefinitionen oben, nicht am
    Vergleich mit anderen Runs.
73
74 ## Output
75

```

```

76 | 1. \_phase2_judge.csv - eine Zeile pro Testdatei, Spalten:
    | stage,run,uc_id,file,exec_category,coverage_score,selector_score,
    | map_interaction_score,assertion_score,vacuous_pass (bei GENERATION_ERROR: Score-Spalten
    | leer; map_interaction_score kann "n/a" enthalten)
77 | 2. \_phase2_judge.json - dieselben Zeilen plus verschachteltes Feld "reasoning" mit den
    | vier Begründungen (coverage, selector, map_interaction, assertion).
78 |
79 | Keine Interpretation im Sinne der Forschungsfrage, nur Bewertung pro Datei.

```

B.1.2: Stufe 5 (Self-Improvement-Loop)

Der Bewertungsprompt für Stufe 5 (`phase2_judge_prompt_stage5.md`) ist mit dem Prompt der Stufen 1 bis 4 (Anhang B.1.1) nahe zu identisch. Die Bewertungsskalen und der Output bleiben unverändert. Abweichend sind lediglich die folgenden Punkte:

- In der Rolle wird der Ausführungskontext angepasst: Die Ausführung erfolgte während des Self-Improvement-Loops und wurde über `map_stage5_phase1.py` in die `exec_category`-Taxonomie überführt.
- In den *Wichtigen Regeln* wird ergänzt, dass auch der Loop-Verlauf nicht bewertet wird sowie eine Regel zur stufenübergreifenden Vergleichbarkeit der Maßstäbe.
- Neu hinzugefügt werden ein Abschnitt zur Besonderheit der Stufe sowie ein Hinweis zur besonderen Aufmerksamkeit beim `assertion_score` (siehe folgende Auszüge).

Listing 7: Neuer Abschnitt „Besonderheit dieser Stufe“

```

1 | Die Spalte 'file' enthält den Pfad (relativ zu src/app/llm/) des final_spec -- des
    | Testcodes der LETZTEN Loop-Iteration. NUR diese Datei wird bewertet. Die UC-Ordner
    | enthalten zusätzlich frühere Iterationen (iter-0 bis iter-n-1) und *.exec.spec.ts-Dateien:
    | Das sind Harness-Artefakte -- NIEMALS öffnen oder bewerten. Die Spalten 'passed' und
    | 'iterations_used' sind Metadaten: iterations_used ist KEIN Qualitätskriterium -- ein PASS
    | nach 4 Iterationen wird identisch bewertet wie ein PASS nach 1.

```

Listing 8: Zusatz beim `assertion_score`

```

1 | BESONDERE AUFMERKSAMKEIT in dieser Stufe: Der Loop optimiert auf PASS. Auch wenn der
    | Feedback-Prompt das Abschwächen von Assertions verbietet, ist strukturell möglich, dass
    | ein Test über die Iterationen hinweg weniger prüft, nur um grün zu werden. Prüfe daher bei
    | exec_category=PASS besonders genau gegen die Expected Results des Use Case, ob die
    | Assertions diese tatsächlich verifizieren. Die Maßstäbe der Skala bleiben dabei
    | unverändert -- strengere Aufmerksamkeit bedeutet nicht strengere Bewertung.

```


B.2: Bewertungsrubrik

Tabelle 6: Skalenstufen der Bewertungsdimensionen *coverage*, *selector*, *map_interaction* und *assertion* mit den Kriterien für die Werte 1 bis 4, wie sie dem Bewertungsmodell im Prompt mitgegeben wurden.

Dimension	Stufe	Beschreibung
coverage	1	Testet den Use Case nicht oder etwas völlig anderes
	2	Deckt weniger als die Hälfte der Schritte ab
	3	Deckt die wesentlichen Schritte ab, einzelne Lücken
	4	Alle Schritte und erwarteten Ergebnisse abgedeckt
selector	1	Selektoren überwiegend erfunden oder falsch
	2	Mischung aus korrekten und erfundenen Selektoren
	3	Selektoren überwiegend korrekt, kleinere Abweichungen
	4	Alle Selektoren korrekt
map_interaction	n/a	Use Case erfordert keine kartenspezifische Interaktion
	1	Keine kartenspezifische Aktion oder Prüfung, obwohl der Use Case sie erfordert; ausschließlich DOM-basiert
	2	Interaktion versucht, aber fehlerhaft (erfundene Map-Model-Aufrufe, falsche Koordinaten- oder Pixellogik) oder ohne Warten auf den Kartenzustand
	3	Im Wesentlichen korrekt, kleinere Abweichungen
	4	Funktional gleichwertig zum Referenztest
assertion	1	Keine oder ausschließlich triviale Assertions
	2	Assertions vorhanden, prüfen aber nicht die Erwartungen
	3	Erwartungen im Wesentlichen geprüft, kleinere Lücken
	4	Erwartungen exakt geprüft; der Test würde bei defekter Funktion zuverlässig fehlschlagen

B.3: Stichprobe der manuellen Bewertungskontrolle

Tabelle 7 führt die Einzelurteile auf. Angegeben sind der vom Bewertungsmodell vergebene Punktwert und das Ergebnis der Kontrolle: ● bestätigt, ○ grenzwertig, aber vertretbar, × abweichend. Die vollständige Fassung (`_review_sample.md`) mit den Begründungstexten des Modells und den Prüfnotizen liegt im Repository.

Tabelle 7: Ergebnis der manuellen Kontrolle der maschinellen Bewertung je Testdatei und Dimension. Angegeben sind der vom Bewertungsmodell vergebene Score und die Einordnung der Kontrolle als bestätigt, grenzwertig, aber vertretbar und abweichend. Grundlage sind die 50 Dateien mit 175 bewertbaren Einzelurteilen.

Nr.	Stufe	Lauf	UC	coverage	selector	map_int.	assertion
1	1	06	UC-01	4●	4●	n/a	4●
2	1	25	UC-02	3●	2●	n/a	3○
3	1	02	UC-03	3○	4●	n/a	1●
4	1	07	UC-04	3●	4●	1●	3○
5	1	32	UC-05	4●	4●	n/a	4●
6	1	32	UC-06	3●	2●	1●	2●
7	1	34	UC-07	4●	3○	2●	4○
8	1	08	UC-08	3●	3●	3○	2●
9	1	14	UC-09	4●	2●	n/a	4●
10	1	23	UC-10	3●	2×	1●	2●
11	2	23	UC-01	4●	4●	n/a	4●
12	2	13	UC-02	4●	3○	n/a	3×
13	2	25	UC-03	4●	3●	n/a	4○
14	2	49	UC-04	4○	3●	1●	4×
15	2	07	UC-05	4●	4●	n/a	4●
16	2	23	UC-06	3●	3●	2●	1○
17	2	49	UC-07	4●	4●	2●	4●
18	2	10	UC-08	4○	3●	3●	2●
19	2	21	UC-09	4●	2●	n/a	4●
20	2	34	UC-10	3●	2●	1●	4●
21	3	03	UC-01	4●	4●	n/a	4●
22	3	12	UC-02	3●	1○	n/a	3×
23	3	33	UC-03	4●	4●	n/a	3●
24	3	18	UC-04	4●	4●	4●	4●
25	3	17	UC-05	4●	4●	n/a	4●
26	3	27	UC-06	4●	2●	3○	3●
27	3	14	UC-07	4○	2●	2●	3●
28	3	39	UC-08	3×	3●	2●	3●
29	3	05	UC-09	3○	2●	n/a	4●
30	3	14	UC-10	4●	4●	3●	4●
31	4	14	UC-01	4●	4●	n/a	4●

Fortsetzung nächste Seite

Fortsetzung von Tabelle 7

Nr.	Stufe	Lauf	UC	coverage	selector	map_int.	assertion
32	4	21	UC-02	3●	2●	n/a	3○
33	4	16	UC-03	4●	4●	n/a	2●
34	4	32	UC-04	4●	3×	4●	4●
35	4	34	UC-05	4●	4●	n/a	4●
36	4	23	UC-06	4●	4●	3●	4●
37	4	24	UC-07	4●	4●	2●	4●
38	4	16	UC-08	4●	3●	3●	2●
39	4	34	UC-09	3●	2●	n/a	4○
40	4	14	UC-10	4●	4●	3×	4○
41	5	05	UC-01	4●	4●	n/a	4●
42	5	43	UC-02	4●	4●	n/a	4●
43	5	03	UC-03	4●	4●	n/a	3×
44	5	06	UC-04	4●	4●	4●	4●
45	5	01	UC-05	4●	4●	n/a	4●
46	5	15	UC-06	4●	4●	3●	3●
47	5	50	UC-07	4●	4●	2●	4●
48	5	05	UC-08	4●	4●	3●	4●
49	5	29	UC-09	4●	4●	n/a	4●
50	5	43	UC-10	4●	4●	3●	4●

Anhang C: Ergebnisse des Generierungslaufs mit GPT-5.4

C.1: Durchführung

Der gesamte Generierungsablauf⁴ wurde ein zweites Mal mit GPT-5.4 anstelle von Qwen3.6-35B-A3B durchgeführt. Verwendet wurde das Modell `deployment_gpt-5.4` über die OpenAI-kompatible Schnittstelle von Azure AI Foundry mit `reasoning_effort="medium"` und einem Ausgabebudget von 65.536 Token, wie im Hauptlauf. Der Temperatur-Parameter kann bei diesem Modell nicht eingestellt werden. Use Cases, OPT-Playwright-Skill, Prompt-Aufbau, die Zahl von 50 Durchläufen und der Aufbau der Demo-Anwendung sind unverändert übernommen worden.

Insgesamt wurden 2.500 Testdateien generiert, je Stufe 500 aus 50 Durchläufen mit zehn Use Cases. Eine Datei in Stufe 1 wurde als `GENERATION_ERROR` klassifiziert und wird für die Phase 2-Bewertung ausgelassen, weshalb die Score-Grundmenge dieser Stufe 499 beträgt.

C.2: Ergebnisse der Ausführung (Phase 1)

Tabelle 8 zeigt die Ausführungsergebnisse. Die PASS-Rate steigt von 20,4% in Stufe 1 auf 50,8% in Stufe 2 und 72,4% in Stufe 3, fällt in Stufe 4 aber auf 64,4% zurück. Der größte Einzelschritt liegt mit 30,4 Prozentpunkten zwischen Stufe 1 und 2. Der Anteil von `INFRA_FAIL` sinkt im gleichen Schritt von 51,8% auf 19,8%, während `ASSERTION_FAIL` leicht von 27,6% auf 29,4% steigt. Im Übergang von Stufe 3 zu 4 nehmen beide Fehlerkategorien wieder zu.

Tabelle 8: Ausführungsergebnisse der Stufen 1 bis 4 nach Kategorien, absolut und in Prozent der Stufengrundmenge (GPT-5.4). Kategorien und Zuordnung entsprechen dem Hauptlauf.

Kategorie	Stufe 1	Stufe 2	Stufe 3	Stufe 4
PASS	102 (20,4 %)	254 (50,8 %)	362 (72,4 %)	322 (64,4 %)
INFRA_FAIL	259 (51,8 %)	99 (19,8 %)	77 (15,4 %)	97 (19,4 %)
ASSERTION_FAIL	138 (27,6 %)	147 (29,4 %)	61 (12,2 %)	81 (16,2 %)
COMPILE_ERROR	0	0	0	0
GENERATION_ERROR	1 (0,2 %)	0	0	0
TIMEOUT	0	0	0	0
<i>n</i>	500	500	500	500

Über die Use Cases verteilt sich das Ergebnis ungleich (Tabelle 9). Sechs der 40 Zellen bleiben ohne einen einzigen PASS, zwei davon entfallen auf UC-08 in den Stufen 3 und 4. Im

⁴ <https://github.com/nils-gb156/ba-webgis-llm-e2e/tree/gpt-v1.0>

Übergang von Stufe 3 zu 4 fallen zudem UC-05 (100 % auf 24 %), UC-06 (94 % auf 58 %) und UC-10 (82 % auf 56 %) teils deutlich zurück. Den einzigen `GENERATION_ERROR` enthält UC-03 in Stufe 1.

Tabelle 9: PASS-Rate je Use Case und Stufe (GPT-5.4), je Zelle 50 Generierungen. Die Zeile „Gesamt“ gibt die gerundete Rate über alle zehn Use Cases an.

UC	Stufe 1	Stufe 2	Stufe 3	Stufe 4	Stufe 5
UC-01	94 %	100 %	100 %	100 %	100 %
UC-02	0 %	2 %	0 %	38 %	100 %
UC-03	0 %	94 %	100 %	98 %	100 %
UC-04	78 %	96 %	98 %	100 %	100 %
UC-05	4 %	98 %	100 %	24 %	100 %
UC-06	10 %	22 %	94 %	58 %	96 %
UC-07	2 %	0 %	56 %	78 %	98 %
UC-08	6 %	64 %	0 %	0 %	100 %
UC-09	8 %	28 %	94 %	92 %	100 %
UC-10	2 %	4 %	82 %	56 %	96 %
Gesamt	20 %	51 %	72 %	64 %	99 %

C.3: Ergebnisse der Qualitätsbewertung (Phase 2)

Median und Mittelwert von *coverage* und *assertion* liegen bereits in Stufe 1 nahe am Maximum und bleiben über alle vier Stufen nahezu konstant. *selector* steigt von einem Mittelwert von 2,55 in Stufe 1 auf 3,28 in Stufe 2 und danach nur noch leicht auf 3,67 in Stufe 4. Der Median liegt ab Stufe 2 bei 4. *map_interaction* erreicht in den Stufen 1 und 2 nur 1,82 bzw. 2,00, springt in Stufe 3 auf 3,75 und verändert sich in Stufe 4 mit 3,78 kaum noch. Die Werte sind in Tabelle 10 zusammengefasst. Abbildung 12 zeigt zusätzlich die mittleren Judge-Scores je Use Case und Dimension. *vacuous_pass* tritt nur in Stufe 1 (1 Fall) und Stufe 2 (6 Fälle) auf.

Tabelle 10: Median (Md) und Mittelwert (Ø) der vier Bewertungsdimensionen je Stufe (GPT-5.4, Skala 1–4).

Stufe	coverage		selector		map_interaction		assertion	
	Md	Ø	Md	Ø	Md	Ø	Md	Ø
1	4	3,56	2	2,55	1	1,82	4	3,71
2	4	3,69	4	3,28	2	2,00	4	3,75
3	4	3,65	4	3,58	4	3,75	4	3,79
4	4	3,70	4	3,67	4	3,78	4	3,90

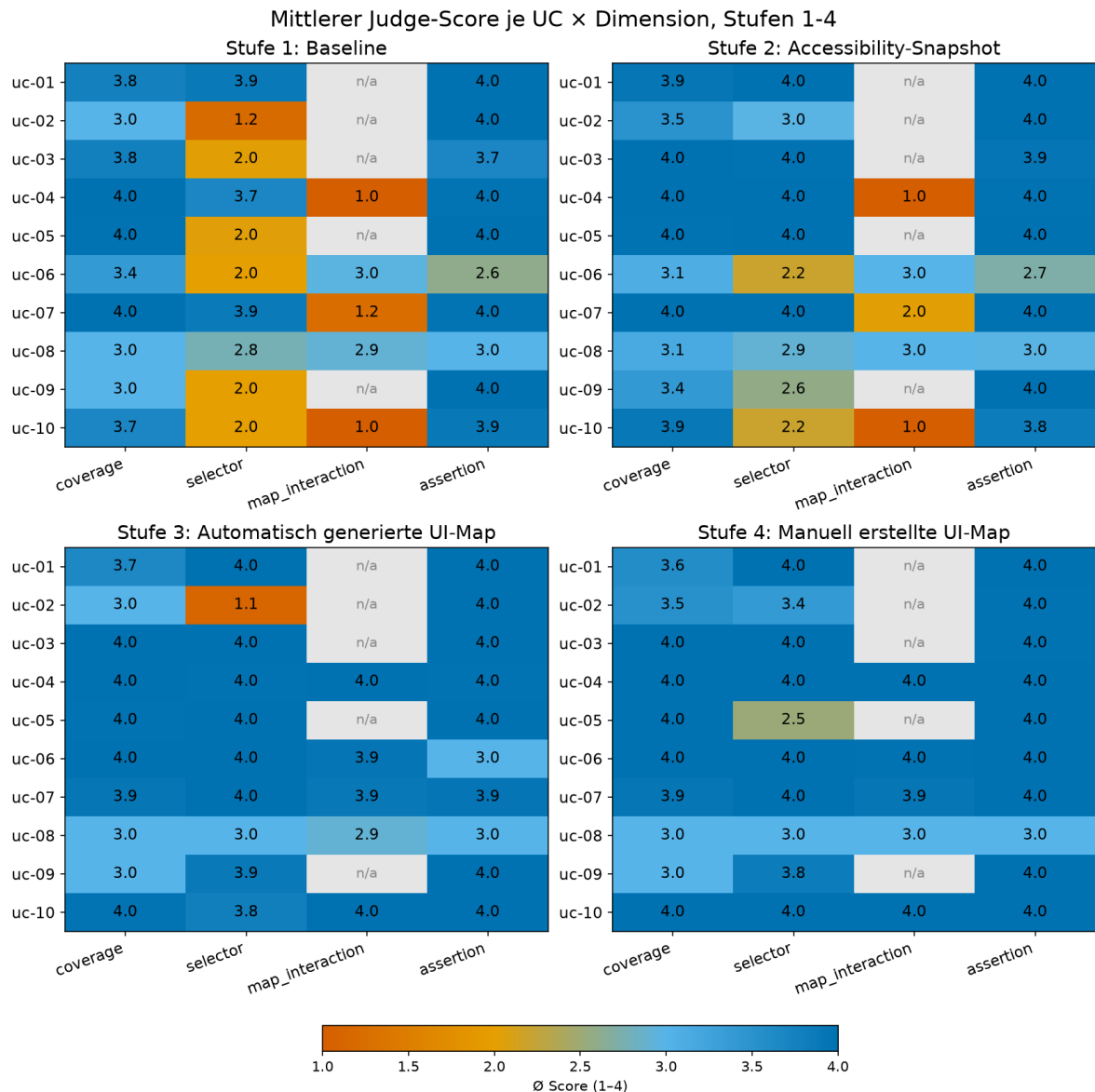


Abbildung 12: Mittlerer Judge-Score je Use Case und Dimension für die Stufen 1 bis 4 (GPT-5.4). Graue Felder kennzeichnen Use Cases ohne Kartenrelevanz, für die *map_interaction* nicht bewertet wird.

C.4: Ergebnisse der Bonus-Stufe 5

Von den 500 Läufen bestehen 495 (99,0%), 4 (0,8%) enden als `ASSERTION_FAIL` und 1 (0,2%) als `INFRA_FAIL`. 299 Läufe (59,8%) bestehen bereits in der initialen Iteration 0 (Abbildung 13). Nach Iteration 1 sind es zusammengerechnet 441 Läufe (88,2%), nach Iteration 3 97,4%. Danach steigt die Rate nur noch langsam. Die 5 Läufe ohne `PASS` verteilen sich auf die drei Use Cases UC-06 (2 Läufe), UC-07 (1 Lauf) und UC-10 (2 Läufe).

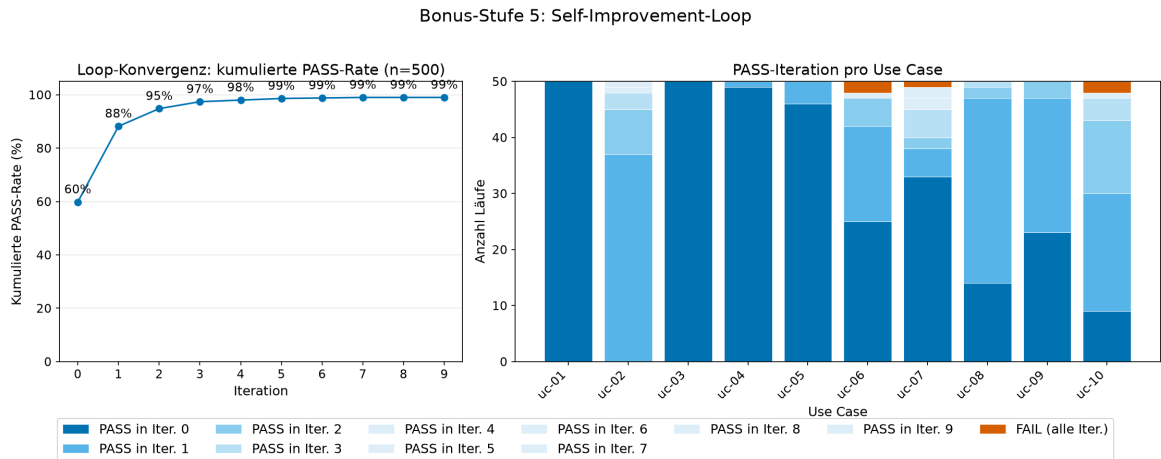


Abbildung 13: Konvergenz des Self-Improvement-Loops der Bonus-Stufe 5 (GPT-5.4). Links die kumulierte PASS-Rate über die Iterationen, rechts die Iteration, in der ein Lauf besteht, je Use Case.

In der Qualitätsbewertung erreicht Stufe 5 bei *selector* (3,87) und *coverage* (3,80) die höchsten Mittelwerte des gesamten Laufs. Bei *assertion* (3,88) und *map_interaction* (3,65) liegt Stufe 5 leicht unter den Werten von Stufe 4 (3,90 bzw. 3,78). Der Median liegt in allen vier Dimensionen bei 4. Über die Use Cases hinweg weicht allein UC-08 deutlich ab, mit 2,64 in *map_interaction* und 3,12 in *coverage* (Abbildung 14). Alle übrigen Use Cases liegen in jeder Dimension bei mindestens 3,30. Als *vacuous_pass* sind 7 der 495 bestandenen Tests eingestuft (1,4%).

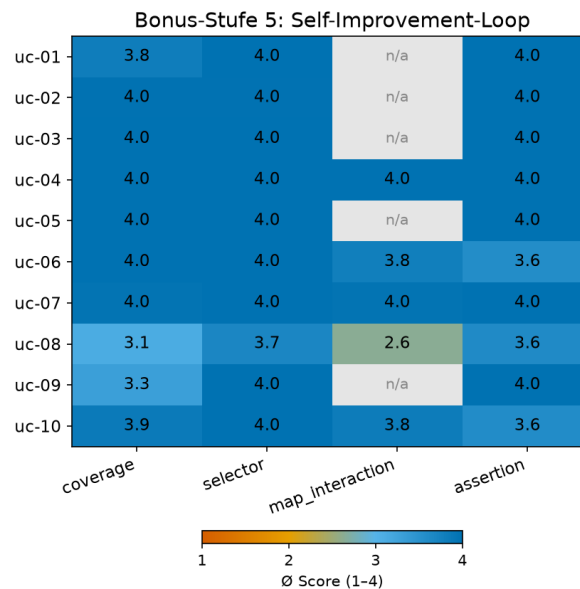


Abbildung 14: Mittlerer Judge-Score je Use Case und Dimension der Bonus-Stufe 5 (GPT-5.4).

C.5: GIS-spezifische Besonderheiten

Die Auswertung basiert auf demselben Vorgehen wie in Abschnitt 6.5. In Stufe 1 verwenden 30 Dateien (6,0 %) mindestens eine Test-ID, die in der Webanwendung nicht existiert. Ab Stufe 2

wird keine einzige Test-ID mehr halluziniert. Der Anteil der Dateien mit `page.locator` und CSS-Ausdrücken sinkt von 52,0 % in Stufe 1 auf 2,2 % in Stufe 2; ab Stufe 3 wird dieses Muster gar nicht mehr verwendet. Die Nutzung der Map-Model-Helfer liegt ab Stufe 3 bei mindestens 90 %. Ein direkter Zugriff auf `__openPioneerMap` bleibt in den Stufen 1 und 2 bei 0 % und liegt ab Stufe 3 zwischen 9,8 und 10,2 %.

Tabelle 11: Anteil der Testdateien je Stufe mit CSS-Lokator, Assertion auf `map-container`, Map-Model-Helfer-Nutzung, direktem Zugriff auf `__openPioneerMap` und halluzinierten Test-IDs (GPT-5.4).

Muster	Stufe 1	Stufe 2	Stufe 3	Stufe 4	Stufe 5
<code>page.locator</code> (CSS)	52,0 %	2,2 %	0,0 %	0,0 %	0,0 %
Assertion auf <code>map-container</code>	0,0 %	66,4 %	49,2 %	78,6 %	51,2 %
Nutzung eines Map-Model-Helfers	0,0 %	0,0 %	90,0 %	90,6 %	92,0 %
direkter Zugriff <code>__openPioneerMap</code>	0,0 %	0,0 %	9,8 %	10,0 %	10,2 %
Datei mit halluzinierter Test-ID	6,0 %	0,0 %	0,0 %	0,0 %	0,0 %

UC-08 erreicht in den Stufen 3 und 4 keinen einzigen `PASS`, obwohl das Zeichnen der Messlinie gelingt. Die betrachteten Dateien berechnen ihre Klickpunkte aus `boundingBox()` und schließen mit genau einem Doppelklick ab. Sie scheitern erst an der Überprüfung des Messergebnisses. 74 % bzw. 66 % der Dateien prüfen das Messergebnis über einen Regex auf Zahl und Einheit `m` oder `km`, wobei viele dieser Prüfungen im Messpanel erfolgen. Allerdings wird das Messergebnis als OpenLayers-Overlay an der gezeichneten Linie angezeigt und nicht im Messpanel. Das erklärt auch die `ASSERTION_FAIL`-Anteile von 100 % bzw. 96 %. In Stufe 2 suchen 31 der 32 erfolgreichen Dateien den Regex seitenweit und treffen damit das Overlay.

Bei UC-07 in Stufe 2 übernehmen 94 % der Dateien die Zielkoordinate aus dem Use Case und rechnen sie über eine eigene Formel in Bildschirmpixel um. Die Elemente werden korrekt adressiert, der Fehler liegt in der Umrechnung auf die Kartenposition. Ab Stufe 3 steigt die `PASS`-Rate auf 56 % und in Stufe 4 auf 78 %. Der Import der Helfer-Funktion `getHighlightedCoordinate` erklärt dies allein nicht. Über die zusätzliche Information zum Zugriff auf `__openPioneerMap` wird die Zielkoordinate über `map.olMap.getPixelFromCoordinate` korrekt in Pixel umgerechnet. Der in Tabelle 11 aufgeführte Anteil von rund 10 % direkter Kartenzugriffe geht damit fast vollständig auf UC-07 zurück.

Die übrigen Auffälligkeiten gehen nicht auf die Bedienung der Karte, sondern auf falsche Annahmen in den Bedienelementen zurück. UC-02 prüft den Hintergrundkartenwechsel in Stufe 2 durchgängig als Combobox mit der Annahme, dahinter stehe ein einfaches `<select>`. Es handelt sich aber um einen Auswahl-Trigger, auf dem weder `selectOption` noch `selectedOptions` funktioniert. In Stufe 3 wird stattdessen eine Radiogruppe

angenommen, was in 82 % der Fälle zu einem nicht gefundenen Element führt und den *selector*-Score auf 1,12 drückt. In Stufe 4 rufen alle 50 Dateien `selectOption` auf, und bestanden haben genau die 19, die zusätzlich einen Klick auf `getByRole('option')` durchführen. Der Einbruch von UC-05 in Stufe 4 geht darauf zurück, dass die Tests den Legendeneintrag über eine exakte Textgleichheit mit „Precipitation“ prüfen. In der Legende steht jedoch „Precipitation (mm)“ mit der Einheit dahinter.

Anhang D: Referenzierte Dateien des Repositories

Die folgende Übersicht listet alle referenzierten Dateien mit ihrem Pfad im Repository <https://github.com/nils-gb156/ba-webgis-llm-e2e>. Grundlage ist der Tag `qwen-v1.0`. Die Dateien des ergänzenden Durchlaufs aus Anhang C liegen unter denselben Pfaden im Tag `gpt-v1.0`.

Tabelle 12: Referenzierte Dateien des Repositories.

Datei	Pfad
<i>Eingaben der Generierung</i>	
<code>use_cases.md</code>	<code>src/app/llm/</code>
<code>SKILL.md</code>	<code>src/app/llm/</code>
<code>generated-ui-map.md</code>	<code>src/app/llm/</code>
<code>manual-ui-map.json</code>	<code>src/app/llm/</code>
<i>Generierung</i>	
<code>generate_tests_stage_1.py</code>	<code>src/app/llm/</code>
<code>generate_tests_stage_2.py</code>	<code>src/app/llm/</code>
<code>generate_tests_stage_3.py</code>	<code>src/app/llm/</code>
<code>generate_tests_stage_4.py</code>	<code>src/app/llm/</code>
<code>generate_tests_stage_5.py</code>	<code>src/app/llm/</code>
<code>generate-ui-map.ts</code>	<code>src/app/llm/</code>
<i>Anwendung und Testbarkeit</i>	
<code>services.ts</code>	<code>src/app/</code>
<code>map-model-helpers.ts</code>	<code>src/app/llm/</code>
<code>failure-snapshot-fixture.ts</code>	<code>src/app/llm/tests/</code>
<i>Evaluation</i>	
<code>run_phase1_eval.py</code>	<code>src/app/llm/</code>
<code>map_stage5_phase1.py</code>	<code>src/app/llm/</code>
<code>phase2_judge_prompt.md</code>	<code>src/app/llm/</code>
<code>phase2_judge_prompt_stage5.md</code>	<code>src/app/llm/</code>
<code>_review_sample.csv</code>	<code>src/app/llm/tests/</code>
<i>Auswertung und Ergebnisse</i>	
<code>phase2_judge_evaluation_prompt.md</code>	<code>src/app/llm/</code>
<code>eval_extract/</code>	<code>src/app/llm/</code>
<code>plot_stage.py</code>	<code>src/app/llm/</code>
<code>plot_all.py</code>	<code>src/app/llm/</code>
<code>eval/</code>	<code>docs/</code>

Literaturverzeichnis

- Alian, Parsa, Nashid, Noor, Shahbandeh, Mobina, Shabani, Taha, Mesbah, Ali* (2024): Feature-Driven End-To-End Test Generation, Version Number: 2, o. O., 2024, URL: <https://arxiv.org/abs/2408.01894>
- Bajammal, Mohammad, Mesbah, Ali* (2018): Web Canvas Testing Through Visual Inference, in: 2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST), 2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST), Vasteras: IEEE, 2018-04, S. 193–203
- Brown, Tom B., Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, Askell, Amanda, Agarwal, Sandhini, Herbert-Voss, Ariel, Krueger, Gretchen, Henighan, Tom, Child, Rewon, Ramesh, Aditya, Ziegler, Daniel M., Wu, Jeffrey, Winter, Clemens, Hesse, Christopher, Chen, Mark, Sigler, Eric, Litwin, Mateusz, Gray, Scott, Chess, Benjamin, Clark, Jack, Berner, Christopher, McCandlish, Sam, Radford, Alec, Sutskever, Ilya, Amodei, Dario* (2020): Language Models are Few-Shot Learners, Version Number: 4, o. O., 2020, URL: <https://arxiv.org/abs/2005.14165>
- Celik, Arda, Mahmoud, Qusay H.* (2025): A Review of Large Language Models for Automated Test Case Generation, in: Machine Learning and Knowledge Extraction, 7 (2025), Nr. 3, S. 97
- Ferreira, Margarida, Viegas, Luís, Faria, João Pascoal, Lima, Bruno* (2025): Acceptance Test Generation with Large Language Models: An Industrial Case Study, in: 2025 IEEE/ACM International Conference on Automation of Software Test (AST), 2025 IEEE/ACM International Conference on Automation of Software Test (AST), Ottawa, ON, Canada: IEEE, 2025-04-28, S. 1–11
- Gou, Boyu, Wang, Ruohan, Zheng, Boyuan, Xie, Yanan, Chang, Cheng, Shu, Yiheng, Sun, Huan, Su, Yu* (2025): Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents, o. O., 2025-06-17, arXiv: 2410.05243[cs.AI], URL: <http://arxiv.org/abs/2410.05243>
- Hou, Xinyi, Zhao, Yanjie, Liu, Yue, Yang, Zhou, Wang, Kailong, Li, Li, Luo, Xiapu, Lo, David, Grundy, John, Wang, Haoyu* (2024): Large Language Models for Software Engineering: A Systematic Literature Review, in: ACM Trans. Softw. Eng. Methodol. 33 (2024), Nr. 8, 220:1–220:79
- Kim, Dong-Kwan* (2026): Towards Adaptive Test Automation: JSON DSLs and LLM Agents for End-to-End Testing, in: International JOURNAL OF CONTENTS, 22 (2026), Nr. 1, S. 77–95

- Mathews, Noble Saji, Nagappan, Meiyappan* (2024): Design choices made by LLM-based test generators prevent them from finding bugs, o. O., 2024-12-18, arXiv: 2412.14137[cs.SE], URL: <http://arxiv.org/abs/2412.14137>
- Olausson, Theo X., Inala, Jeevana Priya, Wang, Chenglong, Gao, Jianfeng, Solar-Lezama, Armando* (2024): Is Self-Repair a Silver Bullet for Code Generation?, o. O., 2024-02-02, arXiv: 2306.09896[cs.CL], URL: <http://arxiv.org/abs/2306.09896>
- Ouyang, Shuyin, Zhang, Jie M., Harman, Mark, Wang, Meng* (2025): An Empirical Study of the Non-determinism of ChatGPT in Code Generation, in: ACM Transactions on Software Engineering and Methodology, 34 (2025), Nr. 2, S. 1–28
- Plein, Laura, Ouédraogo, Wendkûni C., Klein, Jacques, Bissyandé, Tegawendé F.* (2023): Automatic Generation of Test Cases based on Bug Reports: a Feasibility Study with Large Language Models, Version Number: 1, o. O., 2023, URL: <https://arxiv.org/abs/2310.06320>
- Schäfer, Max, Nadi, Sarah, Eghbali, Aryaz, Tip, Frank* (2023): An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation, o. O., 2023-12-11, arXiv: 2302.06527[cs.SE], URL: <http://arxiv.org/abs/2302.06527>
- Sennrich, Rico, Haddow, Barry, Birch, Alexandra* (2016): Neural Machine Translation of Rare Words with Subword Units, o. O., 2016-06-10, arXiv: 1508.07909[cs.CL], URL: <http://arxiv.org/abs/1508.07909>
- Stureborg, Rickard, Alikaniotis, Dimitris, Suhara, Yoshi* (2024): Large Language Models are Inconsistent and Biased Evaluators, o. O., 2024-05-02, arXiv: 2405.01724[cs.CL], URL: <http://arxiv.org/abs/2405.01724>
- Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia* (2017): Attention Is All You Need, Version Number: 7, o. O., 2017, URL: <https://arxiv.org/abs/1706.03762>
- Zhao, Wayne Xin, Zhou, Kun, Li, Junyi, Tang, Tianyi, Wang, Xiaolei, Hou, Yupeng, Min, Yingqian, Zhang, Beichen, Zhang, Junjie, Dong, Zican, Du, Yifan, Yang, Chen, Chen, Yushuo, Chen, Zhipeng, Jiang, Jinhao, Ren, Ruiyang, Li, Yifan, Tang, Xinyu, Liu, Zikang, Liu, Peiyu, Nie, Jian-Yun, Wen, Ji-Rong* (2026): A Survey of Large Language Models, in: Frontiers of Computer Science, 20 (2026), Nr. 12, S. 2012627
- Zheng, Lianmin, Chiang, Wei-Lin, Sheng, Ying, Zhuang, Siyuan, Wu, Zhanghao, Zhuang, Yonghao, Lin, Zi, Li, Zhuohan, Li, Dacheng, Xing, Eric P., Zhang, Hao, Gonzalez, Joseph E., Stoica, Ion* (2023): Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena, o. O., 2023-12-24, arXiv: 2306.05685[cs.CL], URL: <http://arxiv.org/abs/2306.05685>

Internetquellen

- 52°North GmbH, Arne Vogt* (2025): Open Pioneer Trails: An Open-Source Framework for Map-Oriented Web Apps, Blog - 52north, <<https://blog.52north.org/2025/11/10/open-pioneer-trails/>> (2025-11-10) [Zugriff: 2026-06-15]
- 52°North GmbH, Matthes Rieke Managing Director* (2026): Conquering new frontiers in web mapping application development, 52°North Spatial Information Research GmbH, <<https://52north.org/software/software-components/open-pioneer-trails/>> (2026-06-15) [Zugriff: 2026-06-15]
- Anthropic* (2026): Introducing Claude Opus 5, Introducing Claude Opus 5, <<https://www.anthropic.com/news/claude-opus-5>> (2026-07-24) [Zugriff: 2026-08-18]
- Chakra Systems* (2026): Chakra Factory | Chakra UI, <<https://chakra-ui.com/docs/styling/chakra-factory>> (2026-06-18) [Zugriff: 2026-06-18]
- Fowler, Martin* (2012): Test Pyramid, Test Pyramid, <<https://martinfowler.com/bliki/TestPyramid.html>> (2012-05-01) [Zugriff: 2026-06-03]
- Hocevar, Andreas* (2020): Faster, smoother Web maps with new browser features, Maps For HTML Community Group, W3C, <<https://www.w3.org/community/maps4html/2020/04/07/better-web-maps-with-new-browser-features/>> (2020-04-07) [Zugriff: 2026-08-26]
- Hugging Face* (2026): Qwen/Qwen3.6-35B-A3B-FP8 · Hugging Face, Qwen/Qwen3.6-35B-A3B-FP8, <<https://huggingface.co/Qwen/Qwen3.6-35B-A3B-FP8>> (2026-04-22) [Zugriff: 2026-08-18]
- Microsoft* (o. J.): Agents | Playwright, Playwright Test Agents, <<https://playwright.dev/docs/test-agents>> (keine Datumsangabe) [Zugriff: 2026-08-31]
- Microsoft* (o. J.): Assertions | Playwright, Assertions, <<https://playwright.dev/docs/test-assertions>> (keine Datumsangabe) [Zugriff: 2026-08-24]
- Microsoft* (o. J.): Auto-waiting | Playwright, Auto-waiting, <<https://playwright.dev/docs/actionability>> (keine Datumsangabe) [Zugriff: 2026-08-24]
- Microsoft* (o. J.): Evaluating JavaScript | Playwright, Evaluating JavaScript, <<https://playwright.dev/docs/evaluating>> (keine Datumsangabe) [Zugriff: 2026-08-25]
- Microsoft* (o. J.): Fast and reliable end-to-end testing for modern web apps | Playwright, Playwright Dokumentation, <<https://playwright.dev/>> (keine Datumsangabe) [Zugriff: 2026-06-03]
- Microsoft* (o. J.): Locators | Playwright, Locators, <<https://playwright.dev/docs/locators>> (keine Datumsangabe) [Zugriff: 2026-08-24]
- Microsoft* (o. J.): Page | Playwright, Page, <<https://playwright.dev/docs/api/class-page>> (keine Datumsangabe) [Zugriff: 2026-08-24]
- Microsoft* (o. J.): Snapshot testing | Playwright, Snapshot testing, <<https://playwright.dev/docs/aria-snapshots>> (keine Datumsangabe) [Zugriff: 2026-08-25]
- Microsoft* (o. J.): Supported languages | Playwright, Supported languages, <<https://playwright.dev/docs/languages>> (keine Datumsangabe) [Zugriff: 2026-08-24]

- Open Pioneer* (2023a): open-pioneer/trails-core-packages, o. O., 2023-06-04, URL: <https://github.com/open-pioneer/trails-core-packages> [Zugriff: 2026-08-24]
- Open Pioneer* (2023b): open-pioneer/trails-openlayers-base-packages, o. O., 2023-05-05, URL: <https://github.com/open-pioneer/trails-openlayers-base-packages> [Zugriff: 2026-08-24]
- Open Pioneer* (2023c): open-pioneer/trails-starter, o. O., 2023-01-13, URL: <https://github.com/open-pioneer/trails-starter> [Zugriff: 2026-08-24]
- Open Pioneer* (o. J.): @open-pioneer/react-utils, npm, <<https://www.npmjs.com/package/@open-pioneer/react-utils>> (keine Datumsangabe) [Zugriff: 2026-08-24]
- OpenLayers* (2026a): Upgrade Notes, openlayers/openlayers, Repository: openlayers/openlayers; Commit: 5b48463, <<https://github.com/openlayers/openlayers/blob/main/changelog/upgrade-notes.md>> (2026-08-05) [Zugriff: 2026-08-24]
- OpenLayers* (o. J.): Rendering meteorites with Canvas 2D, OpenLayers Workshop, <<https://openlayers.org/workshop/en/webgl/meteorites.html>> (keine Datumsangabe) [Zugriff: 2026-08-26]
- Poenisch, Marie* (2022): Die Testpyramide, Die Testpyramide, <<https://www.openknowledge.de/blog/die-testpyramide>> (2022-08-25) [Zugriff: 2026-06-03]
- Qwen Team, Alibaba* (2026): Qwen3.6-35B-A3B: Agentic Coding Power, Now Open to All, Qwen, Section: blog, <<https://qwen.ai/blog?id=qwen3.6-35b-a3b>> (2026-04-15) [Zugriff: 2026-07-27]